

Watheeq Legal Suite

Legal & Compliance Content — for Legal Counsel Review

Clario360 · Watheeq (internal/lex)

Code-verified · English source for Arabic translation

Confidential — for legal review

Table of Contents

(Right-click → Update Field in Word to populate page numbers.)

1. Introduction & Purpose

1.1 What this document is

This document is the **Legal & Compliance Content** volume of the Watheeq Legal Suite documentation set. Watheeq is the legal-operations and legal-affairs suite of the Clario360 platform: a sovereign, Arabic-first system that manages a legal department's end-to-end work — legal service intake, matters and cases, investigations, consultations, settlements, contract lifecycle management, obligations, legal holds, delegation of authority, and the governance and audit machinery that surrounds all of it.

The purpose of this volume is narrow and deliberate. It is **not** a marketing brochure, a feature list, or a technical architecture reference (those exist as separate deliverables). It is a **legal-and-compliance dossier** written so that a qualified Saudi legal counsel can:

1. Understand the **legal environment** in which Watheeq is deployed and operated inside the Kingdom of Saudi Arabia (KSA);
2. Assess how the platform's **design and controls** align with the principal Saudi statutes and regulations that bear on a legal-department system — chiefly the **Personal Data Protection Law (PDPL)** and its Implementing Regulations, the **Electronic Transactions Law**, records-management and records-retention obligations, and the electronic-invoicing/VAT context administered by the **Zakat, Tax and Customs Authority (ZATCA)**; and
3. Review the **legal terms, assurances and representations** the vendor makes, and identify precisely where the vendor's assurance ends and where the **customer's (controller's) own legal decisions and the authority's own credentialing** begin.

The English text in this volume is the **authoritative source**. It has been written to be precise for a lawyer while remaining readable for a business sponsor. A separate translation into Arabic will be prepared by the client's writer; where a term of art has an established Arabic legal equivalent, that equivalent should govern in the Arabic edition.

1.2 Who this document is for

The primary reader is **legal counsel** — either the client's in-house General Counsel / legal-affairs leadership, or external counsel retained to review the platform before procurement or go-live. Secondary readers are the **Data Protection Officer (or equivalent privacy owner)**, the **compliance and internal-audit functions**, and the **executive sponsor** who must sign off on residual risk.

The document assumes the reader is a legal professional familiar with the Saudi regulatory landscape but **not** necessarily with software architecture. Technical mechanisms are therefore described in plain terms and always tied back to the legal obligation they serve. Where a claim depends on a specific technical control, the control is named so counsel can request independent verification.

1.3 Scope

In scope for this volume:

- The Saudi legal framework relevant to a legal-department platform, and how Watheeq is designed to operate within it (Chapter 2).
- Data protection and privacy posture under the PDPL, including lawful basis, data-subject rights, data minimization, **in-Kingdom data residency**, cross-border transfer restrictions, controller/processor allocation, and breach handling (Chapter 3).
- The platform's compliance-relevant technical controls **as they bear on legal obligations** — for example, application-level field encryption of sensitive contract data, immutable (Write-Once-Read-Many, "WORM") archiving, tamper-evident audit trails, and the governed connectors to Saudi government systems.

Out of scope for this volume (covered elsewhere or expressly reserved):

- The full functional capability catalogue and role/permission matrix (covered in the capability and governance volumes).
- Deep infrastructure, disaster-recovery and business-continuity engineering (delivered by the platform's separate ClarioDR layer, and treated here only where it touches a legal-retention or residency obligation).
- **Legal advice.** This document describes obligations and how the platform is built to support them; it does **not** constitute a legal opinion, and it does not certify compliance. Certification of compliance is a matter for the customer's own counsel and, where applicable, the competent Saudi authority.

1.4 How to read this document — and a word on legal precision

Three reading conventions run throughout this volume, and counsel should hold them in mind:

4. **Vendor assurance vs. customer responsibility.** Compliance with the PDPL and related law is a **shared responsibility**. The vendor builds and represents the **capabilities** of the platform; the customer, acting as **data controller**, makes the **legal decisions** — what personal data to collect, on what lawful basis, for what purpose, how long to retain it, and to whom it may be disclosed. This volume is careful to mark which is which. A capability being present in the software does not, by itself, discharge the controller's legal duty; the controller must **configure and operate** the capability correctly.
5. **"Built" vs. "activated."** Several Saudi-authority integrations — notably **Najiz** (Ministry of Justice courts portal), **Nafath** (national digital identity), and **emdha** (a licensed Trust Service Provider for qualified e-signatures) — are **code-complete and configuration-ready** in the platform but require an **authority-issued credential** to move from a validated sandbox to live production. This is an external credentialing step, not additional software to be built. Wherever this distinction matters legally, it is stated plainly.
6. **No invented citations.** In keeping with responsible drafting, this document **does not fabricate statute article numbers, regulation clause references, or certification identifiers** that cannot be verified from primary sources. Where the precise article or clause matters to counsel's assessment, the obligation is described accurately and the text flags — "[**Counsel to confirm the governing article/clause against the current official text**]" — so that the client's lawyer can pin the citation to the authoritative Arabic instrument in force at the time of review. Saudi data-protection and electronic-transactions instruments have been amended in recent years; the

operative text and its implementing regulations should always be checked against the latest official publication.

Each chapter is self-contained but builds on the last. A reader pressed for time may read the **Arabic executive summary** below and Chapter 3 (Data Protection & Privacy), which carries the most legally material assurances.

1.4a Document map and navigation

This volume is organized so that counsel can move from **context** to **specific assurance** without re-reading. The following map shows what each chapter answers and the legal questions it is meant to help settle:

Chapter	Question it answers	Primary reader	Legal weight
1. Introduction & Purpose	What is this document, who is it for, how should it be read, and where does vendor assurance end?	All	Framing; sets the shared-responsibility convention that governs every later assurance
2. Legal Framework & KSA Jurisdiction	Which Saudi instruments bear on the platform, and how is it designed to operate within them?	Counsel, compliance	Alignment narrative; not a certification
3. Data Protection & Privacy	What is the PDPL posture in detail — residency, security, rights, transfer, breach, roles?	Counsel, DPO	Most legally material; contains the enforced controls

Cross-references within the text use the section number (for example, "see §3.5"). Where a control appears in more than one chapter — residency and WORM immutability appear in both Chapters 2 and 3 — the fuller treatment is in Chapter 3 and the Chapter 2 mention is a framework-level pointer.

1.4b Interpretive conventions and defined terms

To keep the prose precise for a lawyer, the following conventions apply throughout this volume:

- **"The platform"** / **"Watheeq"** means the Watheeq legal suite of the Clario360 platform as described in this documentation set, in the build represented at the date of this document.
- **"The controller"** means the customer entity that determines the purposes and means of processing personal data through the platform (see §3.2).
- **"The vendor"** / **"the operator"** means the supplier of the platform and, in a hosted deployment, the party that processes personal data on the controller's behalf.
- **"Fail-closed"** means that when a control cannot positively confirm that an action is permitted, it **refuses** the action rather than allowing it. Fail-closed behaviour is a conservative, safety-preserving default and appears in several of the platform's most legally material controls (residency, egress, encryption-in-production, signature configuration).
- **"WORM"** means Write-Once-Read-Many: storage that, once written, cannot be altered or deleted within its governing retention constraints.
- **"Built"** / **"code-complete"** / **"configuration-ready"** means the capability exists in the software and requires only configuration (and, for authority integrations, an authority-issued credential)

to operate; it is distinguished from "activated"/"live", which additionally requires that external credential and go-live step.

- "[**Counsel to confirm ...**]" flags a point where a statute article, regulation clause, timeline, threshold, or attestation must be verified by the client's lawyer against the in-force official Saudi text. These flags are intentional and are **not** gaps in the platform; they are the boundary of what a vendor document may responsibly assert about the law.

A short glossary of the recurring Saudi-context terms used in this volume:

Term	Meaning in this document
PDPL	Personal Data Protection Law of the Kingdom (نظام حماية البيانات الشخصية) and its Implementing Regulations
SDAIA	Saudi Data & Artificial Intelligence Authority — the competent supervisory authority for the PDPL
ZATCA	Zakat, Tax and Customs Authority — administers VAT and e-invoicing (Fatoorah)
Najiz (ناجز)	The Ministry of Justice courts/services portal (accessed via the "Takamul" integration channel)
Nafath (نفاذ)	The Kingdom's national digital identity confirmation service
emdha (إمضاء)	A licensed Trust Service Provider (TSP) issuing qualified/advanced electronic signatures backed by PKI certificates
NCDC	National Digital Certification Center — supervises trust-service-provider licensing
TSP	Trust Service Provider — issues qualified electronic-signature certificates
DPA	Data-Processing Agreement between controller and processor
SoD / DoA	Segregation of Duties / Delegation of Authority — the platform's internal-control primitives

1.4c Deployment assumptions underlying this document

The legal assurances in this volume are stated against a set of deployment assumptions. If the customer's actual deployment differs, counsel should re-test the affected assurances:

7. **In-Kingdom deployment.** The platform is deployed against **in-Kingdom** infrastructure and storage, and the in-Kingdom residency flag is set on archive endpoints that hold personal data (§3.5).
8. **Encryption enabled in production.** The deployment runs a production (non-developer) profile, which — by design — cannot start with personal-data field encryption disabled (§3.7).
9. **A data-processing agreement is in place** where the vendor operates the platform on the controller's behalf (§3.2).
10. **Authority integrations are activated only with authority credentials.** Najiz, Nafath and emdha are operated live **only** after the competent authority/TSP issues the required credentials; until then they run in a clearly-flagged sandbox and are never presented as live (§2.3).

11. **The controller has defined a retention schedule** and litigation-hold procedures that the platform's retention and legal-hold controls are configured to enforce (\$2.5).

Where any of these assumptions does not hold, the corresponding assurance in this document should be treated as **conditional** and re-confirmed.

1.5 ملخص تنفيذي (Arabic Executive Summary)

ملاحظة: هذا الملخص التنفيذي مُعدّ لمراجعة المستشار القانوني. النص الإنجليزي في هذا المستند هو المرجع المُعتمد، وهذا الملخص العربي مُقدم للتيسير. عند وجود أي اختلاف في التفسير، يُرجى الرجوع إلى النص الإنجليزي وإلى النصوص النظامية السعودية الرسمية النافذة.

الغرض من المستند

يُعدّ هذا المستند الجزء الخاص بـ المحتوى القانوني والامتثال ضمن حزمة وثائق منصة وثيق للشؤون الغرض منه تمكين المستشار القانوني Clario360 وهي الحزمة القانونية من منصة (WatheeQ) القانونية من فهم البيئة النظامية السعودية التي تعمل ضمنها المنصة، وتقييم مدى مواءمة تصميم المنصة وضوابطها مع الأنظمة السعودية ذات الصلة، ومراجعة التعهّات والإقرارات التي يقدمها المزود، مع تحديد واضح لحدود مسؤولية المزود مقابل مسؤولية العميل بصفته جهة تحكّم في البيانات.

هذا المستند ليس استشارة قانونية ولا شهادة امتثال؛ بل هو وصف دقيق لكيفية بناء المنصة لدعم الالتزامات النظامية، ويبقى إقرار الامتثال من اختصاص مستشار العميل والجهات المختصة في المملكة.

أبرز الضمانات القانونية (بايجان)

12. السيادة على البيانات داخل المملكة: صُمّمت المنصة لتخزين البيانات القانونية داخل المملكة العربية السعودية. تفرض المنصة قائمة سماح صريحة بالمناطق داخل المملكة (مثل الرياض وجدة والدمام الواقعة خارج المملكة. وأي وجهة (me-central-1) والمناطق السيادية المحلية)، مع استبعاد المنطقة قبل كتابة أي بيان (fail-closed) تخزين خارج القائمة تُرفض تلقائياً وبشكل مُغلّق افتراضياً.

13. على مستوى نقطة الاتصال تمنع (egress) قيود نقل البيانات عبر الحدود: تُطبّق سياسة إخراج بيانات تسريب الحقول أو النقل إلى مناطق غير مصرّح بها، مع تسجيل كل محاولة محظورة في سجل التدقيق (دون تسجيل قيم البيانات الحساسة).

14. AES-256 حماية البيانات وتشفيرها: تُشفّر الحقول التعاقدية الحساسة أثناء التخزين باستخدام تشفير على مستوى الحقل، مع خيار حفظ المفتاح خارجياً عبر مخزن مفاتيح مؤسسي. ولا يُسمح GCM للبيانات الإنتاجية بالعمل مع تعطيل تشفير البيانات الشخصية (بفشل التشغيل فشلاً آمناً).

15. وحفظ السجلات: تدعم المنصة أرشفة غير قابلة للتعديل أو (WORM) الأرشفة غير القابلة للتعديل. يمنع (Legal Hold) مع حجز قانوني، (Compliance وGovernance نمطا) الحذف ضمن مدة الحفظ. تجعل أي تعديل قابلاً للكشف (hash-chain) الحذف حتى مع صلاحيات الطوارئ، وسلسلة تجزئة.
16. والتوقيع الإلكتروني (Nafath) التوقيع الإلكتروني والهوية: تُميز المنصة بدقة بين تأكيد الهوية عبر نفاذ. بما يوائم نظام التعاملات الإلكترونية السعودي. وربط، (emdha) المعتمد عبر مزود خدمة ثقة مرخص الجهات الحكومية (ناجز/نفاذ/إمضاء) جاهز فنياً ويُفعّل عند إصدار الجهات المختصة لبيانات الاعتماد.
17. قابلية التدقيق والمساءلة: كل إجراء ذي أثر قانوني مسجّل ضمن سجل تدقيق مقاوم للتلاعب، مع وتفويض الصلاحيات المدعوم بالبنية التحتية للمفاتيح العامة (Segregation of Duties) فصل المهام (PKI).

توزيع المسؤولية

الامتثال مسؤولية مشتركة: يوفّر المزود القدرات والضوابط، بينما يتخذ العميل بصفته جهة التحكم القرارات القانونية (الأساس النظامي للمعالجة، والغرض، ومدد الحفظ، وجهات الإفصاح). ووجود القدرة في البرمجية لا يُغني عن تهيئتها وتشغيلها بشكل صحيح من قبل العميل.

2. Legal Framework & KSA Jurisdiction

2.1 Purpose of this chapter

This chapter sets out the principal elements of the Saudi legal environment that bear on a legal-department platform, and explains — at the level of design intent and enforced control — how Watheeq is built to operate within that environment. It is deliberately framed around **obligations**, not article numbers: the aim is to give counsel an accurate map of *what the platform must respect* and *how it respects it*, leaving the pinning of specific articles to counsel against the current official Arabic texts.

A reader should treat this chapter as a **compliance-alignment narrative**, not a certification. Several of the instruments discussed here (notably the PDPL and the Electronic Transactions Law) have implementing regulations and have been amended; the operative provisions must be checked against the latest official publications by the Saudi Data & Artificial Intelligence Authority (SDAIA), the relevant ministries, and ZATCA. [**Counsel to confirm the in-force versions and their effective dates.**]

2.2 The Saudi legal-governance context

Watheeq is designed as a **sovereign** platform: Arabic-first, deployable inside the Kingdom, and aligned to the assumption that the customer is a Saudi entity (or an entity operating in the Kingdom) whose legal records are governed by Saudi law and must be capable of production before Saudi courts and authorities. Several structural design choices flow directly from this:

- **Language and calendar.** The system is Arabic/RTL by default and uses the **Umm al-Qura Hijri calendar** alongside the Gregorian calendar, with Arabic-Indic numerals and Saudi Riyal (SAR) as the default currency. This is not cosmetic: legal deadlines, court-day calculations and official correspondence in the Kingdom are frequently reckoned against the Hijri calendar and the Saudi working week, and the platform computes SLAs and working-day deadlines against a **KSA working calendar**.
- **In-Kingdom deployment and residency.** The platform is designed to keep legal data inside the Kingdom, with a technical residency control described in Chapter 3.
- **Governance and separation of duties.** The platform builds in enforced **Segregation of Duties (SoD)** and **Delegation of Authority (DoA)**, reflecting the reality that legal and compliance actions in a regulated Saudi enterprise are subject to internal-control and authority-matrix expectations.

2.3 Electronic Transactions Law (نظام التعاملات الإلكترونية)

2.3.1 The obligation

The Saudi **Electronic Transactions Law** and its implementing regulations establish the legal recognition of electronic records, electronic signatures, and electronic authentication in the Kingdom. In broad terms, the law provides that electronic records and signatures are not to be denied legal effect merely because they are electronic, and it distinguishes between ordinary electronic signatures and **certified/qualified electronic signatures** issued through a licensed certification framework. The National Digital Certification Center (NCDC) supervises the licensing of

trust-service providers in the Kingdom. [Counsel to confirm the current article references and the NCDC's present designation/scope.]

The practical consequence for a legal-department platform is that **the evidentiary weight of a signature depends on how it was produced**. A simple click-to-accept or one-time-passcode signature is not equivalent to a **qualified electronic signature** backed by a PKI certificate from a licensed Trust Service Provider (TSP), and an identity confirmation is not itself a signature.

2.3.2 How Watheeq aligns

Watheeq treats these distinctions as first-class and **does not collapse them**:

- **Identity confirmation vs. signature are kept strictly distinct.** The platform models **Nafath** (national digital identity) as an **identity confirmation** — a person has proven who they are — and **emdha** (a licensed TSP) as a **qualified electronic signature** backed by a PKI certificate. The platform never records a Nafath identity confirmation as if it were a signature, and it never records an OTP/basic signature as if it were a qualified TSP signature. Each signature event carries explicit evidence metadata recording the **method** and the **signature kind** (for example, a qualified TSP signature is tagged distinctly from an identity confirmation or a basic OTP signature), so downstream consumers — and, ultimately, a court or auditor — can tell them apart.
- **A legally-robust Saudi e-signature workflow pairs both.** The platform's design is that Nafath confirms the signer's identity and emdha then applies the qualified signature; the two legs are recorded separately and can be combined in a single workflow. This mirrors the way a binding, high-assurance electronic signature is expected to be produced in the Kingdom.
- **Fail-closed configuration.** The emdha signing adapter is built to **fail closed**: a deployment that is misconfigured (missing TSP credentials) will not silently degrade to sending an **unsigned** document as though it were signed. Where the live TSP tenant is not yet provisioned, a clearly-flagged sandbox path is used for demonstration and testing only, and is explicitly marked in the evidence so it can never be mistaken for a live qualified signature.

Assurance boundary. The **evidentiary sufficiency** of any given signature for a particular transaction is a legal question for counsel and depends on the transaction type and the authority's requirements. Watheeq provides the **mechanism and the honest evidentiary record**; it does not, and cannot, opine that a given signature is legally sufficient for a given purpose. Activation of the live emdha TSP path additionally requires an **NCDC/emdha-issued integrator credential** — an external credentialing step.

2.4 Personal Data Protection Law (PDPL) and its Implementing Regulations

2.4.1 The obligation

The Saudi **Personal Data Protection Law (PDPL)** — نظام حماية البيانات الشخصية — together with its **Implementing Regulations** and the regulations on **data transfer outside the Kingdom**, is the central data-protection instrument governing the platform. SDAIA is the competent supervisory authority. At

a high level, the PDPL imposes obligations that will be familiar in shape to any modern data-protection regime, including:

- a requirement for a **lawful basis** to process personal data;
- **purpose limitation** and **data minimization**;
- **data-subject rights** (such as the right to be informed, to access, to request correction, and to request destruction, subject to statutory conditions and exceptions);
- constraints on **cross-border transfer** of personal data outside the Kingdom;
- **security** obligations to protect personal data; and
- **breach notification** duties to the authority and, in defined circumstances, to affected data subjects.

[Counsel to confirm the current article numbers, the precise scope of each data-subject right and its exceptions, the transfer-mechanism conditions, and the breach-notification thresholds and timelines against the in-force PDPL and its Implementing Regulations. These have been amended and should not be quoted from memory.]

Chapter 3 addresses the PDPL posture in detail. This section situates it within the framework.

2.4.2 How Watheeq aligns (framework level)

Watheeq is built so that a customer acting as **data controller** can operate it in a PDPL-aligned way. The principal framework-level alignments — each expanded in Chapter 3 — are:

- **In-Kingdom data residency** is enforced by an explicit region allow-list, with out-of-Kingdom destinations refused fail-closed *before* any data is written.
- **Cross-border egress** is constrained by a per-connector data-residency and field-egress policy, with blocked attempts audited.
- **Security of processing** is supported by application-level **AES-256-GCM** field encryption of sensitive contract data, with an option to hold the key in **external custody** (KMS/Vault), and with production environments prevented from running with personal-data encryption disabled.
- **Accountability** is supported by a **tamper-evident, hash-chained audit trail**, enforced separation of duties, and PKI-backed delegation of authority.
- **Immutability for records that must not be altered** is supported by WORM archiving with legal-hold.

Assurance boundary. The controller remains responsible for the *legal* determinations — lawful basis, purpose, retention periods, disclosure decisions, data-subject-request adjudication, and the maintenance of records of processing. The platform supplies the controls; it does not make these determinations.

2.5 Records management and records-retention obligations

2.5.1 The obligation

Saudi entities — particularly regulated ones and government-adjacent bodies — are subject to **records-management and records-retention** expectations arising from several sources: sector regulators, the commercial and companies framework (which requires certain corporate and

commercial records to be kept for defined periods), litigation and evidentiary needs, and the PDPL's own storage-limitation principle (personal data should not be kept longer than necessary for its purpose). Legal records in particular — contracts, case files, official correspondence, and evidence — frequently carry **long or open-ended** retention needs, and must be **producible in an unaltered form** if their integrity is challenged. **[Counsel to confirm the specific retention periods applicable to the client's sector and record types.]**

Two obligations pull in opposite directions and must both be honoured:

18. **Retain** certain records for the required period, in a form whose integrity can be demonstrated.
19. **Do not retain** personal data beyond the period justified by its purpose (storage limitation under the PDPL).

2.5.2 How Watheeq aligns

- **Immutable archiving (WORM).** For records that must be preserved unaltered, the platform supports an **e-archiving connector backed by S3 object-lock**, applying one of two retention postures:
- **Governance mode** — the archived object is immutable to ordinary credentials and can be reclaimed only by a specially-privileged "break-glass" holder (giving ransomware and accidental-deletion protection).
- **Compliance mode** — the archived object is immutable to **everyone**, including a break-glass holder, until the retention period elapses. This is the records-management default for legal archives and is the platform's default when the mode is unspecified.

A configurable **default retention window** applies (a long records-management window by default), and an explicit retain-until can be supplied per object.

- **Legal hold as the floor.** Independently of retention mode, an **object-lock legal hold** can be placed on an archived record. A legal-held object **cannot be removed even with a governance bypass** — it is the ransomware-safe and litigation-hold floor that survives any retention shortcut. The platform also maintains a separate application-level **legal-hold** capability over records, so that data under active litigation or investigation is protected from routine deletion.
- **Tamper-evidence.** Each archived document version contributes an entry to a **tamper-evident manifest** whose content hash is chained from the previous entry (a hash-chain). A single altered byte breaks the chain, so an auditor can prove the archived corpus has not been silently mutated. The same hash-chain discipline is used across the platform's audit surfaces.
- **Storage limitation, respected.** Because the retention posture and window are **explicit and configurable**, the controller can set retention to match the legal requirement for each record type — long where the law requires long, and no longer than necessary where storage limitation applies. Setting the **correct** period for each record type is a controller decision. **[Counsel to define the retention schedule.]**

Assurance boundary. WORM immutability is a genuine technical guarantee (a Compliance-mode object within retention cannot be shortened or deleted, and the storage layer independently refuses the operation). The **choice of retention period and mode**, and the **placement and release of legal**

holds, are controller decisions with legal consequences and must be governed by the customer's own retention schedule and litigation-hold procedures.

2.6 ZATCA e-invoicing and VAT context

2.6.1 The obligation

The **Zakat, Tax and Customs Authority (ZATCA)** administers Value Added Tax (VAT) and the Kingdom's **electronic invoicing (Fatoorah)** regime, which mandates the issuance, format, and (in its integration phase) the clearance/reporting of electronic invoices for in-scope taxable persons, along with retention of tax records. **[Counsel and the client's tax function to confirm the in-force e-invoicing requirements, formats, and record-retention periods.]**

It is important to be precise about scope here, to avoid overstating the platform's role.

2.6.2 How Watheeq relates to it — and where it does not

Watheeq is a **legal-affairs and legal-operations platform**, not an accounting, billing or tax-compliance system. It does **not** issue tax invoices, does not perform VAT calculation or clearance, and is **not** itself a ZATCA e-invoicing solution. The ZATCA context is nevertheless relevant to a legal department in three limited, honest ways:

20. **Contracts carry monetary and tax-relevant terms.** Legal records managed in Watheeq (contracts, settlements, obligations) frequently reference amounts, VAT treatment, and payment terms. The platform stores these as legal-record content; it does not compute tax from them.
21. **Records-retention overlap.** Where a document is **both** a legal record and a tax record, the **longest applicable retention period** should govern, and the WORM archiving described above can be used to preserve it immutably for the required period. The controller/tax function must set the period.
22. **Integration, if required, is via the governed connector framework.** If the customer wishes to connect Watheeq to a ZATCA-compliant e-invoicing system or an ERP, that is done through the platform's **governed integration framework** (maker-checker onboarding, per-endpoint egress policy, health monitoring, dead-letter recovery). No such tax-authority connector is asserted as a built-in feature of this release; it would be a customer-scoped integration.

Assurance boundary. The platform makes **no representation** of ZATCA e-invoicing compliance and should not be relied upon as a VAT or e-invoicing control. Tax compliance remains with the customer's finance/tax function and its dedicated tax systems.

2.7 Sharia and governing-law considerations

2.7.1 The context

The Kingdom's legal system is founded on Islamic Sharia, supplemented by codified regulations (أنظمة). For a legal-operations platform this has two practical implications that counsel should note:

23. **Substantive legal outcomes are not automated.** Watheeq organizes, routes, records and safeguards legal work; it does **not** adjudicate legal questions, render Sharia rulings, or substitute

for the professional judgment of qualified Saudi lawyers and the competent courts. Any AI-assisted features (for example, clause analysis or drafting assistance) are **decision-support** tools that surface information for a qualified human to review and are deployment-gated; they do not produce binding legal conclusions.

24. **Governing law and dispute resolution of the platform contract itself.** The contract between the vendor and the customer for the supply of the platform will have its own governing-law and dispute-resolution clauses. For a Saudi customer these are ordinarily expected to be **Saudi law** with resolution before the competent Saudi forum. **[Counsel to confirm and settle the governing-law, jurisdiction, and dispute-resolution provisions of the platform agreement itself — these are commercial-contract terms addressed in the agreement, not in the platform.]**

2.7.2 How WatheeQ aligns

- The platform's **Hijri-native calendaring, Arabic-first presentation, and KSA working-calendar** deadline logic reflect the Saudi legal-practice context.
- The **governance model** (SoD, DoA, audit trail, approvals with quorum and distinct-approver rules) is designed to reflect the internal-control and authority-matrix expectations of a regulated Saudi enterprise.
- The platform's **honest evidentiary record** (distinct identity-vs-signature evidence, tamper-evident audit, WORM archiving) is designed to produce records that a Saudi court or authority can rely upon as to *what the system recorded and when*, without overstating what that record proves as a matter of substantive law.

3. Data Protection & Privacy

3.1 Purpose and posture

This chapter states the platform's **PDPL compliance posture** in the detail counsel will most want, and marks precisely the boundary between the platform's *capabilities* and the controller's *legal decisions*. The Saudi PDPL, its Implementing Regulations, and the data-transfer regulations administered under SDAIA are the governing instruments; the article-level specifics must be confirmed by counsel against the in-force texts, which have been amended. **[Counsel to confirm article references, the scope and exceptions of each right, transfer conditions, and breach thresholds/timelines.]**

The platform's overall posture can be summarized in one sentence: **WatheeQ is built to let a Saudi data controller process legal-department personal data inside the Kingdom, on a lawful basis the controller selects, with strong security, minimization and residency controls, and with an auditable trail — while leaving the legal determinations to the controller.**

3.2 Controller and processor roles

Allocating roles correctly is the first PDPL question, because the obligations differ.

Role	Ordinarily held by	Principal responsibilities
Data controller	The customer (the Saudi entity deploying WatheeQ for its legal department)	Determines the purposes and means of processing; selects the lawful basis; sets retention periods; adjudicates data-subject requests; maintains records of processing; decides disclosures; owns breach-notification decisions to the authority/data subjects.
Data processor	The vendor / operator of the platform (to the extent it processes personal data on the controller's behalf — e.g. in a hosted deployment)	Processes only on documented instructions; implements appropriate security; assists the controller with data-subject requests and breach handling; does not process for its own purposes; subject to a processing agreement.
Self-hosted / sovereign deployment	The customer operates the platform in its own in-Kingdom environment	In a fully self-hosted deployment the customer may act as both controller and operator, reducing the processor surface.

Assurance boundary and action for counsel. The exact controller/processor allocation depends on the **deployment model** chosen (self-hosted vs. vendor-operated) and must be fixed in a **data-processing agreement (DPA)** between the parties, addressing instructions, security, sub-processors, cross-border transfer, assistance duties, breach cooperation, and return/deletion on termination. **[Counsel to negotiate and confirm the DPA; the platform's technical controls support, but do not substitute for, that agreement.]**

3.3 Lawful basis

The PDPL requires a lawful basis to process personal data. For a legal department the bases typically relied upon include **consent**, **performance of a contract**, **compliance with a legal obligation**, **protection of vital interests**, and the pursuit of **legitimate interests** where permitted, as well as processing necessary for **actual/anticipated legal claims** — a basis of obvious relevance to a legal-affairs system. **[Counsel to confirm the enumerated bases and their conditions under the in-force PDPL, including any special conditions for sensitive data.]**

How Watheeq supports this. The platform does not choose a lawful basis on the controller's behalf; that is a legal determination. It supports the controller by:

- keeping personal data organized by **matter, case, contract and legal service**, so processing can be tied to an identifiable purpose (which underpins purpose limitation and the demonstration of a basis);
- recording actions in an **audit trail**, supporting the accountability the controller must be able to demonstrate; and
- enabling **legal holds**, so that data needed for actual or anticipated legal claims is protected from routine deletion for the period the claim basis justifies.

Assurance boundary. Selecting, documenting, and — where consent is the basis — capturing and evidencing valid consent, remains the controller's responsibility.

3.4 Data minimization and purpose limitation

The obligation. Personal data must be adequate, relevant and limited to what is necessary for the purpose, and must not be processed for incompatible purposes.

How Watheeq supports this.

- **Field-egress allow-list.** For outbound integrations, each connector endpoint can declare an ``allowed_egress_fields`` allow-list — the specific field names permitted to leave the platform boundary. Any field outside the allow-list is **denied** before a connection is opened, and the denial is **audited** (recording the field **names** and destination region only — never the field **values** and never secrets). This is a concrete data-minimization control at the point of egress.
- **Purpose-scoped organization.** Data is structured around legal purposes (matters, cases, contracts), which supports keeping processing within its declared purpose.
- **Minimized audit content.** The platform's residency/egress and audit surfaces are deliberately built to log **field names and metadata only, not values**, so that the compliance machinery does not itself become an over-collection of personal data.

Assurance boundary. Deciding which fields are necessary (and therefore which belong on the egress allow-list) is a controller configuration and legal decision.

3.5 In-Kingdom data residency — the enforced region allow-list

This is one of the most legally material controls in the platform, and it is enforced in code, not merely asserted in policy.

The obligation. The PDPL and its data-transfer regulations restrict the transfer and storage of personal data outside the Kingdom except under defined conditions. For legal records of a Saudi entity, keeping data **in-Kingdom** is frequently the required or strongly-preferred posture.

How WatheeQ enforces it. The e-archiving WORM backend enforces an **explicit in-Kingdom region allow-list**. When the "in-Kingdom only" flag is set for an archive endpoint, the platform resolves the storage bucket's region and **refuses any region not on the allow-list — fail-closed, before any object is ever written**. The allow-list is intentionally small and explicit; an unrecognized or empty region is treated as **not** in-Kingdom and is refused.

The allow-list of regions treated as in-Kingdom includes sovereign/local Saudi labels and the principal Saudi cities, for example:

Region label	Treated as	Note
ksa-central	In-Kingdom	Sovereign / local S3 (e.g. local operator regions)
sa-riyadh-1, riyadh	In-Kingdom	Riyadh
sa-jeddah-1, jeddah	In-Kingdom	Jeddah
sa-east-1	In-Kingdom	Saudi East
dammam	In-Kingdom	Dammam
in-kingdom	In-Kingdom	Explicit operator assertion for on-prem/sovereign object store
`me-central-1`	EXCLUDED	AWS Middle East (UAE) is NOT in-Kingdom and is explicitly removed from the allow-list so it fails closed

The exclusion of **`me-central-1`** deserves emphasis for counsel: this region is the AWS Middle East (UAE) region — geographically close but **outside the Kingdom**. The platform is deliberately coded to **exclude** it from the in-Kingdom allow-list, so that a well-intentioned but mistaken configuration pointing at UAE storage is **refused**, not silently accepted. Region matching is case-insensitive and an empty region is never treated as in-Kingdom.

Assurance boundary. The residency control is a genuine, enforced technical guarantee for archive placement when the in-Kingdom flag is set. The controller must (a) **set the flag**, (b) **deploy against in-Kingdom storage**, and (c) confirm that the underlying storage provider's region genuinely corresponds to physical in-Kingdom data centres. **[Counsel/infrastructure to obtain the provider's contractual and, where available, attested data-location assurance for the chosen storage.]**

How the guarantee actually surfaces (for counsel's independent verification). Because residency is a control counsel may wish to have independently tested, it is worth stating precisely **when** the refusal happens so it can be demonstrated in an acceptance test:

25. When an archive endpoint is **connected or tested**, the platform resolves the target bucket's region — either from the operator-declared configuration or from the storage provider's live location report.
26. It checks that resolved region against the in-Kingdom allow-list (case-insensitive; an empty region is treated as **not** in-Kingdom).
27. If the "in-Kingdom only" flag is set and the region is **not** on the allow-list, the connection is **refused with a dedicated residency error** — **before any object is written**. There is no path by

which a document is first archived and then found to be out-of-Kingdom; the check precedes the write.

28. The archive-write posture and the residency posture are independent: even a correctly in-Kingdom archive still applies its WORM retention and (where set) legal hold.

This design gives counsel a **testable assurance**: an acceptance test that points an in-Kingdom-flagged archive endpoint at a me-central-1 (UAE) bucket should be **rejected**, and no object should appear in that bucket. A residency breach is therefore something the platform is built to **prevent and evidence**, not merely to detect after the fact.

3.6 Cross-border transfer restrictions

The obligation. Transfer of personal data outside the Kingdom is restricted under the PDPL's data-transfer regulations and permitted only under defined conditions and safeguards. **[Counsel to confirm the current transfer conditions, any approval/adequacy mechanisms, and exceptions.]**

How Watheeq enforces it. Beyond the archive residency control above, the platform applies a **per-endpoint egress policy** to **any** outbound connector call:

- **`allowed\_regions`** — the destination regions an endpoint may egress to (for example, restricting to in-Kingdom). If set, a call to a region outside the list is **denied** and **audited** (egress.blocked) before the connector opens a connection.
- Combined with the **`allowed\_egress\_fields`** minimization control (§3.4), this gives the controller a two-dimensional gate: **where** data may go and **which** fields may leave.
- Denied attempts are recorded as secret-free audit events and surfaced as a running "blocked" tally in the administration console, so a residency or minimization breach is **stopped and evidenced**, not merely logged after the fact.

Assurance boundary. The egress policy is a **configurable** control; region enforcement is opt-in per endpoint. The controller must set `allowed_regions` (and `allowed_egress_fields`) on each endpoint that could transmit personal data, and must ensure any **permitted** cross-border transfer independently satisfies the PDPL's transfer conditions. The platform enforces the configured boundary; it does not itself certify that a permitted transfer meets the statutory transfer conditions.

3.7 Security of processing — encryption and integrity

The obligation. The PDPL requires appropriate technical and organizational measures to protect personal data against unauthorized access, alteration, disclosure, or loss.

How Watheeq supports this.

- **Field-level encryption at rest (AES-256-GCM).** Sensitive contract fields are encrypted at rest using **AES-256-GCM** authenticated encryption, with each encrypted value carrying a versioned ciphertext prefix (enc:v1:). Authenticated encryption means tampering with a ciphertext (or using the wrong key) is **detected** on decryption rather than silently yielding corrupt plaintext.
- **Key custody options.**
- **Software custody** (default) holds a real 32-byte AES-256 key in process, supplied by configuration.

- **External custody** sources the key at startup from an externally-mounted secret — for example a **KMS/Vault**-backed store surfaced by an external-secrets/CSI driver — so the key material lives **out of process**. The same real AES-256-GCM is performed either way; only the key custody changes.
- The key length is strictly enforced at exactly 32 bytes (there is no silent truncation or padding), so a misconfigured key is rejected rather than weakly accepted.
- **Production cannot run with personal-data encryption disabled**. A deliberate **fail-fast guard** prevents a non-developer (production) profile from starting with personal-data field encryption turned off. A production deployment that is misconfigured to disable encryption **fails to start** rather than silently running plaintext. The encryption default in current builds is "software" (real AES-256-GCM), not "off".
- **Backward-compatible read**. Values written before encryption was enabled (legacy plaintext, lacking the ciphertext prefix) are still readable, so enabling encryption does not break access to historical records — a migration-safety property, not a weakening of new writes.
- **Integrity and non-repudiation**. A **tamper-evident, hash-chained audit trail** records legally-significant actions; the archive manifest hash-chain (Chapter 2) protects archived-record integrity; and the **identity-vs-signature evidentiary distinction** (Chapter 2) protects the integrity of signature records.

Assurance boundary. For the strongest custody posture, the controller should deploy **external (KMS/Vault) key custody** and manage key rotation and access under its own key-management policy. Production KMS-region attestation and key-ceremony assurance are **infrastructure and audit gates** the customer's operations and security functions must complete; they are outside the application code. **[Security/infrastructure to confirm the key-management and attestation controls in the target environment.]**

3.7a Sensitive personal data and government-identity data

The obligation. The PDPL affords heightened protection to **sensitive personal data** and to certain identifiers. A legal department's records routinely contain data of exactly this kind: national identity numbers (national ID / *iqama*), litigation and investigation details, health or financial information appearing in case files, and correspondence that may reveal legally-privileged or reputationally-sensitive matters. Processing such data attracts stricter conditions. **[Counsel to confirm the categories the in-force PDPL classifies as sensitive, and the additional conditions applying to each.]**

How WatheeQ supports this.

- **National-identity data is handled through governed channels, not ad-hoc collection.** Identity confirmation is performed via **Nafath** (an identity-confirmation service), and where identity data must leave the boundary for a legitimate purpose, the **field-egress allow-list** (§3.4) governs precisely which identifier fields may egress and to which region — with everything else blocked and audited by field name only.
- **Encryption of sensitive contract fields.** Sensitive fields in contract records are encrypted at rest with AES-256-GCM (§3.7), reducing exposure in the event of a storage compromise.

- **Identity vs. signature separation protects evidentiary integrity.** Because the platform never records an identity confirmation as though it were a signature (§2.3), sensitive identity events are not silently repurposed as legal acts.
- **Least-privilege access.** Enforced Segregation of Duties and role-scoped access mean sensitive records are reachable only by users whose role grants it, and privileged actions (approve/close/release-hold) are separated from authoring actions.

Assurance boundary. Classifying which data in a matter is sensitive, and applying the heightened conditions (including any consent or authority-approval conditions the PDPL attaches to sensitive processing), is a **controller determination**. The platform supplies governed channels, encryption, egress control and least-privilege access; it does not classify the controller's data on its behalf.

3.8 Data-subject rights

The obligation. The PDPL grants data subjects rights that ordinarily include being **informed**, obtaining **access** to their personal data, requesting **correction**, and requesting **destruction/erasure**, subject to statutory conditions and exceptions. [**Counsel to confirm the exact enumeration, the conditions, and the exceptions — including where a legal-claim, records-retention, or other legal obligation *overrides* an erasure request.**]

How Watheeq supports this.

- **Access and correction.** Because personal data is organized by matter, case, contract and legal service and is queryable through the platform, the controller can **locate, review and correct** a data subject's data to respond to access/correction requests.
- **Erasure vs. legal hold and retention — an important tension.** A legal department frequently **cannot** erase data on request, because the data is needed for **actual or anticipated legal claims**, or is subject to a **statutory retention** or **litigation hold**. Watheeq surfaces this tension honestly: records under an active **legal hold** or within a **WORM retention window / Compliance-mode object-lock** are **protected from deletion** — including from a break-glass bypass, in Compliance mode. This is a **feature** for defensibility, but it means an erasure request must be adjudicated by the controller against the legal grounds for continued retention.
- **Auditability of the response.** Actions taken in response to a data-subject request are recorded in the tamper-evident audit trail, supporting the controller's accountability.

Assurance boundary. The platform provides the **means** to locate, correct, retain, and (where lawful) delete. **Adjudicating** each data-subject request — deciding whether an erasure request must be honoured or is overridden by a retention/legal-claim ground, and responding within the statutory timeframe — is a **controller legal decision**. The platform intentionally does not auto-delete records under legal hold or WORM retention.

3.9 Breach handling and notification

The obligation. The PDPL requires the controller to take steps on becoming aware of a personal-data breach, including **notifying the competent authority (SDAIA)** and, in defined circumstances, **affected data subjects**, within the timelines and thresholds set by the regulations. [**Counsel to confirm the**

exact notification thresholds, timelines, and content requirements against the in-force regulations.]

How Watheeq supports this.

- **Detection and evidence.** The **tamper-evident audit trail**, the **archive integrity hash-chain**, and the **egress-blocked audit events** give the controller a factual basis to detect, scope and evidence an incident — for example, to show that a residency/minimization breach was **blocked**, or to reconstruct what was accessed and when.
- **Containment by design.** Fail-closed residency, egress denial, field encryption, and WORM immutability are all **containment** controls: they reduce both the likelihood and the blast radius of a breach (for instance, encrypted-at-rest fields limit the exposure of a storage compromise; Compliance-mode WORM prevents an attacker from destroying evidence).
- **Cooperation surface.** In a vendor-operated deployment, the vendor (as processor) is expected to assist the controller with breach handling under the DPA.

Assurance boundary. The **legal duty to notify** SDAIA and data subjects, the **assessment of notifiability**, and the **content and timing** of any notice are **controller responsibilities**, to be governed by the customer's incident-response and breach-notification procedures and its DPA with the vendor. The platform provides detection, evidence and containment; it does not make the notification decision.

3.10 Summary of the shared-responsibility model

PDPL/legal theme	Platform provides (vendor capability)	Controller decides/operates (customer duty)
Lawful basis	Purpose-scoped organization; audit trail	Selects and documents the basis; captures consent where relied on
Data minimization	allowed_egress_fields allow-list; names-only audit	Configures the allow-list; decides necessary fields
In-Kingdom residency	Fail-closed in-Kingdom region allow-list (UAE me-central-1 excluded)	Sets the flag; deploys against in-Kingdom storage; confirms provider location
Cross-border transfer	Per-endpoint allowed_regions egress gate + audit	Sets regions; ensures any permitted transfer meets statutory conditions
Security of processing	AES-256-GCM field encryption; external key custody; fail-fast production guard; tamper-evident audit	Deploys external key custody; manages rotation; completes attestation gates
Retention & storage limitation	WORM (Governance/Compliance) + configurable retention + legal hold	Sets retention schedule per record type; governs holds
Data-subject rights	Locate/correct/retain/delete-where-lawful; hold-aware protection	Adjudicates each request; responds within statutory time; balances erasure vs. retention
Breach handling	Detection, integrity evidence, containment controls	Assesses notifiability; notifies SDAIA/data subjects; runs incident response
Controller/processor	Technical controls that a DPA can rely on	Negotiates and maintains the DPA; fixes role allocation by deployment model

Closing note for counsel. The platform's data-protection controls are real, enforced, and — in the case of in-Kingdom residency and WORM immutability — verifiable in code and at the storage layer. They are **necessary but not sufficient** for PDPL compliance: compliance is achieved by the controller **operating** those controls correctly, adjudicating the legal questions the platform cannot decide, and formalizing the arrangement in a data-processing agreement. Every point marked [**Counsel to confirm ...**] in this volume is an item the client's lawyer should pin to the in-force official Saudi texts before sign-off.

4. Electronic Signatures — Legal Validity

4.1 Purpose and Scope of This Chapter

This chapter explains, in terms a practising lawyer can act on, how the Watheeq Legal Suite produces electronic signatures that are intended to be **legally valid, admissible, and non-repudiable** under the Kingdom of Saudi Arabia's electronic transactions regime. It describes the two government-facing signing rails Watheeq integrates — **Najiz** (the Ministry of Justice / MOJ portal) and **emdha** (a qualified Trust Service Provider licensed by the National Digital Certification Center, NCDC) — and it explains the legal significance of the distinctions Watheeq draws between a **simple**, an **advanced**, and a **qualified** electronic signature.

It also documents the engineering controls that give a signature its evidentiary force: the **signature evidence / proof envelope**, the **cryptographic verification of provider callbacks (webhooks)**, and the **fail-closed design** that prevents a signature from silently degrading to a weaker, less defensible form than the one the transaction was configured to require.

*Counsel's note. Throughout this chapter, statements about the **effect** of Saudi law (for example, that a compliant electronic signature has the same legal effect as a handwritten signature) describe the design intent and the general framework of the Saudi Electronic Transactions Law and its implementing regulations. **This document deliberately does not cite specific statute article numbers.** Where the exact article, the current NCDC licensing status of a provider, or a sector-specific formality (real-estate, family status, notarised instruments, or matters reserved to a court or notary) is material to a given transaction, **the responsible counsel should confirm the current statutory text and any regulatory circulars before relying on an electronic signature for that transaction type.** The platform is built to **support** a legally valid signature; it is not a substitute for legal judgement about whether a particular instrument may be signed electronically at all.*

4.2 The Legal Foundation: Electronic Signatures in Saudi Law

4.2.1 The functional-equivalence principle

The Saudi Electronic Transactions Law rests on a **functional-equivalence** principle that is common to modern electronic-signature statutes worldwide: an electronic record or an electronic signature is not to be denied legal effect **merely because it is electronic**. Where the statutory conditions are met, an electronic signature carries the **same legal effect as a handwritten (wet-ink) signature**, and an electronic record carries the same effect as a paper record.

Watheeq operationalises this principle at the moment of signing. Every signing experience presented to a signer includes an explicit, bilingual **legal-consent notice** that states, in plain language, that the signer's electronic signature is the legal equivalent of a handwritten signature and that the signer consents to transact electronically. The default notices are built into the platform in both Arabic and English:

- **English (default):** *"By signing electronically, I agree that my electronic signature is the legal equivalent of my handwritten signature and consent to conduct this transaction electronically."*
- **Arabic (default):** *"بتوقيعي إلكترونياً، أوافق على أن توقيعي الإلكتروني يعادل قانونياً توقيعي الخطي، وأوافق على إتمام هذه المعاملة إلكترونياً."*

An envelope may override these defaults with tenant-specific or matter-specific consent wording (the `legal_consent_en` and `legal_consent_ar` fields on the signature envelope), so that a particular contract type can present bespoke consent language. Capturing this **affirmative, recorded consent to transact electronically** is a standard evidentiary building block: it forecloses a later argument that the signer did not agree to be bound electronically.

4.2.2 The three legal pillars every valid e-signature must satisfy

Independently of which rail is used, a defensible electronic signature must establish three things. Watheeq's architecture maps one control (or one integration) to each pillar:

29. **Identity — *who* signed.** The person applying the signature must be reliably identified. In the Saudi context, the strongest available identity anchor for a natural person is **Nafath**, the national single-sign-on and identity-confirmation service operated under the national digital-identity programme. Watheeq treats Nafath as the identity pillar. (Nafath's legal role is the subject of Chapter 5.)
30. **Intent and integrity — *what* was signed and *that it was not altered*.** The signature must be logically bound to the specific document, and any later tampering with the document must be detectable. Watheeq binds a signature to a specific contract or document target, records a cryptographic **evidence hash**, and — for qualified signatures — relies on the TSP's PKI certificate to bind the signer's key to the sealed content.
31. **Non-repudiation — *the signer cannot credibly deny having signed*.** The combination of a confirmed identity, a recorded intent, an integrity-protected document, and an auditable trail of events is what makes the signature **non-repudiable**: the signer cannot plausibly disclaim it after the fact.

The remainder of this chapter shows how each rail (Najiz, emdha) and each supporting control (the proof envelope, webhook verification, the fail-closed design) contributes to these three pillars.

4.3 The Signature "Envelope" — Watheeq's Unit of Signing

Watheeq organises signing around a **signature envelope**. An envelope is the legal and technical container for a single signing ceremony and is the record from which the evidence of signing is later reconstructed. Understanding the envelope is a prerequisite to understanding the evidentiary discussion that follows.

An envelope carries:

- **A target** — the specific thing being signed: either a **contract** (`target_type = contract`, with a `contract_id`) or a **document** (`target_type = document`, with a `document_id`). A signature is never free-floating; it is always bound to an identified legal object.

- **A provider** — the signing rail: native, nafath, najiz, or external (the last being the qualified-TSP rail on which emdha runs).
- **A method** — how the signer authenticates the act: otp (one-time passcode), nafath (national-identity confirmation), certificate (a PKI certificate / qualified signature), or wet_signature (a scanned or in-person handwritten signature captured for completeness).
- **A language** — ar, en, or bilingual, so the signing experience and the consent notice are presented to the signer in a language they can be shown to have understood. This is itself evidentiarily relevant: a signer cannot easily claim not to have understood a consent notice presented in their own language.
- **The consent notices** (legal_consent_en / legal_consent_ar) described in §4.2.1.
- **One or more recipients**, each with a role (signer, approver, or carbon_copy), a signing order, and — importantly — a per-recipient **method and provider**, so different signatories on the same document can sign by different means (for example, one internal approver by OTP, one external counterparty by qualified TSP signature).
- **An evidence hash and evidence metadata** — the cryptographic and descriptive proof discussed in §4.7.
- **A lifecycle status** — draft → sent → viewed → signed (or declined / expired / cancelled), with each transition recorded as an immutable **signature event**.

Every material act against an envelope — creation, sending, viewing, signing, declining, cancellation, and custody recording — produces a **signature event** row that records the actor, the timestamp, the provider status, the IP address and user agent where applicable, and an evidence hash. This event stream is the **audit trail** on which admissibility and non-repudiation ultimately rest (see §4.8).

4.4 The Three Grades of Electronic Signature and Their Evidentiary Weight

Not all electronic signatures are legally equal. Saudi law — consistently with international practice (for example the EU eIDAS tiering that Saudi practice broadly parallels) — recognises that signatures sit on a **spectrum of assurance**, and that the higher the assurance, the greater the evidentiary weight and the harder the signature is to repudiate. Watheeq models this spectrum explicitly so that a matter can be **required** to use a particular grade, and so that the resulting record honestly states which grade was actually achieved.

4.4.1 The three grades

- **Simple electronic signature (basic)**. Any electronic data attached to or logically associated with a record that the signer uses to sign. In Watheeq terms, this is typically the otp method: the signer confirms a one-time passcode delivered to a controlled channel. It proves **possession of a channel** and **intent to sign**, but it is the weakest link to a real-world legal identity.
- **Advanced electronic signature**. A signature that is (a) uniquely linked to the signer, (b) capable of identifying the signer, (c) created using means the signer can maintain under sole control, and (d) linked to the signed data such that any later change is detectable. Watheeq reaches this tier when a strong identity anchor (Nafath, at an adequate assurance level — see Chapter 5) is combined with document-integrity binding.
- **Qualified electronic signature**. An advanced signature that is additionally created by a **qualified signature-creation device** and based on a **qualified certificate** issued by a licensed Trust Service

Provider. This is the tier that most closely approaches — and in many regimes is expressly equated with — a handwritten signature, and it carries the **strongest presumption of validity**. In Watheeq this is the **emdha** rail: emdha is a TSP licensed by the NCDC, and every signature it applies is recorded as `signature_kind = "qualified_tsp"`.

4.4.2 Comparison table — grades, mechanism, and evidentiary weight

Attribute	Simple (basic)	Advanced	Qualified (TSP)
Watheeq method	otp	nafath (identity) + integrity binding	certificate via emdha TSP
Watheeq provider	native	nafath (+ najiz for MOJ delivery)	external (emdha)
Identity assurance	Channel possession only	National ID confirmed via Nafath (LoA-gated)	National ID + PKI-bound qualified certificate
Document-integrity binding	Evidence hash	Evidence hash	Cryptographic seal via qualified certificate
Backed by a licensed CA / TSP	No	No (Nafath confirms identity; it is <i>*not*</i> a CA)	Yes — NCDC-licensed TSP (emdha)
<code>signature_kind` recorded</code>	(basic / native)	identity_confirmed (identity leg)	qualified_tsp
Presumption of validity	Rebuttable, weakest	Rebuttable, stronger	Strongest; closest to handwritten equivalence
Typical use	Internal approvals, low-value consents	Employee/counterparty acts where identity matters	High-value contracts, instruments intended to bind third parties, matters likely to be litigated
Burden of proof on challenge	Falls more heavily on the party relying on it	Shared	Shifts toward the party disputing it

Counsel's note. The precise labels and definitions in the Saudi Electronic Transactions Law and its implementing regulations, and the exact legal consequences attached to each tier (including any category of instrument for which a qualified signature is **mandatory**, or for which electronic signature is **excluded** altogether), should be confirmed against the current statutory text for the specific transaction. The tiering above states the design intent and the widely-recognised functional distinctions; it is not a recitation of statutory definitions.

4.4.3 Why the grade must be recorded, not assumed

The single most important evidentiary property here is that **Watheeq records the grade that was actually achieved, and refuses to overstate it**. A qualified signature and a basic OTP signature are not interchangeable in a dispute. If a system quietly downgraded a required qualified signature to an OTP because the TSP was unavailable — and then labelled the result as "signed" without qualification — it would manufacture a misleading record. Watheeq is expressly built **not** to do this (see §4.9, the fail-closed design). The `signature_kind` and `signature_basis` fields in the evidence metadata exist precisely so that a reader of the record — a judge, an arbitrator, an auditor — can see **which grade of signature actually anchored the transaction**.

4.5 Rail One — Najiz (Ministry of Justice) Integration

4.5.1 What Najiz is and why it matters legally

Najiz is the digital services portal of the Saudi **Ministry of Justice (MOJ)**. Integrating signing with Najiz is significant because it connects a private-sector signing ceremony to the **official justice**

infrastructure of the Kingdom: a signing request routed through the MOJ portal carries a governmental provenance that is directly relevant to how a Saudi court is likely to treat the resulting instrument. In Watheeq, Najiz is the najiz signature provider, and it is the rail contemplated by the Watheeq functional requirement for MOJ e-signature (internally referenced as FR-WATHEEQ-004).

4.5.2 How Watheeq dispatches a signing request to Najiz

When an envelope routed to the najiz provider is sent, Watheeq's Najiz dispatcher performs a tightly-scoped, auditable mapping:

32. **Provider guard.** The dispatcher first confirms the envelope is a Najiz envelope (its provider is najiz, or is unset and therefore defaults to Najiz). An envelope explicitly routed to a **different** provider is **rejected outright** rather than silently sent to the MOJ portal. This prevents a mis-routed envelope from ever reaching the wrong rail.
33. **Field mapping.** The lex envelope is mapped onto a Najiz "signing request" body: the tenant, a stable request reference (the lex envelope ID), the target type and the associated contract/document ID, the title, subject and message, the signing language, the callback URL, the due and expiry timestamps, and one **signatory record per recipient** carrying the recipient's name, contact details, role, sign method, and signing order.
34. **Authentication.** The request is authenticated to Najiz with the **MOJ-issued client credentials** (a client ID and client secret), sent as request headers. These credentials are provisioned per deployment region and are **never bundled with the software** — they are supplied only through environment-gated configuration.
35. **Idempotency.** The lex envelope ID is sent as both the request reference and an **idempotency key**. A retried request after a transient MOJ-side fault therefore re-references the **same** signing request rather than creating a duplicate — an important control for the integrity of the legal record (no phantom second envelope).
36. **Acknowledgement mapping.** Najiz's acknowledgement is translated back into Watheeq's **provider-dispatch proof**: the Najiz-side request ID becomes the `provider_envelope_id`; per-recipient Najiz signatory IDs are stored against each recipient; and the provider status, delivery status, and any provider-supplied **evidence hash** are captured into the envelope's evidence record. If Najiz returns an empty request ID or an invalid signatory reference, the dispatch **fails** — Watheeq will not persist a signing acknowledgement it cannot correlate.

4.5.3 Inbound Najiz callbacks (webhooks)

When a signatory acts within Najiz (signs, declines, etc.), the MOJ portal calls back into Watheeq. That inbound callback is **HMAC-verified before it is trusted** (see §4.8). Only after the callback's cryptographic signature is validated does Watheeq translate it into a signature provider event and update recipient, provider, and custody state. This ordering — **verify first, mutate state second** — is what stops a forged or tampered "the counterparty has signed" callback from ever altering the legal record.

4.6 Rail Two — emdha Qualified Signatures via a Licensed Trust Service Provider

4.6.1 What emdha is, and the crucial distinction from Nafath

emdha (emdha.sa) is a **Trust Service Provider (TSP)** licensed by the **National Digital Certification Center (NCDC)** of Saudi Arabia. Unlike Nafath — which confirms **identity** and is emphatically **not a certificate authority** — emdha **issues a qualified/advanced electronic signature backed by a PKI certificate**. This is the distinction that carries the most legal weight in the entire chapter, and Watheeq keeps the two concepts **strictly separate** in its data model:

- **Nafath** → ``identity_confirmed``. A person proved **who they are**.
- **emdha** → ``signed (TSP)``. A **qualified electronic signature** was actually applied.

A fully-defensible, legally-binding Saudi e-signature workflow typically **pairs both**: Nafath confirms the signer's identity, and **then** emdha applies the qualified signature. Watheeq's emdha dispatcher owns **only the TSP signing leg**. It never collapses identity into signature, and it never treats a Nafath identity confirmation as if it were a signature. This separation is not a technicality — it is exactly the separation a court would want to see, because it lets the record answer **two distinct questions** independently: **"was the signer really who they claimed to be?"** (identity, Nafath) and **"was a qualified signature actually made?"** (signature, emdha).

4.6.2 How the emdha (qualified TSP) signature is produced

When an envelope routed to the external provider is sent through emdha, the dispatcher:

37. **Enforces the correct rail.** It accepts only an envelope whose provider is external (the TSP rail) or unset; an envelope routed elsewhere is rejected, so a mis-routed envelope fails fast rather than being signed on the wrong rail.
38. **Requests a **qualified** signature explicitly.** The signing request sent to emdha carries `signature_level = "qualified"`. Watheeq is not asking emdha for "some signature"; it is asking for a **qualified electronic signature** — the highest tier.
39. **Authenticates with the NCDC/emdha integrator credential.** Live operation requires an emdha/NCDC-issued client ID and client secret plus a signed-callback HMAC secret, all provisioned per deployment and **never bundled with the binary**.
40. **Records the grade in evidence.** Every event emdha produces is recorded as ``method = certificate`` and carries, in the evidence metadata, ``signature_kind = "qualified_tsp"`` and ``signature_basis = "saudi_e_transactions_law"``. This is how a downstream reader can tell a TSP-backed qualified signature apart from a Nafath identity confirmation or a basic OTP signature at a glance.
41. **Correlates the acknowledgement.** emdha's response supplies a request ID (persisted as the `provider_envelope_id`), per-signatory emdha signature IDs, a delivery status, and an evidence hash — all captured into the proof envelope. An empty request ID or an unparseable signatory reference causes the dispatch to **fail** rather than persist an uncorrelatable "signature."

4.6.3 Honest gov-gating — sandbox vs. live

Watheeq is candid, in both its code and its operational grading, about the fact that the emdha rail is **government-gated**: live qualified signing requires real NCDC/emdha onboarding (integrator credentials, an in-Kingdom signing-API base URL, a signed-callback HMAC secret, and a registered callback URL on the emdha tenant). Until those land for a given deployment, the emdha connector **grades itself `not\_configured` / `planned` — never "healthy"** — so an operator is never misled into believing a live qualified-signing capability exists when it does not.

To support demonstrations, UAT, and automated testing **without fabricating live TSP success**, the emdha dispatcher offers an explicit **sandbox mode**. A sandbox dispatch produces a deterministic, locally-computed acknowledgement with **no network call**, and it is unmistakably flagged in the evidence:

- `dispatch_mode = "sandbox_mock"`
- `live = false`
- an explicit note: `"emdha sandbox/UAT acknowledgement — NOT a live qualified TSP signature; awaiting NCDC/emdha onboarding"`

A **live** dispatch, by contrast, records `dispatch_mode = "live"` and `live = true`. **This flag is decisive evidence.** It means the record itself will always disclose whether a given "qualified" signature was a real TSP signature or a sandbox rehearsal. A signature that was produced in sandbox mode can never masquerade in the evidence as a live qualified signature — which is precisely the kind of misrepresentation that would destroy a record's credibility in a dispute.

***Counsel's note.** The evidentiary weight of an emdha qualified signature depends on emdha holding a **current** NCDC licence at the time of signing and on the qualified certificate being valid and unrevoked. Both should be confirmed for high-value or contentious instruments. The platform records that a qualified TSP signature was requested and returned; the **legal status** of the TSP and the certificate at the moment of signing is a matter for verification against the NCDC's registry and the certificate's own validity.*

4.6.4 Najiz and emdha compared — choosing the right rail

Najiz and emdha are not competitors; they play different legal roles, and a mature workflow may use both. The table below summarises how counsel should think about the choice.

Dimension	Najiz (MOJ)	emdha (TSP)
What it is	Ministry of Justice digital services portal	NCDC-licensed Trust Service Provider
Watheeq provider value	najiz	external
What it contributes	Governmental (justice-infrastructure) routing and acknowledgement of a signing request	The qualified electronic signature itself, PKI-certificate-backed
Grade produced	Depends on the recipient method routed through it	Qualified (<code>signature_kind = qualified_tsp</code>)
Is it a CA / TSP?	No — it is an MOJ portal	Yes
Credentials required	MOJ-issued client ID + secret + callback HMAC secret	NCDC/emdha integrator client ID + secret + callback HMAC secret

Gov-gating in Watheeq	Gov-gated; requires real MOJ credentials to go live	Gov-gated; grades not_configured/planned until real credentials land; sandbox mode available for UAT
Best suited to	Instruments where MOJ provenance is legally advantageous	Instruments requiring the strongest, most defensible signature grade

The practical guidance for counsel: use **emdha** where the transaction needs the **strongest evidentiary grade** (a qualified signature approaching handwritten equivalence), and route through **Najiz** where the **governmental provenance of the MOJ** is legally material to how the instrument will be received. Pair either signing rail with **Nafath** to anchor the signer's identity (Chapter 5). Because Watheeq records the provider, the method, the grade, and the live/sandbox flag on every envelope, the resulting record always discloses *which* rail and *which* grade were actually used — there is no ambiguity for a later reader to exploit.

4.6.5 Per-recipient routing and mixed ceremonies

A single envelope may carry several recipients, and — importantly — **each recipient may sign by a different method and provider**. The signature model stores provider and method at both the envelope level and the recipient level. This lets a realistic legal workflow proceed in one coherent ceremony: for example, an **internal approver** may confirm by otp (a basic signature adequate to an internal sign-off), while the **external counterparty** signs by certificate through emdha (a qualified signature adequate to bind a third party). Recipients also carry a **signing order**, so a sequential ceremony (approver first, then external signatory) is expressed natively. Each recipient's act is recorded as its own event with its own evidence, so the record faithfully reflects that different signatories signed to different standards — which is exactly what the evidence should show, rather than falsely flattening everyone to a single grade.

4.7 The Signature Evidence / Proof Envelope

4.7.1 What the proof envelope is

Everything a court, arbitrator, or auditor would need to reconstruct and defend a signature is captured in what Watheeq calls the **evidence / proof envelope** — a structured, provider-agnostic record attached to the signature envelope, its recipients, and each signature event. It is the documentary answer to the question *"prove that this was signed, by whom, when, how, and that nothing was altered."*

The proof envelope is assembled from several layers:

42. **The `provider\_outbound\_proof`** — the record of *dispatch*. When a signing request is sent to a provider, Watheeq records the adapter used, the delivery status, the dispatch mode (e.g. live, sandbox_mock, deterministic_local), the provider envelope ID, the recipient count, and an evidence hash. This proves *that the request was made, to whom, and in what mode*.
43. **The evidence hash** — a cryptographic digest that binds the proof to the specific envelope, provider envelope ID, and recipient set. Because it is a hash, **any later alteration of the underlying facts would change the digest**, making tampering detectable. For provider-backed signatures, a provider-supplied evidence hash (from Najiz or emdha) is captured in preference to, or alongside, the locally-computed one.

44. **The evidence metadata** — the descriptive record: which adapter and portal handled the signature (provider_adapter, provider_portal), the **grade** of signature (signature_kind, e.g. qualified_tsp), the **legal basis** (signature_basis = "saudi_e_transactions_law"), whether the dispatch was live or sandbox (live, dispatch_mode), and the precise dispatch timestamp.
45. **The signature-event trail** — the immutable, time-ordered sequence of created / sent / viewed / signed / declined / cancelled / custody_recorded events, each stamped with the actor, timestamp, provider status, and (for signer actions) IP address and user agent.
46. **Custody evidence** — when the fully-signed artefact is recorded, Watheeq stores a **content hash** of the signed file, an optional **seal hash**, the file identity and size, the provider, the signing time, and retention/custody metadata. This is the chain-of-custody record for the executed document itself, and it is designed to interoperate with WORM (write-once-read-many) retention so the executed instrument cannot be silently altered after the fact.

4.7.2 How the proof envelope supports admissibility

The proof envelope is what turns a signature from an **assertion** into **admissible, weighty evidence**. In a Saudi (or any) evidentiary contest, the party relying on the signature must typically show:

- **Attribution** — that the signature is genuinely that of the named signer. **Supported by:** the provider recipient IDs, the identity anchor (Nafath), and — for qualified signatures — the PKI certificate binding.
- **Integrity** — that the document signed is the document now produced, unaltered. **Supported by:** the evidence hash and the custody content hash; any alteration changes the digest.
- **Reliability of process** — that the signing process was trustworthy. **Supported by:** the event trail, the recorded consent, the language of presentation, the HMAC-verified provider callbacks, and the honest live/sandbox and signature_kind labelling.
- **Time** — when each act occurred. **Supported by:** the UTC timestamps on every event and dispatch.

Because these elements are captured **contemporaneously and automatically** (not reconstructed after a dispute arises), they carry the hallmarks of a **regularly-kept business record** — which materially strengthens their admissibility and their persuasive weight.

4.8 Webhook Verification — Cryptographic Authentication of Provider Callbacks

4.8.1 Why webhook verification is a legal control, not just a security one

Signing is asynchronous: after Watheeq dispatches a signing request, the provider (Najiz or emdha) later calls back to report that a signatory viewed, signed, or declined. That callback is what flips the legal record from "sent" to "signed." **If a callback could be forged, the entire signature record would be worthless** — anyone could POST a fake "the counterparty has signed" event and manufacture a binding contract. Webhook verification is therefore not merely a security nicety; it is the control that protects the **authenticity of the executed record**.

4.8.2 The HMAC-SHA256 mechanism

Every inbound provider callback is authenticated with an **HMAC-SHA256** message-authentication code computed over the **raw callback body** using a **shared secret** that only Watheeq and the provider know (the provider's signed-callback HMAC secret, provisioned per deployment and never logged). The properties that make this a strong control:

- **Verify-before-parse.** The raw body is **never parsed or acted upon before the HMAC is validated**. A forged or tampered callback is rejected before it can touch any recipient, provider, or custody state. This ordering is enforced explicitly in both the shared signature-webhook validator and the dedicated Nafath webhook verifier.
- **Constant-time comparison.** The computed and received signatures are compared using a **constant-time** comparison, which defeats timing side-channel attacks against the secret.
- **Tolerant signature encoding.** The signature header may be bare hex or prefixed (sha256=, hmac-sha256=, or v1=), so Watheeq interoperates with the header conventions of different providers without weakening the check.
- **Optional timestamp binding.** Where the provider signs timestamp.body rather than the body alone, Watheeq honours it — binding the signature to a specific time and helping defeat replay of an old, legitimately-signed callback.
- **Fail-closed on a missing secret.** If validation is requested but **no secret is configured**, the callback is **rejected** (a "webhook secret is required" error). Watheeq will never accept an unauthenticated callback by falling back to "no verification." Likewise a missing signature, an unsupported algorithm, or an empty payload is rejected.

4.8.3 Provider-specific callback handling

- **Najiz.** Inbound MOJ callbacks are translated into a signature provider event, carrying the raw bytes and the HMAC headers straight through to the shared validator, so **the same HMAC-SHA256 verification guards every Najiz callback** before any state change.
- **emdha.** Inbound TSP callbacks are likewise HMAC-verified through the shared validator and are annotated as signature_kind = "qualified_tsp", so a qualified-signature callback stays distinct from an identity confirmation throughout its life.
- **Nafath.** The Nafath identity webhook has its own dedicated verifier (VerifyNafathWebhook) that performs the same HMAC-SHA256-over-raw-body, constant-time check, and additionally enforces the identity **assurance-level gate** (see Chapter 5). Watheeq can even **prove its own inbound loop** in-process: a synthetic, correctly-signed webhook can be generated and run back through the real verifier to demonstrate that an endpoint's secret and signature path are wired correctly — without any external call and without mutating any state.

4.8.4 Idempotent event handling

Beyond authenticity, Watheeq guards against **duplicate** callbacks. A provider event ID is checked against previously-seen events, and a database-level uniqueness constraint on provider event IDs is a backstop against a race. A provider that legitimately retries a callback therefore cannot cause a signature to be **double-recorded** — the legal record reflects each act exactly once.

4.9 The Fail-Closed Design — No Silent Downgrade

4.9.1 The principle

The governing safety principle across Watheeq's signing stack is **fail-closed**: when the system cannot prove that a strong, valid signing basis exists, it **refuses to proceed** rather than quietly substituting a weaker one. This is the single most important property for the **legal defensibility** of the resulting signatures, because the most dangerous failure mode of an e-signature platform is not an outright error — it is a **silent downgrade** that produces a record which **looks** valid but rests on a weaker foundation than the transaction required.

4.9.2 Where fail-closed is enforced

- **Provider construction fails closed on missing credentials.** The Najiz dispatcher will not construct without its MOJ endpoint, client ID, and client secret. The emdha **live** dispatcher will not construct without its endpoint, client ID, and client secret. A misconfigured deployment therefore **cannot silently fall back to a different, weaker transport** — it fails to start that rail at all.
- **Mis-routed envelopes are rejected.** Each rail rejects an envelope routed to a different provider, so a Najiz envelope can never be signed by emdha and vice versa.
- **Uncorrelatable acknowledgements are rejected.** An empty provider request ID or an invalid/empty signatory reference causes the dispatch to fail; Watheeq will not persist a "signature" it cannot tie back to a specific recipient and envelope.
- **Callbacks fail closed.** As in §4.8, a missing secret, missing signature, unsupported algorithm, empty payload, or a failed HMAC all **reject** the callback. There is no "accept unverified" path.
- **Identity assurance fails closed.** The Nafath assurance gate treats an absent or unrecognised assurance level as **not meeting** any positive minimum, and it clamps a misconfigured minimum back up to the secure default rather than down (Chapter 5). A verified-but-weak identity confirmation is preserved in the record but flagged as **not** a valid e-sign basis.
- **Sandbox can never masquerade as live.** The emdha sandbox acknowledgement is permanently labelled `live = false` / `dispatch_mode = "sandbox_mock"`, so a rehearsal signature can never be mistaken in evidence for a live qualified signature.

4.9.3 Why "no silent downgrade" is what makes signatures defensible

Put plainly: **a signature is only as defensible as the weakest step that was silently accepted along the way.** If a platform were willing to accept an unverified callback, or to fall back from a qualified TSP signature to an OTP when the TSP was down, or to treat an unknown identity-assurance level as "good enough," then **every** signature it produced would be tainted by that possibility — a skilled opponent would argue that **this** signature, too, might have taken the weak path. Watheeq forecloses that argument by construction: at each decision point, the choice is **either a provably-strong basis or an explicit, recorded failure** — never a silent, undocumented compromise. That is the property that lets counsel stand behind the signatures Watheeq produces.

4.10 What Makes a Watheeq Signature Admissible and Non-Repudiable — Summary

Legal requirement	Watheeq control(s)
Consent to transact electronically	Bilingual legal-consent notice presented and recorded on every envelope; per-envelope override supported
Attribution (who signed)	Nafath identity confirmation (LoA-gated); provider recipient IDs; PKI certificate binding for qualified signatures
Integrity (what was signed, unaltered)	Evidence hash on envelope/event; custody content hash + seal hash; WORM-friendly custody record
Intent to sign	Explicit recipient signing action; recorded event with actor, IP, user-agent, timestamp
Trustworthy process	HMAC-SHA256 verified provider callbacks (verify-before-parse, constant-time); idempotent event handling; honest grade/live labelling
Correct grade of signature	signature_kind (qualified_tsp for emdha), signature_basis, dispatch_mode/live flags — the record states the grade actually achieved
Non-repudiation	The combination of the above, captured contemporaneously in an immutable event trail, with fail-closed guarantees that no weak step was silently accepted
Governmental provenance (where used)	Najiz (MOJ) routing carries official justice-infrastructure provenance

*Counsel's final note for this chapter. Watheeq is engineered to *produce and preserve* the evidentiary elements of a valid Saudi electronic signature and to *refuse to overstate* what was achieved. Whether a *particular* instrument may lawfully be executed electronically at all — and which grade of signature it requires — remains a question of the applicable statute, regulation, and sector rules for that instrument type. Confirm the current statutory text and the provider's current NCDC licensing status before relying on an electronic signature for any high-value, third-party-binding, or reserved-formality instrument.*

5. Identity Verification (Nafath) — Legal Basis

5.1 Purpose and the Central Distinction

This chapter explains the **legal role of Nafath** in the Watheeq signing architecture and why that role must be understood as **distinct from — and complementary to — the act of signing itself**.

The single most important idea in this chapter is stated in one sentence in Watheeq's own code and is worth restating for counsel: **Nafath confirms identity; it is not a certificate authority**. A "verified" Nafath outcome means the citizen approved an identity challenge — it establishes *who the person is*. It is the **basis for** an identity-confirmed signature; it is **not the signature itself**. The signature (in its qualified form) is applied by the TSP, emdha (Chapter 4). Watheeq keeps these two facts in **separate fields with separate meanings**, and this separation is exactly what a Saudi court or arbitrator needs in order to reason cleanly about a disputed electronic act.

- **Nafath answers:** *'"Is the signer really the person they claim to be?"* (identity)
- **emdha answers:** *'"Was a qualified electronic signature actually applied?"* (signature)

Conflating the two — treating an identity confirmation as though it were a signature — is a common and dangerous error in e-signature systems. Watheeq is built specifically **not** to make it.

5.2 What Nafath Is

Nafath is the Kingdom of Saudi Arabia's **national digital-identity and single-sign-on service**. It allows a person to prove their identity to a relying party by approving an authentication challenge — most commonly an **in-app push with number-matching** (the citizen matches, in the Nafath app, a number displayed by the relying service), or a **biometric / liveness** confirmation. Nafath is operated as part of the national digital-identity programme and is the **authoritative, government-anchored** way to confirm that a person interacting with a service is the holder of a specific Saudi national identity.

Crucially, **Nafath is an identity-confirmation service, not a Trust Service Provider and not a Certificate Authority**. It does not issue signing certificates and it does not, by itself, apply a cryptographic signature to a document. Its output is an **assertion of identity at a certain strength**, which Watheeq then uses as the *identity pillar* of a signing ceremony.

5.3 The Legal Role — Identity Versus Signature

5.3.1 Two separate legal facts, deliberately kept separate

Watheeq models the outputs of the two services as two distinct legal facts:

	Nafath	emdha (TSP)
Legal fact established	The signer's identity is confirmed	A qualified electronic signature was applied
Watheeq state	identity_confirmed	signed (TSP) / signature_kind = qualified_tsp
Role in the act	The basis / precondition for signing	The signature itself
Is it a CA / TSP?	No	Yes (NCDC-licensed)
What it proves	*Who* the person is	*That they signed*, with integrity

	binding
--	---------

A legally-robust workflow **pairs both**: Nafath first confirms **who is about to sign**, and emdha then produces the **qualified signature**. Keeping the two facts separate means the evidence can independently establish each. If, in a later dispute, a party challenges **identity**, the Nafath confirmation answers it; if a party challenges **the signature**, the emdha qualified-signature record answers it. A system that fused them would leave a gap that a skilled advocate could exploit.

5.3.2 Status is not assurance

Watheeq draws a second, subtler distinction that matters legally. A Nafath transaction has a **status** (did the citizen act?) and an **assurance level** (how strong was the way they acted?). These are deliberately independent:

- **Status** answers **"did the citizen approve?"** — normalised to pending, verified, declined, expired, or error.
- **Assurance level (LoA)** answers **"was the approval strong enough to anchor a signature?"**

A transaction can be **verified** (the citizen approved) yet still **fail the assurance gate** if it was approved by a weaker method than the signing basis requires. In Watheeq's words: **status answers "did the citizen approve?"; LoA answers "was the approval strong enough to anchor a signature?"** This separation is what allows Watheeq to accept that **an approval occurred** while still, correctly, **refusing to treat a weak approval as a valid basis for signing**.

5.4 Levels of Assurance (LoA) and Their Legal Significance

5.4.1 The assurance ladder

Not every identity confirmation is equally strong, and the strength of the identity confirmation directly affects the **evidentiary weight** of any signature anchored on it. Watheeq normalises Nafath's authentication contexts onto a five-rung **level-of-assurance (LoA)** ladder:

LoA (Watheeq)	Meaning	Strength	Valid basis for signing?
none	No proven assurance (absent / unrecognised)	0 — weakest	No (fail-closed)
single_factor	Knowledge-only (password / OTP)	1	No — below the e-sign minimum
app_push	In-app push approval without number-match (single tap)	2	No — below the minimum
number_match	In-app push with number-matching	3	Yes — the hard minimum
biometric	In-app biometric / liveness confirmation	4 — strongest	Yes (above the minimum)

5.4.2 The hard minimum and why it exists

Watheeq sets a **hard minimum** assurance level for treating a Nafath confirmation as a signing / delegation-of-authority (DoA) basis: **number_match** — the in-app number-matching approval. The legal rationale is that number-matching defeats the most common real-world attacks on push approval (accidental taps and "push-bombing"/consent-fatigue attacks), because the citizen must **actively read a number from the relying service and match it in the Nafath app**. A single-tap push

(app_push) or a knowledge-only factor (single_factor) does not carry that assurance, and Watheeq therefore **refuses to anchor a signature on them**.

An operator may **raise** the bar (for example, to require biometric) for a particularly sensitive matter. But an operator can **never silently lower it** below number_match: a misconfigured, lower, or unrecognised minimum is **clamped back up** to the secure default. The bar can go up; it cannot be quietly weakened.

5.4.3 Fail-closed assurance enforcement

The assurance gate is **fail-closed** in three specific senses that counsel should note:

47. **Unknown means insufficient.** An absent, empty, or unrecognised assurance level maps to none and satisfies **no positive minimum**. A transaction whose assurance **cannot be proven** is treated as **not strong enough** — never optimistically upgraded.
48. **Verified-but-weak is preserved but flagged.** If a transaction is verified but its assurance is below the minimum, the confirmation is **not discarded** — it is recorded — but it is flagged ``valid_esign_basis = false``. The record honestly says "identity was confirmed, but not strongly enough to sign on." A downstream signing flow **cannot** anchor on it.
49. **The webhook body is verified before it is read.** As with all provider callbacks (Chapter 4), the inbound Nafath webhook is **HMAC-SHA256 verified over the raw body with a constant-time comparison before the body is ever parsed**. A forged or tampered identity callback therefore can **never** mutate identity state. Only after the signature checks out is the status mapped and the assurance gate applied.

5.5 How Nafath Supports Non-Repudiation and KYC-Style Assurance

5.5.1 Non-repudiation

Non-repudiation is the property that a signer **cannot credibly deny** having performed a legal act. Nafath contributes to it directly and powerfully:

- **Authoritative identity anchor.** Because Nafath ties the act to a **specific Saudi national identity** confirmed through the government identity programme, the signer cannot plausibly claim "that wasn't me." The identity is not asserted by the counterparty or by Watheeq — it is confirmed by the national service.
- **Strong-factor confirmation.** Because Watheeq requires at least number-matching (and permits biometric), the signer cannot plausibly claim the approval was an accident or the result of a stolen password. The signer had to actively match a number, or present a biometric, in their own Nafath app.
- **Contemporaneous, verified evidence.** The Nafath confirmation is captured as a verified webhook event with a transaction ID, the confirmed national ID (where the service echoes it), the normalised status, the assurance level, the minimum required, whether the assurance was satisfied, and a UTC receipt time. This is a self-authenticating record of **who was confirmed, how strongly, and when.**

- **Separation from signature.** Because identity and signature are recorded separately, non-repudiation is established on **two independent axes** — the signer can deny neither being identified (Nafath) nor having signed (emdha).

5.5.2 KYC-style assurance for legal acts

For legal acts that carry a **know-your-customer / know-your-counterparty** expectation — onboarding a client, executing a high-value contract, granting or exercising a delegation of authority — Nafath provides government-grade identity assurance at the point of the act itself. Rather than relying on a copy of an ID document collected at some earlier time (which may be stale, forged, or belong to someone else), Watheeq confirms, **at the moment of the legal act**, that the person performing it holds the claimed national identity and confirmed it with a strong factor. This is materially stronger than document-based KYC and is well-suited to the assurance expectations placed on regulated entities and on legally significant instruments.

5.5.3 The complete picture — Nafath + emdha + the proof envelope

Bringing Chapters 4 and 5 together, a fully-assured Watheeq signing ceremony produces a record that answers every question a court will ask:

50. **Who?** Nafath confirmed the signer's national identity at an assurance level of at least number-matching (fail-closed if weaker).
51. **Did they intend to sign, and consent to do so electronically?** The recorded signing action and the bilingual legal-consent notice.
52. **Did they actually sign, and to what standard?** emdha applied a **qualified TSP signature** (`signature_kind = qualified_tsp`), recorded as live (not sandbox).
53. **What did they sign, and has it changed since?** The evidence hash and custody content/seal hashes, WORM-protected.
54. **When did each step happen, and can the callbacks be trusted?** UTC-stamped events, each provider callback HMAC-verified before it was allowed to change any state.

Each answer is captured **contemporaneously and automatically**, each rests on a **fail-closed** control that refuses to accept a weak or unverifiable step, and each is kept **distinct** from the others so it can be independently proven. That is what makes a Watheeq signature — anchored by Nafath identity and completed by an emdha qualified signature — **admissible and non-repudiable** under the Saudi electronic-transactions framework.

***Counsel's note.** Nafath's legal role is to **confirm identity**; it does not, by itself, create a signature or satisfy any statutory formality reserved to a notary or court. The adequacy of Nafath assurance for a **specific** legal act, and whether a given act additionally requires a qualified TSP signature or a reserved formality, should be confirmed against the current statutory and regulatory text for that act type. Where the confirmed national ID is required to be retained or matched against a client record, confirm that the retention and the matching comply with the applicable data-protection (PDPL) obligations covered elsewhere in this document.*

6. Records Management & Retention

6.1 Purpose and Scope of this Chapter

This chapter describes how the Watheeq Legal Suite governs the **lifecycle of legal records** — from the moment a document, contract, matter, investigation, or consultation record is created, through its active working life, into long-term preservation, and finally to its controlled disposition or permanent retention. It is written for legal and compliance readers who must be able to answer three questions with confidence:

55. **Can our legal records be trusted?** — i.e., is a record we produced two, five, or ten years ago provably the same record today, unaltered by anyone including our own administrators?
56. **Can our legal records be preserved when they must be?** — i.e., when litigation, a regulatory investigation, or a preservation duty arises, can we guarantee that relevant records are frozen and survive any attempt (accidental, malicious, or well-intentioned) to delete them?
57. **Can our legal records be disposed of properly when they should be?** — i.e., when a retention obligation has genuinely expired and no preservation duty attaches, can records be destroyed in a controlled, authorised, and evidenced manner, without exposing the organisation to spoliation or over-retention risk?

The Watheeq records-management design answers all three questions through a combination of **Write-Once-Read-Many (WORM) storage**, a **default ten-year retention window**, a **legal-hold mechanism that overrides retention expiry**, a **multi-gate disposition control**, and **in-Kingdom data residency enforcement**. Each of these is implemented in the product and is described below in both operational and legal terms.

Counsel confirmation point.** This chapter describes the **technical controls** the platform provides. The **retention periods, disposition authorities, and preservation triggers** that an organisation adopts are legal and regulatory determinations that must be set by the organisation's counsel and records-management policy owners. Where this chapter states a default (e.g., "ten years"), that default is a configurable starting posture, not a legal opinion that ten years is the correct period for a given record type under Saudi law. **Counsel should confirm the retention schedule per record class.

6.2 The Legal Records Lifecycle

A legal record inside Watheeq passes through a predictable lifecycle. Understanding these stages is essential because the controls that apply — and their legal weight — differ at each stage.

6.2.1 Lifecycle Stages

Stage	Description	Governing control
Creation / Capture	A document version, contract, matter event, or case record is created. A content hash is computed at capture.	Content-hash capture; audit-log entry (Chapter 7)
Active use	The record is edited, reviewed, routed through approvals, and referenced in day-to-day legal work.	Versioning; optimistic concurrency; append-only audit logs

	Each change is versioned and audited.	
Archival (sealing)	A finalised document version is written to WORM archive storage under an object-lock retention window and chained into a tamper-evident manifest.	WORM object-lock; archive manifest hash-chain
Preservation (legal hold)	Where a duty to preserve arises, the record is placed on legal hold, which overrides the ordinary retention clock.	Legal hold (application-layer + object-lock legal-hold)
Retention	The record remains immutably preserved for the configured window (default ten years), immune to deletion.	WORM retention window (RetainUntil)
Disposition	Once retention has genuinely elapsed and no hold is active, the record may be disposed of through a controlled, multi-gate destruction path.	Disposition gates (break-glass + no-hold + retention-elapsed)

The critical legal insight is that **movement between these stages is one-directional and gated**. A record cannot be quietly deleted while it is under retention; it cannot be disposed of while it is under legal hold; and — under the strongest posture — it cannot even be altered by a system administrator until retention has genuinely expired. These properties are not policy promises enforced by a well-behaved application; they are enforced at the **storage layer**, independently of the application, which is what gives them evidentiary and defensive weight.

6.2.2 Records Are Immutable at the Storage Layer, Not Just the Application Layer

A recurring weakness in legal systems is that "immutability" is only skin-deep: the application refuses to offer a delete button, but a database administrator with direct access, or an attacker who has compromised administrative credentials, can still alter or erase the underlying data. Watheeq is deliberately designed so that immutability holds at **two independent layers**:

- **Application layer.** The archiving connector exposes no code path that shortens retention or silently mutates an archived object. Its destructive operation (dispose) is guarded by explicit gates.
- **Storage layer.** The underlying object store enforces WORM object-lock retention and legal-hold independently. Even if the application layer were bypassed entirely, the object store itself refuses a premature deletion.

This is described in the product's own design commentary as **defence-in-depth**: the storage layer independently confirms the floor that the application layer also enforces. For a lawyer, this dual-layer property is the difference between "our software is configured not to delete records" (a weak assurance) and "our records cannot be deleted, and any attempt is refused and evidenced by the storage substrate" (a strong assurance suitable for regulatory and litigation contexts).

6.3 WORM Storage: The Three Postures

The foundation of Watheeq records management is **Write-Once-Read-Many (WORM)** storage, delivered through S3-compatible object-lock (and equivalent CMIS/SharePoint retention seams). The

platform supports three distinct WORM postures — **None**, **Governance**, and **Compliance** — each with materially different legal implications.

6.3.1 The Three Modes at a Glance

Mode	Who can delete/alter within retention	Intended use	Legal posture
None	Anyone with ordinary credentials	Self-serve buckets the operator has <i>*explicitly*</i> opted out of locking	No records-management assurance; not for legal records
Governance	Only a holder of a special break-glass privilege (s3:BypassGovernanceRetention)	Routine records where an authorised, evidenced override must remain possible	Ransomware-safe against ordinary credentials; override remains available to a controlled few
Compliance	No one — not even a break-glass holder — until retention elapses	Legal archives requiring un-bypassable retention	Strongest posture; immutable to <i>*everyone*</i> including administrators

The platform's **default when the mode is unspecified or unrecognised is Compliance** — the safest, records-management-grade posture. This is a deliberate fail-safe design choice: an operator who forgets to configure the mode does not accidentally end up with mutable legal records; the system defaults to the **most** protective setting.

6.3.2 Governance Mode — Immutability with an Authorised Escape Hatch

Governance mode makes archived records immutable against **ordinary credentials**. A user, an application, or an attacker holding normal access cannot shorten the retention window, cannot overwrite the object, and cannot delete it. This gives the essential **ransomware-safe** property: a compromised ordinary credential — the most common real-world attack vector — cannot destroy the archive.

However, Governance mode retains a controlled escape hatch. A holder of a specific, elevated privilege (in S3 terms, s3:BypassGovernanceRetention, commonly called a **break-glass** privilege) can reclaim or delete a Governance-locked object even within its retention window. This is a deliberate design for records where the organisation needs immutability against everyday risk **but must retain the ability to act** — for example, to correct a demonstrable error, respond to a lawful erasure order, or manage genuine storage lifecycle. The critical control is that this power is (a) held by a very small number of controlled principals, (b) itself a distinct, separately-granted privilege, and (c) evidenced whenever exercised.

Legal implication. Governance mode is appropriate where the organisation's legal position is: **"These records must be protected against tampering and ransomware, but we — through a controlled, authorised, and audited break-glass process — must be able to override retention in genuinely exceptional cases."** It provides strong tamper-evidence and a defensible immutability story, tempered by an acknowledged, evidenced override. Counsel should note that the **existence** of a break-glass override, even if never used, may be relevant to how strongly a court or regulator treats the immutability claim — the platform's answer to this is that any override is itself audited (Chapter 7).

6.3.3 Compliance Mode — Immutability Against Everyone

Compliance mode is the records-management default for legal archives and is materially stronger. Under Compliance, an archived object is immutable to **everyone, including a break-glass holder**, until the retention window genuinely elapses. There is no privilege — administrative, root, or otherwise — that can shorten retention, overwrite the object, or delete it before its retain-until date. The storage layer itself refuses the operation.

This has a precise and powerful legal meaning: **for the duration of the retention window, no human being inside the organisation can alter or destroy the record**. Not the general counsel, not the system administrator, not the CISO, not a person with the highest credentials in the enterprise. The record is fixed by the physics of the storage substrate, not by the discipline of the people who operate it.

Legal implication. Compliance mode is the appropriate posture where the organisation's legal position must be: *"These records cannot be altered or destroyed by anyone until their retention period expires, full stop."* This is the posture that most strongly supports:

- **Regulatory retention mandates** that require un-bypassable preservation.
- **Evidentiary reliability** — a Compliance-locked record carries the strongest possible assurance that it is the original, unaltered record, because **no one** had the technical ability to change it.
- **Insider-threat and coercion resistance** — because even the most privileged insider cannot alter the record, an argument that "an administrator could have changed it" is defeated at the storage layer.

The platform's product design explicitly identifies Compliance as *"the records-management default for legal archives"* — reflecting that, for a legal suite, the strongest immutability posture is the expected baseline, not an exceptional add-on.

6.3.4 Choosing Between Governance and Compliance — A Legal Decision

The choice of posture is **not primarily a technical decision; it is a legal and records-management decision**. The following framing helps counsel decide:

- Choose **Compliance** when the record class carries a **regulatory or statutory retention mandate** that requires un-bypassable preservation, or when the evidentiary value of absolute immutability outweighs any conceivable need to override retention. This is the recommended default for finalised legal records — executed contracts, closed-matter files, investigation outcomes, formal legal opinions, and regulatory filings.
- Choose **Governance** when the record class needs strong tamper- and ransomware-resistance **but** a genuine, foreseeable, controlled override capability must remain — for example, records that may be subject to a lawful erasure order under data-protection law, where the organisation must retain the technical means to comply. Even here, the override is a small, audited, break-glass path.
- Choose **None** only for records that are explicitly **not** legal records and where the operator has deliberately opted out of locking. **None must never be used for legal records**, and the platform's default-to-Compliance behaviour is designed to prevent None from being reached by accident.

Counsel confirmation point. The mapping of **record classes** to **WORM postures** is a policy decision. Counsel should approve a schedule that assigns each legal record class (contracts, matters, investigations, consultations, correspondence, regulatory filings) to a posture, and should specifically consider the interaction between Compliance-mode immutability and any right-to-erasure or data-subject-deletion obligations under the Personal Data Protection Law (PDPL). **Confirm the erasure-vs-immutability treatment with counsel per record class.**

6.4 The Ten-Year Default Retention Window

6.4.1 What the Default Is and Where It Applies

When a record is archived under Governance or Compliance mode and no explicit retention date is supplied for that specific archive operation, the platform applies a **default retention window of ten years**, aligned with common records-management practice. This default is codified in the archiving transport itself: the retain-until date defaults to a ten-year window from the moment of sealing.

Three properties of the default matter for legal readers:

- 58. **It is a floor set at seal time.** The retain-until date is stamped onto the object when it is written to WORM storage. From that instant, the object cannot be deleted before that date under Governance mode (except by break-glass) or under Compliance mode (by no one).
- 59. **It is overridable **upward** per operation.** An individual archive operation may supply an explicit, **longer** retention window where a particular record class requires it. The default is a sensible baseline, not a ceiling.
- 60. **It is configurable.** The ten-year window is the shipped default; the organisation may configure a different default retention period appropriate to its records schedule.

6.4.2 The Legal Significance of a Default, Not a Prompt

A subtle but important design decision is that retention is a **default that applies automatically**, not a value a user must remember to enter. If retention depended on a human entering a date each time, the inevitable result would be records archived with no retention (or a wrong retention) whenever the step was skipped. By defaulting to a ten-year window at the storage layer, the platform ensures that **every legally-archived record carries a substantive retention floor by default**, protecting the organisation against the far more common failure mode of **under**-retention through human omission.

Counsel confirmation point. Ten years is a **product default aligned to general records-management practice**, **not** a determination that ten years is the legally correct retention for any specific record type. Retention periods for legal records in the Kingdom may be driven by commercial-law limitation periods, sector-specific regulatory requirements, tax and accounting rules, employment-law obligations, and litigation-hold considerations, each of which may prescribe shorter or (more often) longer periods. **Counsel must confirm and approve the retention schedule per record class; the platform will enforce whatever period is configured.**

6.5 Legal Hold: Preservation that Survives Retention Expiry

6.5.1 The Problem Legal Hold Solves

Retention windows answer the question "how long must we keep this?" Legal hold answers a different and legally-critical question: **"this record must not be destroyed — regardless of any retention schedule — because we have a duty to preserve it."** Such duties arise when litigation is reasonably anticipated or pending, when a regulatory investigation is under way, when a preservation order or discovery obligation attaches, or when internal-investigation integrity requires that evidence be frozen.

The defining requirement of a legal hold is that it must **override the ordinary lifecycle**. A held record must not be destroyed even if:

- its retention window has expired;
- an administrator issues a deletion;
- an automated lifecycle process attempts disposal;
- a break-glass holder attempts an override; or
- a ransomware actor or malicious insider attempts destruction.

6.5.2 How Watheeq Implements Legal Hold — a Dual Floor

Watheeq implements legal hold at **two independent layers**, and the platform's own design describes the combination as the **"ransomware-safe FLOOR that survives any retention bypass."**

1. Application-layer legal hold. The legal suite maintains legal-hold records against subjects — a **contract**, a **matter**, or a **document** — each carrying a reason, an issuing actor, and lifecycle timestamps. Before any destructive operation, the application checks whether the subject is currently held. If it is, the operation is refused. Critically, this check applies **regardless of break-glass**: an active legal hold blocks disposition even for a privileged operator attempting an authorised override.

2. Object-lock legal hold at the storage layer. Independently of the application record, the platform can place an **object-lock legal-hold flag** directly on the archived object in the WORM store. This flag has a special and powerful property: **a legal-held object cannot be removed even with a Governance bypass**. In other words, the object-lock legal hold defeats the very break-glass privilege that would otherwise permit deletion of a Governance-mode object within retention. It is the storage substrate's own preservation lock.

The combination is what the design calls the **ransomware-safe floor**: a record that is both application-held and object-lock-held cannot be destroyed by ordinary credentials, cannot be destroyed by a break-glass holder, cannot be destroyed by retention expiry, and cannot be destroyed by ransomware — because the storage layer itself refuses the deletion until the hold is deliberately and authorisedly lifted.

6.5.3 Legal Hold Overrides Retention Bypass — the Precise Guarantee

To state the guarantee precisely, in the exact order the platform enforces it during a disposition attempt:

61. **Governance/Compliance retention gate.** Under Compliance mode, a disposition attempt is refused before retention elapses, for everyone. Under Governance mode, a disposition attempt is refused for ordinary credentials.
62. **Legal-hold gate — always blocks.** An active legal hold **always** blocks disposition, **break-glass or not**. This is the override-of-the-override: even the privileged principal who could bypass Governance retention is refused while a hold is active.
63. **Storage-layer object-lock legal hold.** Even if the application gates were somehow bypassed, the object-lock legal-hold on the object independently refuses removal at the storage substrate.

This ordering is the concrete embodiment of the principle that **preservation duties trump retention schedules and administrative privilege alike**. For counsel, this is the property that defends against the single most damaging litigation risk in records management — **spoliation** — the destruction of evidence that should have been preserved. Because a legal hold survives retention expiry and defeats break-glass, a record that has been properly placed on hold cannot be destroyed by the ordinary passage of time or by an over-eager cleanup, which is precisely the failure that produces adverse-inference sanctions.

6.5.4 Releasing a Hold — Fail-Closed by Design

Lifting a legal hold is itself controlled. The platform will **refuse to release the storage-layer object-lock hold while an application-layer legal hold is still active** on the subject — it returns a refusal such as *"an active lex legal hold still preserves this document."* This "fail-closed" behaviour prevents the dangerous sequence of releasing the storage lock while the organisation's own records still say the record must be preserved. A hold is only fully liftable when the organisation has affirmatively decided, in its own records, that the preservation duty has ended.

*Counsel confirmation point. The **decision to place, maintain, and release** a legal hold is a legal judgement about when a preservation duty arises and ends. The platform enforces the hold mechanically once counsel decides. **Counsel should own the legal-hold trigger criteria and the release authority, and should ensure the hold reason recorded in the system accurately reflects the preservation basis.***

6.6 Disposition and Destruction Controls

6.6.1 Why Controlled Disposition Matters Legally

Over-retention is a legal risk in its own right: records kept beyond their lawful purpose increase discovery burden, data-protection exposure, and breach impact. A mature records programme therefore must be able not only to **preserve** records but also to **dispose** of them — **but only when it is lawful and safe to do so, through an authorised and evidenced process**. Uncontrolled deletion is spoliation; over-retention is exposure; the correct answer is **controlled, gated, evidenced disposition**.

6.6.2 The Three-Gate Disposition Model

WatheeQ's destructive dispose operation is deliberately the most heavily guarded operation in the records lifecycle. It is **blocked** unless **all** of the following gates are satisfied, checked in sequence:

Gate 1 — Authorisation (break-glass).

Under **Compliance** mode, a disposition attempt requires explicit break-glass authorisation; without it, the operation is refused outright ("blocked: compliance WORM requires break-glass authorization"). Under any WORM mode other than None, disposition likewise requires break-glass. This ensures no ordinary user or routine process can ever initiate destruction of a WORM-protected legal record.

Gate 2 — No active legal hold (absolute).

An active legal hold **always** blocks disposition, **break-glass or not**. If the subject document, contract, or matter is held, the operation is refused ("blocked: an active legal hold preserves this document"). This gate cannot be overridden by authorisation; preservation duties are absolute against destruction.

Gate 3 — Retention genuinely elapsed.

The stamped retain-until date must have passed. If retention has not elapsed, the operation is refused with the specific retain-until date disclosed ("blocked: retention has not elapsed (retain_until=...)"). Time itself, not an administrator's assertion, governs this gate.

Only when authorisation is present, no hold is active, and retention has genuinely elapsed does the platform proceed to the storage layer — and even then, it performs a **best-effort confirmation that no object-lock legal hold is still in force** before issuing the deletion. If the storage object-lock still protects the object, the deletion is refused by the storage substrate itself, and the platform reports it as "blocked by storage object-lock (retention/hold still in force)."

6.6.3 Defence-in-Depth: The Storage Layer Independently Confirms

The platform's design is explicit that the application-layer gates and the storage-layer object-lock are **independent, redundant controls**. The application only ever attempts a deletion after its own compliance gate has passed (break-glass present, no active hold, retention elapsed); if for any reason a premature deletion were attempted, the storage object-lock **independently refuses it**, and that refusal is surfaced as a distinct, evidenced outcome rather than a silent failure. As the design states, a refusal at the storage layer is "the storage layer independently confirming the floor."

For a lawyer, this dual-independent-control structure is significant: it means the immutability and preservation guarantees do not rest on the correctness of a single piece of software. Two independent systems must both be defeated to destroy a protected record, and the storage layer is the last, un-bypassable line.

6.6.4 The Disposition Decision Table

The following table summarises when disposition is permitted, and is a useful reference for counsel and records officers:

WORM mode	Break-glass?	Active legal hold?	Retention elapsed?	Result
Compliance	No	—	—	Refused (break-glass required)

Governance	No	—	—	Refused (break-glass required)
Any WORM	Yes	Yes	—	Refused (legal hold is absolute)
Any WORM	Yes	No	No	Refused (retention not elapsed)
Compliance	Yes	No	Yes	Permitted — then storage re-checks object-lock hold
Governance	Yes	No	Yes	Permitted — then storage re-checks object-lock hold
None	—	(no hold)	—	Ordinary delete (not a legal record)

The recurring pattern is that **the strictest applicable control wins**, and **legal hold is absolute** — it defeats even a fully-authorised, retention-elapsed disposition attempt.

Counsel confirmation point. Break-glass authority for disposition should be assigned to a small, named group under a documented approval procedure, and each disposition should be recorded against an approved disposition schedule. **Counsel should approve the disposition authority matrix and confirm that the disposition process satisfies the organisation's records-destruction policy and any regulatory notification requirements.**

6.7 In-Kingdom Storage and Data Residency (PDPL)

6.7.1 The Residency Requirement

Saudi data-protection and sovereignty expectations require that certain categories of data — including sensitive legal records — be stored within the Kingdom. Watheeq enforces this through an **in-Kingdom-only** residency control on the archive backend, applied **fail-closed** and **before any object is ever written**.

6.7.2 How Residency Is Enforced — Fail-Closed, Before Any Write

The archiving connector resolves the region of the target object-store bucket (from configuration or from a live bucket-location query) and checks it against an **explicit allow-list of in-Kingdom regions**. The enforcement has several deliberately strict properties:

- **Enforced at connection-test time and before every archive write.** An endpoint whose resolved region is not in-Kingdom is **refused rather than silently archived abroad**. Residency is not a post-hoc audit; it is a precondition to writing.
- **Fail-closed on ambiguity.** An empty or unrecognised region is treated as **not in-Kingdom**. The system does not assume residency it cannot verify; the default answer to "is this in-Kingdom?" is **no** unless the region is affirmatively on the allow-list.
- **Explicit exclusions are documented.** Notably, the AWS Middle East (UAE) region — although geographically in the region and superficially plausible — is **explicitly excluded** from the in-Kingdom allow-list because the UAE is **not** in the Kingdom. This exclusion is deliberate and is enforced so that a well-meaning operator cannot mistake a nearby region for a compliant one.

The allow-list itself is intentionally small and explicit, covering sovereign and local object-store regions (for example, operator-labelled in-Kingdom regions and on-premises/sovereign deployments the operator has affirmatively asserted). The design principle is *"the list is intentionally small and explicit; an unrecognised region fails closed."*

6.7.3 Legal Significance

For counsel, the fail-closed, pre-write residency control provides a strong compliance posture: the organisation can represent that legal records archived through Watheeq are **verified to be stored in-Kingdom before they are written**, and that any misconfiguration results in a **refusal to archive** rather than an inadvertent cross-border transfer. This directly supports data-localisation obligations and materially reduces the risk of an unlawful transfer occurring silently.

Counsel confirmation point.** The allow-list of in-Kingdom regions must be reviewed against the organisation's specific hosting arrangements and against the current PDPL and sovereignty guidance. Where the organisation uses a sovereign or on-premises store labelled as in-Kingdom, counsel and infrastructure owners should confirm that the physical location genuinely satisfies the residency requirement. **Confirm the in-Kingdom region set and the hosting arrangements with counsel and the sovereignty/compliance owner.

6.8 Governance vs Compliance Posture — Consolidated Legal Guidance

Because the Governance-versus-Compliance choice is the single most consequential records-management decision, this section consolidates the guidance for legal readers.

6.8.1 The Core Distinction Restated

- **Governance** = immutable to ordinary users; **an authorised break-glass holder can still override** within retention. Ransomware-safe against everyday credentials; retains a controlled, evidenced escape hatch.
- **Compliance** = immutable to **everyone including break-glass** until retention elapses. No override exists; the record is fixed for the retention period by the storage substrate.

6.8.2 Legal Implications Compared

Consideration	Governance	Compliance
Tamper resistance vs ordinary users	Strong	Strong
Tamper resistance vs privileged insiders	Override exists (break-glass)	None can override
Ransomware resilience	Yes	Yes
Suitability for statutory un-bypassable retention	Limited (override exists)	Yes
Evidentiary weight of "unaltered" claim	High, but override must be explained	Highest — no one could alter it
Flexibility to comply with lawful erasure within retention	Available via break-glass	Not available until retention elapses
Recommended default for finalised legal records	Only where erasure flexibility is genuinely needed	Yes (product default)

6.8.3 The Central Trade-Off Counsel Must Weigh

The trade-off is between **absolute evidentiary strength** and **operational/legal flexibility**:

- **Compliance** maximises evidentiary strength and defeats any "an administrator could have altered it" argument, but **removes the ability to erase or correct within retention**, which may conflict with data-subject erasure rights or the need to remove genuinely erroneous or unlawful content before retention ends.
- **Governance** preserves an authorised, audited ability to act within retention, but a court or regulator assessing an immutability claim may weigh the existence of the override (which the audit trail in Chapter 7 evidences whenever exercised).

The platform's stance — and the recommended baseline for a legal suite — is that **finalised legal records default to Compliance**, with Governance reserved for the narrow classes where a lawful, foreseeable override capability is genuinely required. The default-to-Compliance behaviour ensures the stronger posture is the one reached by inattention.

*Counsel confirmation point. Because Compliance mode makes records un-erasable until retention elapses, counsel must specifically reconcile this with any PDPL right-to-erasure exposure for the affected record classes, and decide — per class — whether the retention period, and the immutability during it, are legally defensible. **This reconciliation is a legal determination counsel must make and document.***

6.9 Chapter Summary

Watheeq's records-management design gives legal and compliance owners a defensible, dual-layer answer to the trust, preservation, and disposition questions:

- **WORM storage** in three postures — with **Compliance** (immutable to everyone, including administrators, until retention elapses) as the records-management default for legal archives, **Governance** as the ransomware-safe posture with an authorised, audited override, and **None** reserved for explicitly non-legal buckets. The system **defaults to Compliance** when the posture is unset, so inattention yields the strongest protection.
- A **ten-year default retention window**, aligned to records-management practice, applied automatically at seal time so that every archived legal record carries a substantive retention floor by default — configurable and overridable upward per record class.
- **Legal hold** implemented as a **dual, independent floor** — application-layer and object-lock — that **survives retention expiry and defeats break-glass**, providing the ransomware-safe preservation guarantee that defends against spoliation.
- **Multi-gate disposition** requiring authorisation, absence of any legal hold, and genuinely-elapsed retention, with the storage layer independently confirming the floor as defence-in-depth.
- **Fail-closed in-Kingdom residency** enforced before any object is written, refusing rather than silently permitting cross-border archival.

Each of these controls is enforced not merely as application policy but at the **storage substrate**, which is what gives the resulting records their evidentiary weight — the subject of Chapter 7.

7. Audit Trail & Evidentiary Integrity

7.1 Purpose and Scope of this Chapter

Chapter 6 established that Watheeq's records are **immutable and preserved**. This chapter addresses the complementary and equally important question: **can we prove what happened to a record, and prove it to a standard a court or regulator will accept?** A record that cannot be shown to be unaltered, and an action whose actor, time, and content cannot be reliably reconstructed, has limited value in a dispute. The purpose of an evidentiary-grade audit trail is to convert the organisation's records and actions into **evidence** — material that supports admissibility, non-repudiation, chain of custody, and, ultimately, dispute defence.

Watheeq achieves this through several mutually-reinforcing mechanisms:

- 64. **Append-only audit logs** across the legal domain (matters, cases, requests, SLA clocks, investigations, settlements, calendars, service catalogues) that are **immutable at the database layer**.
- 65. **Content-hash chaining** — every archived document version is fixed by a cryptographic hash captured from the source of truth.
- 66. **Manifest chaining** — those content hashes are linked into a tamper-evident hash-chain, so that altering any archived record breaks the chain in a mathematically detectable way.
- 67. **A platform-wide immutable audit log** of all platform actions.
- 68. **Recovery-event audit trails** in the disaster-recovery subsystem, so that even the extraordinary act of recovering data is itself evidenced.

This chapter explains each mechanism and, more importantly, explains **why each one matters legally** — how it supports admissibility, non-repudiation, and the defence of a dispute.

Counsel confirmation point.** This chapter describes technical properties (append-only storage, hash-chaining, content-hash capture). The *legal effect* of these properties — their weight as evidence before a specific Saudi court or regulator, and the procedural steps needed to introduce them as evidence — is a matter for counsel to assess against the applicable rules of evidence and procedure. Where this chapter uses words like "admissibility" and "non-repudiation," it describes the *technical basis* the platform provides to *support* those legal conclusions; it does **not** assert the legal conclusions themselves. **Counsel should confirm the evidentiary treatment.

7.2 The Anatomy of an Evidentiary-Grade Audit Trail

Before describing the specific mechanisms, it is worth stating the four properties that, together, make an audit trail evidentiary-grade. Watheeq is designed to satisfy all four.

- 69. **Completeness** — every material action is recorded, so the trail is the full story, not a selective one.
- 70. **Immutability** — recorded entries cannot be altered or deleted after the fact, by anyone.
- 71. **Attribution** — each entry reliably records *who* did *what*, *when*, and to *which* record.

72. **Tamper-evidence** — any alteration to the recorded data, or to the records the trail describes, is **detectable**, not merely prohibited.

The distinction between immutability (property 2) and tamper-evidence (property 4) is important and often misunderstood. Immutability means the system **refuses** changes. Tamper-evidence means that **if** a change were somehow made — for example, by an actor with direct storage access outside the application — the change would be **provably detectable**. A trail that is both immutable and tamper-evident is far stronger than one that is merely immutable, because it does not require the fact-finder to **trust** that the system was never bypassed; it allows the fact-finder to **verify** it.

7.3 Append-Only Audit Logs Across the Legal Domain

7.3.1 The Append-Only Design Pattern

Throughout the legal suite, material actions write an **append-only audit row** inside the **same database transaction** as the action they record. This co-transactional design means the audit entry and the action it describes either both succeed or both fail — there is no window in which an action occurs without its audit entry, and no way to record an action and then quietly suppress its audit.

Each legal vertical carries its own append-only trail, including:

- **Matters** — a tenant-isolated, tamper-evident trail of matter lifecycle events (create, triage, status transitions, contract link/unlink, update).
- **Legal requests (the "spine")** — every material status transition of a legal request records an immutable row with the action, from-status, to-status, reason, and actor.
- **SLA clocks** — the SLA clock lifecycle (start, acknowledgement, escalation levels, breach) is recorded with escalation level and reason, so the service-level history of every legal request is fully reconstructable.
- **Investigations** — the investigation audit log carries **before/after JSONB snapshots** plus a free-text reason, so not only **that** something changed but **what it changed from and to** is preserved.
- **Cases, classifications, settlements, judgments** — each with its own append-only trail, several carrying before/after snapshots written inside the mutation transaction.
- **Calendars and the service catalogue** — even configuration changes to working calendars, holidays, and the service catalogue are audited with before/after snapshots.

7.3.2 Immutability Enforced at the Database Layer

The immutability of these trails is not merely a matter of the application declining to offer edit or delete functions. It is enforced by the database itself through **Row-Level Security (RLS)** configured in a specific, deliberate way:

- Each audit table has **Row-Level Security enabled and forced** (ENABLE ROW LEVEL SECURITY and FORCE ROW LEVEL SECURITY). The "forced" setting is critical: it applies RLS **even to the table owner**, closing the usual loophole by which a table's owner bypasses row-level policies.

- Each audit table grants **only** a tenant-isolation read policy and an **INSERT-only** write policy. There is deliberately **no UPDATE policy and no DELETE policy**.

The legal consequence of this configuration is precise and powerful. Under forced RLS, **an operation for which no permitting policy exists is denied to every session — the table owner included.**

Because the audit tables define no UPDATE and no DELETE policy, **no session can update or delete an audit row.** The trail is append-only **at the database layer**, not merely at the application layer. As the platform's own migration commentary states, this means the ledger is **"immutable"** and the property **"holds at both layers and is proven by test."**

This is the same property established in Chapter 6 for records: immutability that does not depend on the correctness or good behaviour of the application, because the substrate itself refuses the destructive operation.

7.3.3 Tenant Isolation

Every audit table is **tenant-isolated**: rows are scoped to a tenant, and RLS policies restrict access to the tenant that owns them. For an organisation operating within Watheeq, this means its audit trail is cryptographically and administratively separated from any other tenant's — a data-segregation property that matters both for confidentiality and for the integrity of the organisation's own evidentiary record.

7.3.4 Legal Significance of the Append-Only Trails

For counsel, the append-only, forced-RLS, co-transactional audit design supports several evidentiary propositions:

- **Completeness** — because the audit row is written in the same transaction as the action, the organisation can represent that every material legal action produced a corresponding, contemporaneous audit entry.
- **Immutability** — because no UPDATE/DELETE policy exists under forced RLS, the organisation can represent that recorded audit entries cannot have been altered or deleted, even by a database administrator or the table owner.
- **Attribution** — because each entry records the actor, action, from/to status, reason, and (for several verticals) before/after snapshots, the trail reliably reconstructs who did what and why.
- **Contemporaneity** — because entries carry creation timestamps and are written at the moment of the action, they are contemporaneous business records rather than after-the-fact reconstructions.

7.4 Content-Hash Chaining: Fixing the Content of Records

7.4.1 What a Content Hash Is and Why It Matters

A **content hash** is a cryptographic fingerprint (SHA-256) of a record's exact bytes. Two properties make it evidentiary gold:

73. **Determinism** — the same content always produces the same hash.
74. **Collision-resistance** — it is computationally infeasible to find two different contents that produce the same hash, and infeasible to alter content while preserving its hash.

The practical consequence is that a content hash **fixes** the content. If the organisation recorded a document version's content hash at the time of capture, then at any later time — years later — it can re-hash the document and compare. If the hashes match, the document is provably the **same**, **unaltered** document. If they differ by even a single byte, the alteration is detected.

7.4.2 The Content Hash Is Captured from the Source of Truth

WatheeQ captures the content hash from the **document version itself** — the `DocumentVersion.ContentHash` — described in the design as *"the source of truth."* When a document version is archived to WORM storage, the platform also computes the **SHA-256 of the bytes actually written to the archive** (the "object hash"). Recording **both** the source-of-truth content hash and the object hash means the organisation can prove not only that the document has not changed, but that **the bytes placed into the immutable archive are exactly the bytes the document store held** — closing the gap between "what we archived" and "what our document management said we archived."

7.4.3 Legal Significance

Content-hash capture is the technical foundation of the assertion *"this is the original, unaltered document."* For a lawyer producing a document in a dispute — a contract, a legal opinion, an investigation report — the ability to demonstrate that its content hash matches the hash recorded at the time of creation is a powerful answer to any challenge that the document has been altered, back-dated, or fabricated. It converts the document from something whose authenticity must be **asserted** into something whose authenticity can be **verified** by re-computation.

7.5 Manifest Chaining: Making the Whole Corpus Tamper-Evident

7.5.1 From Individual Hashes to a Hash-Chain

Fixing individual records is valuable; fixing the **whole set of records, in order, so that no record can be secretly inserted, removed, or altered** is stronger still. WatheeQ achieves this by chaining the individual content hashes into a **tamper-evident archive manifest** — a hash-chain that is, by design, *"identical in shape to the lex audit-log chain."*

Each archived document version contributes one **manifest entry**. Each entry contains:

- the sequence number of the entry in the chain;
- the document identifier and version;
- the **content hash** (from the source of truth);
- the **object hash** (SHA-256 of the bytes actually written to WORM);
- the **previous entry's hash** (`prev_hash`); and
- the **entry hash** — a SHA-256 computed over the sequence, document, version, content hash, object hash, and previous hash together.

The genesis (first) entry uses an empty previous hash; every subsequent entry chains off the prior entry's hash. This produces a linked chain in which each link cryptographically depends on the link before it.

7.5.2 Why the Chain Is Tamper-Evident

The chaining property means that **a single altered byte breaks the chain**. If anyone alters an archived object, or alters a manifest entry, or attempts to insert or remove an entry, the recomputed hashes will no longer match the recorded chain from that point forward. An auditor verifying the manifest recomputes each entry hash and checks that each links correctly to the next; any discontinuity pinpoints exactly where the tampering occurred. As the design states, *"the manifest is what an auditor verifies to prove the archived corpus has not been silently mutated."*

Two further design properties strengthen this:

- **One chain per document.** The sequence is per-document, so each document's archival history forms its own verifiable chain (enforced by a uniqueness constraint on tenant, document, and sequence).
- **The entry hash is stamped back into the document metadata.** After each successful archive, the resulting entry hash is written back into the document's archive metadata, so the document record itself carries a pointer into the verifiable chain, and the two can be cross-checked.

7.5.3 The Manifest Is Both Persisted and WORM-Anchored

The manifest exists as a persisted, append-only table **and** the underlying objects are WORM-protected in the archive backend. The design describes the object as *"WORM-protected in the backend (S3 object-lock / CMIS retention); this table is the verifiable index over it."* In other words, the chain is not merely a convenient index that could itself be rewritten — it sits over content that is independently immutable at the storage layer (Chapter 6). To defeat the tamper-evidence, an adversary would have to defeat **both** the WORM object-lock **and** reconstruct a consistent hash-chain, which is computationally infeasible.

7.5.4 Legal Significance

For counsel, the manifest hash-chain supports the strongest form of the evidentiary claim: not just "this individual document is unaltered," but **"our entire archived legal corpus is provably complete and unaltered, and any tampering would be mathematically detectable and localisable."** This is precisely the property that answers a challenge such as "how do we know a document was not quietly removed from, or inserted into, the archive after the fact?" — the chain would not verify if it had been.

7.6 Platform-Wide Immutable Audit Log

Beyond the legal-domain trails, the platform maintains a **platform-wide audit log** of all platform actions — an *"immutable audit trail of all platform actions,"* partitioned monthly for scale. It records, for every action, the originating service, the action performed (for example, a login or a role assignment), the resource type and identifier affected, the old and new values (for updates), and contextual metadata such as IP address and user agent.

The platform-wide log complements the legal-domain trails: where the legal trails capture **legal** actions (matter transitions, hold placements, disposition attempts), the platform log captures **system and identity** actions (authentication, authorisation, role changes, configuration). Together they provide a complete picture spanning both the legal and the operational planes.

The platform audit design is also hash-chain aware: the platform's own operational tooling recognises that the audit_logs store carries entry-hash / previous-hash chaining and treats it as WORM/hash-chain-sensitive material that must not be casually mutated. **Counsel should confirm the operational assurances around the platform log with the platform/compliance owner**, as the specific chaining and retention guarantees for the platform-wide log are an operational configuration.

7.7 Recovery-Event Audit: Evidencing Even the Extraordinary

A sophisticated evidentiary design must account for the **extraordinary** events too — most notably, disaster recovery. If data is recovered from a backup or a clean recovery point, a sceptic might ask: "was the record you produced the original, or something reconstructed or substituted during recovery?" Watheeq's disaster-recovery subsystem answers this by writing an **append-only audit trail of recovery actions**.

Every recovery or rehearsal action across the DR sub-solutions writes **one immutable row** recording **who** (the actor), **what** (the action), **when** (the logical instant it occurred), and **which** application, runbook, or event it acted on. This audit event store is, like the legal trails, **append-only by construction**: it grants a read policy and an insert policy and **no update or delete policy**, so under forced RLS *"every session (owner included) is denied UPDATE and DELETE at the DATABASE layer."* The design states that immutability *"holds at both layers and is proven by test."*

The recovery audit uses a **documented, machine-readable action vocabulary** — for example, that a clean recovery point was selected, that an integrity check was evaluated, that an approval was requested and granted, and that a return-to-production occurred. This produces a **regulator-ready evidence timeline** of exactly what happened during a recovery, including the integrity checks and approvals that governed it.

Legal significance. The recovery audit closes a subtle but real evidentiary gap: it ensures that even the act of recovering data — the one moment where records leave their steady-state and could, in principle, be tampered with — is itself fully evidenced, immutably, with attribution and integrity checks recorded. An organisation can therefore demonstrate not only that its records are immutable in steady state, but that their integrity was verified and evidenced through any recovery event.

7.8 Chain of Custody

Chain of custody is the ability to account for a record's integrity and control continuously, from creation to production in evidence, such that a fact-finder can be satisfied it was not altered or substituted at any point. Watheeq's mechanisms combine to provide a continuous chain of custody:

Custody question	Watheeq answer
Who created/handled the record, and when?	Append-only audit trails record the actor, action, and timestamp of every material event, co-transactionally.
Is the content the same as when captured?	The content hash captured at creation can be re-computed and compared; a match proves the content is unaltered.
Was the record altered in storage?	WORM object-lock (Compliance/Governance) prevents alteration; the manifest hash-chain makes any alteration detectable.

Was a record secretly inserted or removed?	The per-document manifest hash-chain would fail to verify if the sequence were altered.
Could an administrator have tampered with the trail itself?	Forced RLS with no UPDATE/DELETE policy denies alteration of audit rows to every session, owner included.
Was integrity preserved through recovery?	The recovery audit records integrity checks and approvals immutably.

The distinguishing strength of Watheeq's chain of custody is that **each link is independently verifiable and independently immutable**. The custody claim does not rest on the testimony of a custodian that "nothing was changed"; it rests on cryptographic verification (hashes and chains) and on storage-substrate enforcement (WORM and forced RLS) that make alteration either impossible or detectable.

7.9 Non-Repudiation

Non-repudiation is the property that an actor cannot credibly deny having performed an action they in fact performed. Watheeq supports non-repudiation through the combination of **attribution** and **immutability**:

- Every material action records the **authenticated actor** who performed it, within an audit entry that is written co-transactionally and cannot subsequently be altered or deleted.
- Because the entry is immutable at the database layer (forced RLS, no UPDATE/DELETE), the actor cannot later suppress or rewrite the record of their action.
- Because before/after snapshots and from/to statuses are captured for many verticals, the **substance** of the action — not merely its occurrence — is preserved.

The result is that an assertion such as "I never approved that contract" or "I never placed that record on hold" can be tested against an immutable, attributed record of the action. For internal-investigation, disciplinary, and dispute contexts, this is a materially strong position.

*Counsel confirmation point. Non-repudiation in a strict cryptographic sense (e.g., digital signatures tied to a specific individual's key) is distinct from the attribution-plus-immutability model described here. Where the organisation requires the stronger, signature-based form of non-repudiation for particular actions (for example, executed agreements), **counsel should confirm the signing arrangements** and how they integrate with the audit trail.*

7.10 Immutability and Its Legal Weight

The recurring theme across Chapters 6 and 7 is **immutability at the substrate, not merely at the application**. It is worth drawing out why this distinction carries disproportionate legal weight.

When a system is immutable only at the application layer, an opponent can argue: *"Yes, the application does not offer a delete button — but a database administrator, or an attacker with admin credentials, could have altered the data directly, so the record's integrity cannot be trusted."* This argument is difficult to rebut because it is, for many systems, technically true.

Watheeq is designed to defeat this argument at its root:

- **WORM Compliance mode** means that within the retention window, **no one** — not the administrator, not the root user, not a break-glass holder — can alter or delete the record. The "an administrator could have changed it" argument fails because, technically, no administrator could have.
- **Forced RLS with no UPDATE/DELETE policy** means audit rows cannot be altered or deleted by any session, **including the table owner**. The "a DBA could have edited the log" argument fails for the same reason.
- **Hash-chaining** means that even if some alteration were somehow achieved outside these controls, it would be **detectable** — so the fact-finder is not asked to trust that the controls were never bypassed; they can verify it.

This layered design — **immutability enforced by the substrate, plus tamper-evidence that makes any bypass detectable** — is what elevates Watheeq's records and audit trails from "internal business records the organisation vouches for" toward "verifiable evidence whose integrity can be independently demonstrated." That elevation is the entire point of an evidentiary-grade design.

*Counsel confirmation point. The *weight* a specific Saudi court or regulator will give to these technical properties, and the procedural foundation required to introduce hash-verification or WORM-immutability evidence, are legal determinations. This chapter establishes the *technical basis* on which such arguments can be built; **counsel should confirm how best to present and rely on these properties in a given forum.***

7.11 How This Supports Admissibility and Dispute Defence

Bringing the mechanisms together, the following table maps each evidentiary challenge an opponent might raise to the Watheeq control that answers it. This is intended as a practical reference for counsel preparing to rely on Watheeq-held records in a dispute.

Challenge raised by an opponent	Watheeq control that answers it
"This document was altered / back-dated / fabricated."	Content hash captured at creation is re-computed and matched (or the discrepancy pinpointed).
"A record was quietly removed from, or slipped into, the archive."	Per-document manifest hash-chain fails to verify if the sequence was tampered with.
"An administrator could have edited the underlying data."	WORM Compliance mode: no one can alter within retention. Forced RLS: no session can alter audit rows.
"The audit log was edited to cover this up."	No UPDATE/DELETE policy under forced RLS — audit rows are un-editable by every session, owner included; hash-chaining would reveal any out-of-band edit.
"You destroyed evidence you should have preserved."	Legal hold survives retention expiry and defeats break-glass; a held record cannot be disposed.
"The record was changed during a recovery/restore."	Recovery audit trail immutably records integrity checks, approvals, and actions.
"These records were stored abroad / outside required jurisdiction."	Fail-closed in-Kingdom residency enforced before any write.
"The person now denies taking this action."	Immutable, attributed audit entry records the authenticated actor, action, and time co-transactionally.

The cumulative effect is a records-and-audit architecture that is designed to **support admissibility** (contemporaneous, complete, immutable business records with verifiable integrity), **support non-repudiation** (attributed, un-editable action records), **establish chain of custody** (each link

independently verifiable and immutable), and **defend against spoliation and tampering allegations** (preservation that survives retention and privilege; tamper-evidence that makes any bypass detectable).

7.12 Chapter Summary

Watheeq converts records into evidence through five reinforcing mechanisms:

75. **Append-only audit logs** across every legal vertical, written co-transactionally and made **immutable at the database layer** by forced Row-Level Security with no UPDATE/DELETE policy — immutable even to the table owner, and **proven by test**.
76. **Content-hash capture** from the source-of-truth document version, fixing each record's content so that any later alteration is detectable by re-computation.
77. **Manifest hash-chaining** that links archived content hashes into a per-document chain in which a single altered byte breaks the chain — making the whole archived corpus tamper-evident and any tampering localisable.
78. **A platform-wide immutable audit log** of all system and identity actions, hash-chain-aware and partitioned for scale.
79. **Recovery-event audit** that evidences even the extraordinary act of data recovery, immutably and with integrity checks recorded.

Together with the WORM immutability and legal-hold preservation of Chapter 6, these mechanisms give the organisation records whose **integrity can be independently verified**, whose **history is complete and un-editable**, and whose **custody is continuously accountable** — the architecture of evidence, not merely of storage.

*Overarching counsel confirmation point. This document describes the **technical controls** Watheeq provides and the **evidentiary properties** they are designed to support. The legal characterisation of those properties — retention adequacy, admissibility, evidentiary weight, and the reconciliation of un-bypassable retention with data-subject rights — are determinations for the organisation's counsel under the applicable Saudi statutes, regulations, and rules of procedure. No statute, certification, or regulatory number is asserted in this chapter; where a specific legal requirement or standard is to be relied upon, **counsel should confirm the precise reference and its application**.*

8. Delegation of Authority (DoA) — Legal Validity

8.1 Purpose and legal framing

In any organisation of the scale of Abdullah Al Othaim Investment Company, legal acts — approving a contract, authorising the commencement of litigation, signing a settlement, closing a legal request — are almost never performed by the person who ultimately **holds** the authority to bind the company. They are performed by delegates: a section manager acting for the Legal Director, a business-unit approver acting within a financial ceiling, a stand-in acting during a period of leave. The legal question that follows every such act is deceptively simple and persistently difficult: ***who actually authorised this, and did they have the authority to do so?***

WatheeQ answers that question not with a signature scanned onto a page, and not with a name typed into a free-text field, but with ***cryptographically verifiable Delegation-of-Authority (DoA) evidence***. When a WatheeQ approval policy requires it, the person approving a legal act must present a triple of proof:

80. an **X.509 certificate** that identifies the authority holder;
81. a **detached digital signature** over a canonical authority payload; and
82. the **signed payload** itself, which carries the financial limit of the delegated authority.

The platform validates all three against a trusted certificate authority before the approval is allowed to take effect. The result is that the record of **who authorised what** is not a human assertion that can later be disputed — it is a mathematical fact bound to a certificate that a trusted authority issued to a named person. This chapter explains how that mechanism works, why it produces **legal certainty** rather than mere convenience, and how it interlocks with the authority matrices, thresholds, and expiry controls that govern delegation in practice.

This is a genuine capability in the codebase. The validation engine lives in a dependency-free, standard-library-only cryptographic package (internal/lex/crypto/authority_evidence.go), and the approval-time enforcement lives in the legal workflow service (internal/lex/service/workflow_service.go). The description below is a faithful account of that implementation, expressed for a legal reader.

8.2 What "Delegation of Authority" means in Watheeq

Delegation of Authority is the formal, documented conferral of a defined power onto a named person, usually subject to a **ceiling** (a monetary or scope limit) and a **validity period**. Watheeq treats DoA as a first-class governance object with three distinguishable layers, each of which is enforced separately so that a weakness in one layer cannot silently override another:

- **The role layer** — *may this kind of person approve this kind of act at all?*
 - **The identity layer** — *is the person approving this specific act a different person from the one who authored it?*
 - **The authority layer** — *does the approver actually hold the delegated financial authority they are claiming, and can they prove it cryptographically?*
- This is answered by the Role Matrix and RBAC (Chapters 9 and 10): only a role that holds the relevant approve capability reaches an approval control point.
- This is answered by the dynamic Segregation-of-Duties guard (Chapter 9): author must not equal approver.
- This is the subject of the present chapter. It is answered by PKI validation of the submitted authority evidence.

Only when all three layers are satisfied does the approval succeed. The cryptographic authority layer is the innermost and strongest of the three: it is the one that converts an approval from *a person with an approver role clicked approve** into *a person whom a trusted certificate authority has certified as

holding SAR X of delegated authority signed this exact decision"*.

8.3 The shape of authority evidence

When an approver acts on a legal record whose approval policy requires authority

evidence, the decision request carries an **authority-evidence block**. In the

API this is the `authority_evidence` object on the workflow decision request. Its

fields (from `dto.ApprovalAuthorityEvidence`) are:

Field	Meaning	Legal significance
role	The authority role being asserted (e.g. cfo)	Names the office whose delegation is claimed
authority_amount	The plain-text financial ceiling claimed	The stated limit of the delegation
currency	ISO currency code (e.g. SAR)	The unit the ceiling is expressed in
evidence_id	Identifier of the authority document/instrument	Links the act to a delegation instrument of record
certificate_pem	The X.509 leaf certificate (PEM), plus any intermediates	Cryptographically identifies the authority holder
signature_b64	The detached digital signature (base64)	Binds the signer's private key to the payload
signature_alg	The signature algorithm (e.g. ECDSA-SHA256, RSA-SHA256)	Fixes the cryptographic method used
signed_payload_b64	The canonical payload that was signed (base64)	Carries the cryptographically-bound authority amount
trusted_roots_pem	Optional per-request PEM bundle of trusted roots	Allows a request-scoped trust anchor

Two of these fields deserve emphasis because they carry the whole legal weight of

the mechanism:

- ``certificate_pem`` is not a name. It is an X.509 certificate — the same class of credential that underpins TLS on the public web and qualified electronic signatures under electronic-transaction law. It contains a **subject** (the identity of the holder), an **issuer** (the certificate authority that vouches for that identity), a **serial number**, and a **validity window** (*notBefore* / *notAfter*). Because a certificate authority has bound the subject identity to a public key, possession of the matching private key is proof of identity.
- ``signed_payload_b64`` is the canonical JSON document that the holder actually signed. When present it carries an `authority_amount` and `currency`. Critically, this amount is **cryptographically bound** — it cannot be altered after signing

without invalidating the signature. This is what allows Watheeq to compare the **proven** authority ceiling (from inside the signed payload) against the **claimed** ceiling and against the value of the act being approved.

The distinction between the plain-text authority_amount (an **assertion**) and the signed-payload amount (a **proven fact**) is exactly the distinction that gives DoA its legal certainty. A plain-text field can be typed to any value; a signed payload cannot be edited without breaking the signature.

8.4 Three-pronged cryptographic validation

The heart of the mechanism is the authority-evidence validator

(AuthorityEvidenceValidator). When it validates a piece of evidence it runs **three prongs, and every prong must pass**. A failure in any prong is a failure of the whole validation, and the approval is rejected. The three prongs are:

8.4.1 Prong 1 — Chain of trust and validity window

The validator first parses the certificate bundle into a **leaf certificate** (the authority holder's own certificate) and any **intermediate certificates** supplied alongside it. It then establishes two things:

83. **Validity window**. The certificate must be inside its own notBefore / notAfter window, measured against the validator's clock. This is checked **explicitly and first**, so that an expired credential produces a specific, legally meaningful outcome — "the delegation certificate was outside its validity period" — rather than a generic failure. This is the cryptographic embodiment of **delegation expiry**: authority that has lapsed cannot be used, full stop.

84. **Chain to a trusted root**. The leaf must chain — through any supplied intermediates — to a **trusted root certificate authority** that the platform has been configured to trust. If the certificate was issued by an authority the platform does not trust, or the chain cannot be built, validation fails

with a "chain not trusted" outcome. The validator additionally requires that the leaf certificate carry a **digital-signature key usage**: a certificate not intended for signing cannot be used to authorise an act.

The trust anchor is configurable. The platform holds a ****default trusted-roots pool**** established at startup, and an individual request may also carry its own `trusted_roots_pem` bundle. The design **fails closed**: if no trusted roots are configured at all — neither a default pool nor a per-request bundle — then chain validation cannot succeed and the evidence is rejected. There is no path by which an untrusted or unrooted certificate is quietly accepted.

8.4.2 Prong 2 — Detached signature verification

Once the certificate is trusted and in-date, the validator verifies the **detached signature** over the exact payload bytes, using the public key inside the leaf certificate. The algorithm is selected from the declared `signature_alg`, and the platform supports the standard ECDSA and RSA families over SHA-256, SHA-384, and SHA-512:

- ECDSA-SHA256 / ECDSA-SHA384 / ECDSA-SHA512
- RSA-SHA256 / RSA-SHA384 / RSA-SHA512

The signature is checked against the digest of the payload under the declared hash. If the payload has been altered by so much as a single byte, or if the signature was produced by a different key than the one in the certificate, the check fails with a "signature invalid" outcome. If the certificate's key type does not match the declared algorithm family — for example an RSA algorithm declared against an ECDSA key — the check fails with an "unsupported algorithm" outcome rather than silently proceeding.

This prong is what establishes **non-repudiation**. Because only the holder of the private key corresponding to the certificate's public key could have produced a signature that verifies, a valid signature is proof that **this specific holder**

signed **this specific decision payload**. The approver cannot later claim they did not authorise it, and no other party could have forged the authorisation.

8.4.3 Prong 3 — Payload extraction and the bound limit

Finally, if the signed payload is canonical JSON, the validator extracts the **cryptographically-bound authority amount** and currency from it. This is the proven ceiling — the figure the authority holder actually signed. The validator returns it to the caller as part of the verified-authority result, alongside the certificate's subject, issuer, serial number, and validity dates.

The payload prong is deliberately tolerant of shape: a signed blob that is not canonical JSON (or that carries neither an amount nor a currency) is still valid evidence of **identity and intent** — it simply does not carry a machine-comparable ceiling. This lets the same mechanism serve both value-bounded delegations (a CFO signing within a SAR ceiling) and unbounded authorisations (an executive directive to commence action) without a change of machinery.

8.4.4 Optional revocation

Beyond the three core prongs, the validator supports ***offline revocation checking***. When enabled, a certificate whose serial number appears on a configured revocation list is rejected with a "revoked" outcome, even if its chain and signature are otherwise valid. This models the real-world event in which a delegation is **withdrawn before its certificate expires** — for instance, when an authority holder leaves the company or a delegation is rescinded. The revocation set is supplied by configuration (LEX_APPROVAL_AUTHORITY_REVOKED_SERIALS), and revocation checking is toggled by ApprovalAuthorityRevocationEnabled. Network OCSP/CRL fetching is intentionally out of scope for this seam; the check is offline and deterministic so it behaves identically in every environment and can be reasoned about by auditors.

8.5 From validation to approval decision

The cryptographic validator is a pure engine. The **policy** that decides whether and how strictly it is applied lives in the legal workflow service, in the `validateAuthorityEvidencePKI` step that runs as part of a workflow approval decision. Its behaviour is precise and conservative:

- **It runs only for an `approve` decision.** A reject decision does not require the approver to prove authority — one does not need delegated financial power to decline. This is verified behaviour: the validator is not invoked at all on a non-approve decision.
- **It runs only when the approval policy requires authority evidence.** Each approval policy carries a `RequireAuthorityEvidence` flag and, optionally, a `RequiredAuthorityAmount`. If the policy does not require evidence, the PKI step is a no-op. This lets the organisation apply cryptographic DoA selectively — to high-value contracts and sensitive legal acts — without imposing it on routine, low-risk approvals.
- **Strict mode versus fallback.** The service is in **strict mode** only when a validator is wired *and* trusted roots are configured. In strict mode, cryptographic material (`certificate_pem`, `signature_b64`, `signature_alg`) is **required**: an approval on a policy that demands evidence, but which arrives without cryptographic material, is rejected before the validator is even called. When no trusted roots are configured, the service falls back to accepting plain-text evidence — but it **logs a warning** whenever cryptographic material was supplied yet could not be verified, so that an operator can see that a piece of evidence was accepted un-verified. The plain-text checks that already ran are never relaxed; PKI is strictly additive.

8.5.1 Threshold enforcement against the proven ceiling

When strict validation succeeds and the signed payload carries a bound authority

amount, the service performs the decisive comparison. The proven ceiling must

cover both:

- the policy's RequiredAuthorityAmount — the minimum delegated authority the policy demands for this class of act; and
- the **contract value** (or the value of the act being approved).

If the proven ceiling is below either figure, the approval is rejected with an

"authority evidence bound amount is below the required approval authority" (or

"below the contract value") outcome. In concrete terms: a person holding a

cryptographically proven SAR 500,000 delegation **cannot** approve a SAR 1,200,000

contract, no matter what role they hold or what they typed into the plain-text

amount field. The ceiling that governs is the one inside the signature, not the

one anyone asserted.

This is the point at which **thresholds** become legally robust. In most systems a

delegation ceiling is a number in a table that an administrator can edit. In

Watheeq, when cryptographic DoA is in force, the operative ceiling is the one the

authority holder **signed** — it is bound to their certificate and cannot be raised

after the fact without producing a new, differently-signed instrument.

8.5.2 Mapping cryptographic failures to legal outcomes

Every failure mode of the validator is mapped to a specific, human-readable

validation error keyed to the offending field. This matters for a legal reader

because it means the **reason** an approval was refused is recorded precisely:

Cryptographic outcome	Recorded meaning
Certificate outside validity window	The delegation certificate had expired or was not yet valid
Chain not trusted	The certificate was not issued by a trusted authority
Signature invalid	The decision payload was altered, or the wrong key signed it
Unsupported algorithm	The declared signature method was not accepted
Certificate revoked	The delegation had been withdrawn
Malformed evidence	The submitted material could not be parsed

A non-sentinel (unexpected) validator error is surfaced as an internal error

rather than being swallowed — the approval does not proceed on an unclassified failure. The system's default posture at every branch is to **deny on doubt**.

8.6 Authority matrices, thresholds, and expiry in practice

The cryptographic layer described above does not stand alone; it operationalises the **authority matrix** that the organisation maintains. In practice, three governance dimensions come together at an approval control point:

85. **The authority matrix (who may approve what, up to what value).** This is expressed through the Role Matrix and the approval policy. The Role Matrix grants the relevant approve capability to the appropriate roles (see Chapter 10), and the approval policy attaches a RequiredAuthorityAmount to the class of act. Together they define the threshold ladder: routine acts need only a role with the approve verb; high-value acts additionally demand proven authority at or above the required amount.
86. **Thresholds.** A threshold is enforced twice over in strict mode: the plain evidence must claim an amount at least equal to the policy requirement, and the **cryptographically bound** amount must cover both the requirement and the value of the act. The signed ceiling is authoritative.
87. **Expiry.** Delegations lapse. Watheeq enforces lapse cryptographically through the certificate validity window — an expired certificate is rejected before any other check — and operationally through revocation, which handles the case where a delegation is withdrawn **before** its certificate would naturally expire. The combination means that neither a **stale** nor a **revoked** delegation can authorise an act.

8.7 Why cryptographic validation gives legal certainty

The core value proposition of DoA validation, stated for a lawyer, is this: it converts the answer to **"who authorised this?"** from **testimony** into **evidence**.

- **Identity certainty.** The approver is identified by an X.509 certificate that a trusted certificate authority issued to a named holder. Identity is not a login name that could be shared or spoofed; it is a credential bound to a private key the holder controls.
- **Integrity certainty.** The exact decision — including the authority ceiling — was signed. Any subsequent alteration of the payload invalidates the signature. The document of authorisation cannot be edited after the fact without detection.
- **Authority certainty.** The ceiling that governs the approval is the one inside the signature, checked against both the policy requirement and the value of the act. A delegate cannot exceed their proven authority.
- **Non-repudiation.** Because only the holder of the private key could have produced a verifying signature, the approver cannot credibly deny having authorised the act. This is the property that matters most in a dispute.
- **Temporal certainty.** The certificate's validity window and the revocation list together ensure that only a **currently valid** delegation can act. An authority that has expired or been withdrawn is inert.
- **Fail-closed posture.** Every ambiguous or failing path denies the approval. A missing trust anchor, an unparseable certificate, an unclassified error — none of these "open the door" by accident.

Taken together, these properties mean that a Watheeq approval carried out under strict DoA is defensible in a way an ordinary electronic approval is not. If a counterparty, a regulator, or an internal audit later asks *"on what authority was this legal act taken?"*, the answer is a certificate, a signature, and a bound ceiling — a chain of proof that can be independently re-verified, rather than a name in a log and a hope that it was really that person.

9. Segregation of Duties & Governance Controls

9.1 Why Segregation of Duties is a legal control, not merely an IT one

Segregation of Duties (SoD) is the principle that no single person should control all stages of a transaction that carries risk of error or abuse. In legal and compliance terms it is one of the oldest anti-fraud and conflict-of-interest controls there is: the person who **drafts** an instrument should not be the person who **approves** it; the person who **approves** should not be the person who **audits**. When SoD is absent, a single individual can create, authorise, and conceal an improper act with no independent check — the classic precondition for fraud, self-dealing, and unaccountable error.

WatheeQ treats SoD as a **governance invariant enforced in code**, not as a policy statement that depends on good behaviour. The controls described in this chapter are implemented as authorisation guards and role constraints that the platform applies on every sensitive action. They fall into four interlocking mechanisms:

- 88. **Static SoD (SSD)** — a single user may not simultaneously **hold** two mutually-exclusive roles.
- 89. **Dynamic SoD** — the specific user who **authored** a record may not approve or close that same record, even if they hold an approver role.
- 90. **Distinct two-round approvers** — where two tiers of approval are required, a single person may not satisfy both.
- 91. **No coarse-permission fallback on sensitive routes** — an ordinary "edit" permission may not be used to slip a record past an approve/close control point.

Together these ensure that the separation between **creating**, **approving**, **closing**, and **auditing** a legal act is real and cannot be collapsed onto one person.

9.2 Static Segregation of Duties (SSD): mutually-exclusive roles

The first control operates at the level of **role assignment**. Certain legal

roles are inherently in tension with one another — holding both would let one

person sit on both sides of a control. Watheeq encodes these as **role-exclusion**

pairs and enforces them both when roles are **seeded** and when they are

assigned.

The exclusion logic (internal/auth/legal_role_exclusions.go) defines three

categories of conflict:

Excluded pair	Reason (the conflict it prevents)
Legal Officer ⊥ Cases & Investigations Section Manager	The case drafter must not also be the manager who approves and closes the same case
Legal Advisor ⊥ Contracts Section Manager	The contract recommender must not also be the manager who gives final sign-off
Any operational role ⊥ Auditor / Compliance Officer	No one who operates on legal records may also be the read-only auditor over them

The third rule is the broadest and the most important for governance. Rather than

being hand-maintained, the "any operational role" side is **derived from the Role**

Matrix itself: the code computes the set of every role that holds at least one

mutating verb (add, edit, approve, close, assign, or distribute) on a legal

domain, and excludes each of them from also holding the Auditor role. Because the

set is derived, it can never drift out of step with the matrix — if a new

operational role were added, it would automatically become excludable from the

auditor. The auditor is thereby guaranteed to be an **independent counterparty**:

a person who can **see** everything but **operate** nothing.

These pairs are enforced in two places, which is why SSD is not merely

aspirational:

- **At role assignment.** The role-granting path calls the exclusion checker (CheckRoleExclusion) before conferring a new role. If the candidate role conflicts with a role the user already holds for the same organisational scope, the grant is refused with a descriptive error naming the conflicting pair. The

check is symmetric and order-independent — it does not matter which role was held first.

- **At seeding, with assertion.** The Role Matrix seeder writes the exclusion pairs into a `legal_role_exclusions` table and then **verifies** that they are all present. This is deliberately an **asserted** seed rather than a best-effort one (see §9.6): a silent failure to establish the exclusions would be a governance hole, so the platform treats an incomplete exclusion set as a startup-readiness failure.

The effect is a bright line: the platform will not allow the same individual to occupy two roles that would let them approve their own drafting, sign off their own recommendation, or audit their own operations.

9.3 Dynamic Segregation of Duties: author must not equal approver

Static exclusions prevent one person from holding two **conflicting roles**. But a subtler risk remains: a single person might legitimately hold a role that **can** approve, and also be the person who **drafted the particular record** in front of them. Nothing in static role membership prevents that individual from approving their own work. Dynamic SoD closes this gap.

The dynamic guard (`RequireDistinctActor`, in `internal/lex/middleware`) enforces the invariant that ***the person who authored a record may not approve or close that same record — regardless of the capability they hold***. It runs **on top of** the ordinary permission check: the permission gate has already established that the actor **may approve in general**; the dynamic guard then establishes that this particular actor is not the author of **this particular record**.

Its logic:

92. Read the target record's identifier from the request path.

93. Resolve, for that record, **who authored it** (its `created_by` / `initiated_by`)

and **who has already approved a prior step** on it.

94. **Deny** if the current user is the author, or is any prior approver. Otherwise, allow the action to proceed.

The design's most important property is that it is **fail-safe**: every error path denies. Specifically:

- A mis-wired guard (no resolver) denies rather than silently opening.
- An unauthenticated caller is refused.
- A record whose author cannot be resolved — because it is not found, or it carries no recorded author — is denied, because the platform **cannot prove** author $\neq$ approver.
- A lookup failure denies.

And crucially: **there is deliberately no administrative bypass**. Dynamic SoD binds the privileged operator too. This is a direct response to a real governance weakness in earlier designs, where a person holding **both** a drafting role and an approver role could self-approve. Under the current design, even a Legal Director cannot approve a record they themselves authored — the separation between authoring and approving is absolute at the level of the individual. For a legal reader, this is the mechanism that guarantees the ***four-eyes principle*** on every legal act: the eyes that created the record and the eyes that authorise it are always, provably, different.

9.4 Distinct two-round approvers

Some legal acts warrant **two** independent approvals — a first-tier review followed by a higher-tier sign-off. The obvious failure mode is that one person, holding both tiers' authority, satisfies both rounds themselves, reducing "two approvals" to one. Watheeq's approval engine forbids this through an ***opt-in distinct-approver constraint*** on two-round chains.

Where a review chain is configured to require distinct approvers (for example, the defendant first-response memo two-tier review), the chain is stamped with a

require_distinct_approvers flag. This flag is carried in the chain configuration and — importantly — **survives reconstruction** when the approval is resumed at a later step, so it cannot be lost between the two rounds. At decision time, the engine's DistinctApproverConflict check rejects the situation where a *single user decides both tiers*, while accepting two *distinct* deciders.

The tested, enforced behaviour is unambiguous:

- **Same actor deciding both tiers → separation-of-duties conflict (rejected).**
- **Two distinct deciders → no conflict (the review proceeds).**

This turns a two-tier review into a genuine two-person control. It is not enough to *have* a second approval step; the second step must be filled by a *different human being*. Combined with dynamic SoD (§9.3), which already excludes the author from either round, a two-round distinct-approver chain guarantees that a significant legal act passes through the hands of at least three independent people: the author, the first approver, and the second approver.

9.5 No coarse-permission fallback on sensitive routes

The final SoD mechanism is the most technical but is essential to understanding why the others cannot be circumvented. Throughout Watheeq, routine operations are permitted by *either* a fine-grained per-domain capability *or* a coarse "write"-class permission (lex:write) — a deliberate design choice for backward compatibility and operational simplicity. This coarse fallback is appropriate for ordinary edits. It is **not** appropriate for the control points that SoD protects.

The risk it would otherwise create is the **status-route bypass**. Many legal records move through a state machine (FSM), and a single "change status" route can drive a record through ordinary edit-class transitions *and* into terminal close-class states (closed / cancelled) or decision-class states (approved / rejected). If the status route accepted the coarse lex:write permission for all of these, then a holder of mere edit authority could drive a record straight into

an **approved** or **closed** state — bypassing the approve / close control points entirely, and with them, the SoD guards that hang off those control points.

Watheeq closes this hole with a **status-elevation guard**

(enforceStatusElevation, in internal/lex/handler/status_authz.go). The guard classifies the target of every status transition:

- An **edit-class** transition (e.g. intake → phase-1) is permitted by the ordinary edit tier, including the coarse `lex:write` fallback.
- A **close-class** transition (closed / cancelled) requires the exact `lex:<domain>:close` permission — **with no edit or `lex:write` fallback**.
- An **approve/decision-class** transition (approved / rejected) requires the exact `lex:<domain>:approve` permission — again **with no fallback**.

In both elevated cases, the guard **also** re-applies the dynamic-SoD check (author ≠ actor), so that the four-eyes principle holds even when a decision is reached via a status transition rather than via a dedicated approve/close endpoint. The result is that there is **no side door**: whether an actor tries to approve or close a record through the dedicated route or by nudging its status, the platform demands the elevated capability and an independent actor, and denies anything less.

This "no coarse fallback on sensitive routes" rule is what makes the whole SoD edifice load-bearing. Without it, every fine-grained control could be sidestepped by whoever held the broad `lex:write`. With it, the fine-grained approve and close verbs are the **only** keys that open those doors, and those verbs are handed out sparingly and exclusively per the Role Matrix.

9.6 The asserted seeder: governance controls that must be present

A recurring theme across these controls is that Watheeq refuses to run in a half-configured state where a governance control is silently absent. The Role Matrix seeder is **asserted, not best-effort**. After it seeds the 14 legal roles

and the SoD exclusion pairs, it runs a verification that:

- confirms **all 14 legal roles** are present for the tenant, and
- confirms **every SoD exclusion pair** is present.

If either is incomplete, the platform treats it as a **startup-readiness failure**.

This is a deliberate governance stance: a silent seeding no-op could otherwise

leave a tenant relying on the coarse lex:write fallback with none of the

fine-grained SoD controls in place — precisely the insecure state the whole design

exists to prevent. By making the presence of the controls a precondition for the

service being considered ready, Watheeq guarantees that a running Legal Affairs

service **has** its SoD matrix, rather than merely **intending** to.

A related subtlety worth recording for completeness: permission **enforcement** in

this platform is driven entirely by an in-code permission map, while the database

seed of the roles exists for the management UI, org-binding, and audit. The two

are kept in lock-step because both are generated from the **same** single source of

truth for the role definitions. This means the SoD constraints that are **enforced**

and the role definitions that are **displayed and audited** can never diverge.

9.7 How these controls prevent conflicts of interest and fraud

Read together, the four mechanisms form a defence-in-depth that maps directly onto

recognised anti-fraud and conflict-of-interest doctrine:

Fraud / conflict scenario	The control that prevents it
One person holds both the drafting role and its own approving role	Static SoD — the role pair is mutually exclusive and the grant is refused
One person holds both an operational role and the auditor role	Static SoD — every operational role is excluded from the auditor
A person with an approver role approves the very record they drafted	Dynamic SoD — author ≠ approver, with no admin bypass
A single person satisfies both tiers of a two-tier review	Distinct two-round approvers — same actor on both tiers is a conflict
A holder of ordinary edit rights pushes a record to "approved" or "closed" via the status route	No coarse fallback — the status-elevation guard demands the exact approve/close key and an independent actor
A tenant runs without its SoD controls configured	Asserted seeder — startup fails until the controls are present

For legal defensibility, the significance is that Watheeq can demonstrate, for any

completed legal act, that it *could not* have been created and authorised by a single person. The separation of duties is not a description of intended practice that an investigator would have to test against the logs; it is a property that the system *enforced at the moment of action* and would have refused to violate. That is the difference between a control that supports an audit finding and a control that pre-empts the finding.

10. Roles, Responsibilities & Access Control

10.1 The mapping from role to legal responsibility

Access control in Watheeq is not a generic list of permissions bolted onto a legal application. It is a direct, deliberate translation of the organisation's **legal responsibility structure** into enforceable authority. Every one of the 14 legal roles corresponds to a real office in the Legal Affairs function, and the capabilities that role holds correspond to what that office is **responsible and accountable** for. When a person holds a role, they hold exactly the authority their position warrants — no more (the principle of least privilege) and no less (so they can discharge their duties).

This chapter sets out the 14 roles, the verbs that express what each may do, how least-privilege and role-scoped visibility are enforced, and how the caller learns their own effective authority at runtime. The material is drawn from the Legal System Role Matrix as implemented in `internal/auth/legal_roles.go`.

10.2 The verb model: independent capabilities

Before the roles themselves, it is worth understanding the **unit** of authority.

Watheeq expresses authority as **per-domain verbs**. A capability slug takes the form `lex:<domain>:<verb>` — for example `lex:case:approve` or `lex:contract:distribute`. The legal domains are: request, case, investigation, settlement, contract, consultation, and document; plus the configuration/governance domains: sla, escalation, catalog, notification, role, audit, integration, and security.

The verbs are, deliberately, **independent flags**. This is a foundational design decision with real governance consequences:

- Holding edit does **not** imply approve. Being able to draft a case does not confer the authority to approve it.

- Holding approve does **not** imply edit, and close does **not** imply approve. There is **no reverse or cross implication** between verbs.
- The only implications the system draws are narrow and safe: any operational verb implies the **view** of the same domain (if you can edit a case you can see cases), a configuration manage verb implies the lower verbs of that same configuration domain, and a domain wildcard expands to that domain's own verbs. That is the entire set of implications, and it never crosses the drafting/approving line.

Two verbs are singled out as **restricted work-allocation verbs**, each an independent flag not implied by editing:

- **`assign`** on cases — the authority to **allocate** case work, treated as a section-manager-level power.
- **`distribute`** on contracts — the authority to **route** contract work, treated as a supervisor/manager-level power.

Making these independent is what lets the matrix grant, say, a supervisor the ability to distribute contract work without also granting them the ability to sign contracts off. In a coarser model where "manage the contract" was a single permission, that separation would be impossible.

The importance of independent verbs to SoD (Chapter 9) is direct: because edit and approve are separate keys, a role can be a **drafter** (edit) without being an **approver** (approve), and the two duties can be placed in different offices. The verb model is the atomic layer on which the entire separation-of-duties structure is built.

10.3 The 14 legal roles

The roles are organised into four tiers — **Business, Legal, Oversight,** and **Admin** — reflecting where in the organisation the office sits and what kind of authority it carries. The table below summarises each role, its tier, and the

essence of its responsibility. Arabic names are carried in the system for the bilingual Watheeq deployment.

#	Role (EN)	Tier	Core responsibility
1	Requester / Employee	Business	Raises legal requests and consultations for services they are eligible for
2	Department Manager (Requesting)	Business	Requester-side (DoA) approver of requests; escalation Level-2 recipient
3	Business Unit CEO	Business	High-value requester-side approver per the DoA matrix
4	CEO / Executive Management	Business	Issues the directive to commence legal action; top of the requester-side DoA
5	Legal Director (Head of Legal)	Legal	Owens the service catalogue; top legal authority and approver
6	Cases & Investigations Section Manager	Legal	Runs cases/investigations; assigns work and approves outputs
7	Contracts Section Manager	Legal	Runs contract review; holds final contract sign-off
8	Case Supervisor	Legal	Follow-up and first-tier case review; escalation Level-1 recipient
9	Contracts Supervisor	Legal	Distributes and first-tier contract review
10	Legal Officer / Handling Lawyer	Legal	Direct handler: pleadings, memos, hearings, investigations
11	Legal Advisor / Consultant	Legal	Reviews contracts and answers legal consultations (recommends only)
12	Shared Services Unit Manager	Oversight	System-owner oversight; escalation Level-3 recipient
13	Auditor / Compliance Officer	Oversight	Read-only audit and compliance — an SoD safeguard
14	System Administrator	Admin	Configures catalogue, calendar, roles, integrations, and security

10.3.1 The separation lines embedded in the roster

Several deliberate boundaries are visible in the roster, and each is a governance

choice rather than an accident of design:

- **Business-tier approval is DoA, not legal sign-off.** The Department Manager, Business Unit CEO, and CEO hold request:approve — they authorise *requests* under the Delegation-of-Authority matrix. They do **not** hold case:approve or consultation:approve; approving legal work-product is a Legal-tier responsibility, not a business-tier one. The CEO's ability to "issue a directive to commence action" is expressed as case:add (an initiation), not as an approval of legal output.
- **The Legal Officer drafts but does not approve.** The handling lawyer holds case:add and case:edit but **not** assign, approve, or close. They create and develop the work; a supervisor or manager approves it, and a manager assigns and closes it. This is the drafting/approving separation at the level of a single office.
- **The Legal Advisor recommends but does not sign off.** The advisor may view, add, and edit contracts and consultations, but holds **no** approve, **no** distribute, and **no** close. Their role is to *recommend*; final sign-off belongs to the Contracts Section Manager. This is why Advisor and Contracts Section Manager are a mutually-exclusive SSD pair (Chapter 9).
- **The Auditor sees everything and changes nothing.** The Auditor / Compliance Officer holds **view/read only** across every legal domain, plus audit read and the governance views — and critically holds **no** coarse lex:write. This is what makes the auditor a clean SoD counterparty: independent oversight with no operational footprint. It is enforced, not merely intended, by the role definition and asserted by test.
- **The System Administrator configures but does not operate.** The Admin holds configuration authority (catalogue, SLA, escalation, notification, role assignment and management, integration, security) and audit read — but holds

no legal-operational verb (no add/edit/approve/close/assign/distribute on any legal domain) and **no** coarse lex:write. Administering the *system* is kept strictly separate from operating on *legal records*. Note too that role administration is split: the Admin holds role:assign and role:manage, but the Legal Director holds only role:view — the top legal operator can *see* the role structure but the authority to change it sits with the System Administrator (and is further constrained by anti-escalation downstream).

10.4 Roles × duties: the responsibility matrix

The following matrix shows, for the key legal domains, which roles hold the **elevated** duties — approve, close, and the restricted assign/distribute verbs — that carry real legal weight. (View/add/edit are held more broadly by the operational roles and are omitted here for focus; a checkmark denotes the role holds that verb.)

Role	request:ap prove	case:assign	case:appro ve	case:close	contract:di stribute	contract:ap prove	contract:cl ose
Departmen t Manager (Requestin g)	✓						
Business Unit CEO	✓						
CEO / Executive Manageme nt	✓						
Legal Director	✓	✓	✓	✓	✓	✓	✓
Cases & Investigati ons Section Manager	✓	✓	✓	✓			
Contracts Section Manager	✓				✓	✓	✓
Case Supervisor	✓		✓				
Contracts Supervisor	✓				✓		
Legal Officer							
Legal							

Advisor							
---------	--	--	--	--	--	--	--

A few observations for the legal reader:

- The **Legal Director** is the only role with the full elevated set across cases and contracts — the top legal authority.
- **Assign** (case work allocation) sits only with the Director and the Cases Section Manager; **distribute** (contract work allocation) sits only with the Director, the Contracts Section Manager, and the Contracts Supervisor. These work-allocation powers are deliberately narrow.
- The **Case Supervisor** can approve (first-tier) but cannot assign or close; closing a case is reserved to the manager. The **Contracts Supervisor** can distribute and review but cannot approve or close — final sign-off is the manager's.
- The **Legal Officer** and **Legal Advisor** hold none of the elevated verbs — they draft, handle, and recommend, and their work flows to a distinct approver.

This matrix **is** the authority matrix of §8.6, viewed from the role side. It is the place where legal responsibility and system authority are made to coincide.

10.5 Least privilege and role-scoped visibility

10.5.1 Least privilege by construction

The roles are built to the **principle of least privilege**: each role carries the narrowest capability set that lets its office do its job. Nothing is "rounded up to the highest authority in a domain". A role that needs to draft cases gets case:add and case:edit — not the whole case domain. A role that needs to distribute contracts gets contract:distribute — not contract:approve. This is a conscious departure from a coarser earlier model in which holding one verb in a domain tended to leak the others; the independent-verb model (§10.2) is what makes tight scoping possible.

10.5.2 The coarse fallback, and where it stops

For backward compatibility and operational smoothness, most routine view/add/edit routes accept *either* the fine-grained per-domain verb *or* a coarse `lex:read / lex:write`. The requester and junior roles carry a coarse `lex:read` (and operational roles a `lex:write`) so that own-scope reads and writes keep working across the platform. But — as Chapter 9 sets out in detail — this coarse fallback is *stripped* from the elevated approve/close control points and from the sensitive status-transition routes. There, only the exact per-domain `approve / close` key opens the door. The coarse permission is a convenience for the everyday; it is never a skeleton key for the decisions that matter.

10.5.3 Role-scoped visibility through org-RBAC

Beyond *what verbs* a role holds, Watheeq scopes *which records* a user can act on through an **organisation-scoped RBAC layer** (`RequireOrgVerb`, in `internal/lex/middleware`). On destructive and administrative routes, after the coarse capability gate has run, this inner gate checks that the actor holds an org role binding that authorises the verb *for the specific target entity's branch of the organisation*. A user who is, say, an approver within one business unit does not thereby gain approval authority over another unit's records unless a binding in the target entity's ancestry authorises it.

This gate, too, is fail-safe: a registry lookup error on a destructive route fails **closed** (a registry outage must not silently permit a delete), and a target entity with *no* authorising binding anywhere in its ancestry is treated as **unsatisfied** — the action is denied rather than allowed by default, so an unconfigured org node cannot be used to escalate. Privileged platform operators (holders of the admin wildcard) are governed by the coarse layer and bypass the org gate, matching the platform's wildcard model. The net effect is *responsibility that is scoped to the part of the organisation a person is*

actually responsible for**.

10.6 Knowing your own authority: the effective-permission contract

A recurring practical need is for a user — and the user interface acting on their behalf — to know *exactly what they are entitled to do*, so that the application can present the right actions and hide the ones the user cannot perform. Watheeq serves this through the **effective-permission contract** at GET /api/v1/lex/me.

Two design points make this contract trustworthy:

95. **It is computed from the same source the enforcer uses.** The effective permissions are produced by EffectivePermissions, which looks each of the caller's role slugs up in the authoritative in-code permission map and applies the **exact same** verb-expansion (expandGrants) that the server's authorisation checks apply. There is no separate, forkable copy of the logic that could drift. What /lex/me reports you can do is, by construction, what the server will actually let you do.

96. **It is a UX aid, not an authorisation path.** The endpoint exists so the client can render an accurate, role-scoped experience — the persona's landing page, the actions it should offer, the navigation it should show. It does **not** replace server-side enforcement. Every route continues to enforce authority independently at the point of action. The active persona is a presentation model; the backend remains the sole authority. This separation matters legally: the interface is a convenience, but the *control* is server-side and cannot be defeated by manipulating the client.

The endpoint is also designed for **discoverability over silent denial**: a caller who holds no legal-affairs role is not met with a bare 403; they receive a well-defined "no legal role assigned" state and are directed to a neutral landing. Access is scoped, but the boundaries are communicated rather than hidden.

10.7 How access control maps to accountability

The final point ties the three chapters together. Access control in Watheeq is the operational face of **accountability**:

- Because roles map to real offices and carry least-privilege authority, **what a person did** can always be tied to **what their office is responsible for**.
- Because verbs are independent and the elevated verbs are narrowly held, **who could have approved or closed a given record** is a small, known, and defensible set of people.
- Because the coarse fallback stops at the control points, no one outside that set could have reached those decisions through a side door.
- Because org-RBAC scopes visibility and action to the relevant part of the organisation, authority does not leak across business-unit boundaries.
- Because the effective-permission contract is computed from the enforcement map itself, the authority a person is **shown** is the authority they **have**.

For a Saudi legal reader assessing the platform, the through-line is this: Watheeq does not merely **record** who did what. Through its role model, its least-privilege scoping, its independent verbs, and its server-side enforcement, it **constrains** who **could** have done what — and it does so in a way that mirrors the organisation's own chart of legal responsibility. When the three controls of these chapters operate together — cryptographic Delegation-of-Authority (Chapter 8), Segregation of Duties (Chapter 9), and role-based access control (Chapter 10) — the platform can answer, with evidence rather than assertion, the three questions that matter most about any legal act: **who was authorised to do this, did they and no one improper actually do it, and could they prove their authority?**

11. Confidentiality & Legal Privilege

11.1 Purpose and Scope of This Chapter

A legal department's most valuable — and most sensitive — asset is the confidential and legally-privileged material it holds: draft contracts before signature, counterparty commercial terms, negotiation positions, investigation files, legal opinions, and communications between in-house counsel and the business it advises. If that material is exposed, mishandled, or accessed by someone without a legitimate need, the consequences are not merely operational; they can waive privilege, breach a non-disclosure obligation, or contravene the Kingdom of Saudi Arabia's Personal Data Protection Law (PDPL) and its Implementing Regulations.

This chapter explains, in business-and-legal terms, how the Watheeq Legal Suite (the Clario360 legal application) protects confidential and privileged material. It addresses:

- **Field-level AES-256 encryption** of the most sensitive contract data at rest;
- **Need-to-know access control** through a granular, role-based and organisationally-scoped permission model;
- **Viewer redaction and legal holds** that constrain what a given user may see or change;
- **Custody chains and audit trails** that record who did what, and when;
- **The handling of attorney-client privileged content** within the platform, and the honest limits of what a software control can and cannot do about privilege.

Throughout, the description reflects controls that are actually implemented in the platform's code. Where a protection is an operational or contractual matter rather than a code control — for example, key-custody attestation in a production cloud region — that distinction is stated plainly so that a reviewing lawyer can allocate the residual risk correctly.

11.2 The Confidentiality Model at a Glance

Watheeq protects confidential material through **four independent, layered controls**. No single control is relied upon alone; each is designed to remain effective even if another is bypassed.

Layer	Control	What it protects against
1. At rest	Field-level AES-256-GCM encryption of the most sensitive contract fields	Database theft, backup exfiltration, direct database access by an operator
2. Access	Need-to-know RBAC + per-tenant + organisational scoping	A logged-in user reaching matters or fields outside their remit
3. Integrity / mutability	Legal-hold guard and contract state machine	Unauthorised deletion, alteration, or premature disposal of material under preservation
4. Accountability	Event/audit trail and versioning	Undetected access or change; inability to reconstruct "who saw what"

The remainder of this chapter takes each layer in turn.

11.3 Layer 1 — Field-Level Encryption of Sensitive Contract Data

11.3.1 What is encrypted, and why field-level

WatheeQ does not rely solely on the database's own transparent disk encryption. It additionally performs **application-level, field-level encryption**: the sensitive values are encrypted by the application **before** they are written to the database, and decrypted **only** when the application reads them back on behalf of an authorised request. This is a materially stronger posture than disk-level encryption alone, because the protected values remain ciphertext even to anyone who obtains a copy of the database, a backup, or a replica.

The fields encrypted at rest are the ones that carry the most confidentiality- and privacy-sensitive content of a contract record:

Field	Nature of the data	Legal sensitivity
document_text	The full extracted text/body of the contract	Core confidential and potentially privileged content
party_b_entity	The counterparty legal entity details	Commercial confidentiality; may be personal data
party_b_contact	Counterparty contact / personal details	Personal data under PDPL
payment_terms	Commercial payment terms	Commercial confidentiality

The same encryption custody is reused for other sensitive material handled by the service, including the integration mailbox ingest secret, investigation content, integration-endpoint credential configuration, and settlement data — so the protection is applied consistently across the confidential surfaces of the suite, not only to the contract table.

11.3.2 The cryptography actually used

The encryption is **AES-256 in Galois/Counter Mode (AES-256-GCM)** — an authenticated encryption scheme. This matters legally as well as technically:

- **Confidentiality:** AES-256 is the symmetric cipher standard accepted for protecting sensitive data.
- **Authenticity / tamper-evidence:** GCM produces an authentication tag. If a stored ciphertext is altered (for instance, by someone editing the database directly), decryption **fails** rather than silently returning corrupted or attacker-chosen plaintext. Tampering is therefore detected, not absorbed.
- **Non-reuse:** each encrypted value is sealed with a fresh, random 96-bit nonce, so encrypting the same plaintext twice does not produce the same ciphertext, and patterns cannot be inferred from the stored values.

Each encrypted value is stored with a version-tagged envelope prefix (enc:v1:) followed by the base64 encoding of the nonce and the sealed ciphertext-and-tag. The version prefix is deliberate: it allows the encryption scheme to be upgraded in future (v2, and so on) without ambiguity about how any given stored value was produced.

11.3.3 Backward compatibility and legacy plaintext

Values that do **not** carry the enc:v1: prefix are treated as legacy plaintext and returned unchanged on read. This is a pragmatic migration property: it allows records written before encryption was

enabled to remain readable while newly-written records are encrypted. For a lawyer's purposes, the salient point is this: **the presence of the prefix is the reliable signal that a value was encrypted at rest**. A data-handling review or DPA due-diligence exercise should confirm that, for the relevant tenant and time period, sensitive fields carry the encryption envelope. Re-encryption is idempotent — a value that is already encrypted is not double-encrypted — so remediation (encrypting historical rows) can be run safely.

11.3.4 Key custody: three modes, one of which is forbidden in production

Where the encryption key lives is the single most important question a reviewing lawyer should ask, because encryption is only as strong as the custody of its key. Watheeq supports three configurable key-custody modes, controlled by the `LEX_CONTRACT_FIELD_ENCRYPTION_MODE` setting:

Mode	Key custody	Intended use	Real AES-256-GCM?
off	No encryption	Development/test only	No (plaintext at rest)
software	32-byte AES-256 key supplied to the process (base64) via <code>LEX_CONTRACT_FIELD_ENCRYPTION_KEY</code> ; held in process memory	In-process software custody	Yes
external	32-byte key sourced at runtime from a file the deployment mounts from an external key store (e.g. a KMS/Vault secret surfaced by an External-Secrets/CSI driver); key material lives out of process	Externally-custodied (KMS/Vault) deployment	Yes

Two design decisions here are worth a lawyer's attention because they change the risk allocation:

97. **Encryption is on by default, and cannot be silently disabled in production.** The service default is software (real encryption), not off. More importantly, the configuration validator enforces a **security floor**: a deployment running under a **protected** (non-development) profile — that is, production, staging, qa, uat, or any label other than development/dev/local/test/empty — **will refuse to start** if field encryption is set to off, or if it is set to software/external without a usable key. This is a fail-fast guard: a production Watheeq that boots at all is a Watheeq with PII field-encryption enabled. A misconfiguration cannot quietly leave sensitive data in plaintext; it stops the service instead. The allowlist is deliberately conservative — a **typo** in the environment label is treated as a protected profile, so an unrecognised value never disables encryption by accident.
98. **The ``external`` mode is an honest seam, not a marketing claim.** In external mode the key is read from a mounted file that a deployment populates from an external key store, and the resolver re-reads on each use so a rotated (remounted) key is picked up. The code performs **the same real AES-256-GCM** regardless of mode — the difference is purely **where the key lives**. The platform is explicit that **production KMS-region attestation remains an infrastructure and audit gate**, i.e. the assurance that the external key is genuinely held in a hardware-backed, in-Kingdom

KMS is an operational control that must be verified in the deployment, not something the application code can prove by itself.

Reviewer's takeaway. For a sovereign, in-Kingdom deployment holding privileged material, external mode backed by a KSA-resident KMS/Vault (with region attestation verified operationally) is the posture to specify. software mode is genuinely encrypted but custodies the key in the application's own configuration/memory, which is a weaker separation of duties. off is a development convenience that the platform itself forbids outside development.

11.3.5 What field-level encryption does **not** do

To set expectations honestly:

- It protects the **enumerated sensitive fields**. Non-sensitive columns (e.g. contract title, status, dates) are stored in the clear so they remain searchable and reportable. A DPA/data-map exercise should treat title and description as clear-text metadata.
- It protects data **at rest**. Data in transit is protected separately by transport security (TLS) at the gateway and between services.
- It does **not** substitute for access control. A user who is authorised to read a contract will see the decrypted values, exactly as intended. Encryption defends against unauthorised access to the **storage**; the next layer defends against unauthorised access by **logged-in users**.

11.4 Layer 2 — Need-to-Know Access Control

11.4.1 Tenant isolation is the outer boundary

Every record in Watheeq is scoped to a `tenant_id`. A tenant is the customer organisation (for the reference deployment, "Abdullah Al Othaim Investment Company"). All queries, all seeded configuration, and all provisioning are tenant-scoped. One customer's legal matters, contracts, and personal data are not reachable from another customer's context. For a DPA, this is the primary logical-isolation control between controllers.

11.4.2 Role-based access and the 14-role separation-of-duties matrix

Within a tenant, access follows a **need-to-know, role-based** model. Watheeq provisions a **14-role legal separation-of-duties (SoD) matrix** per tenant (seeded idempotently by the legal-affairs role seeder), including roles such as legal-system-admin and the legal-director / reviewer / approver roles that the escalation ladder walks. Roles are seeded into the shared IAM database (`platform_core`) and are **also** enforced through the application's code-level permission map, so that even when the central roles table is temporarily unreachable the permission checks still hold (the provisioning of roles into `platform_core` is explicitly non-fatal for exactly this reason).

Access is expressed through **granular permission slugs** rather than a single coarse "legal user" grant. The permission vocabulary is intentionally fine-grained and **per-domain, per-verb**. For each legal domain — cases, contracts, investigations, settlements, consultations, requests, documents — there are independent verbs, for example `lex:contract:view | add | edit | distribute | approve | close` and `lex:case:view | add | edit | assign | approve | close`, alongside approval-specific grants (`lex:approval:read | write | admin`) and governance/configuration grants (`catalog, SLA, escalation,`

notification, role, integration, security, audit and report slugs). Coarser lex:read / lex:write grants exist as a fallback layer, but the design intent — reflected in the code that resolves these slugs — is that reading, authoring, assigning/distributing, approving and closing are **independent** grants that are **not** rounded up to the highest authority: being able to see a matter does not imply being able to approve it, and being able to author does not imply being able to sign off.

The frontend does not trust the access token alone for this: because the JWT carries only the coarse permission slugs, the application hydrates the user's **granular effective permissions** from a dedicated endpoint (GET /api/v1/lex/me) and unions them into its permission checks. This means the user interface presents only the actions the user is actually entitled to perform — a need-to-know surface, not a full menu with server-side rejection after the click.

11.4.3 Segregation of duties: author ≠ approver

Confidentiality is not only about **who can read**; it is also about ensuring that no single individual can both create and unilaterally bless a sensitive item. Watheeq enforces **dynamic separation of duties**: the author of an item is prevented from being its approver, evaluated per record at decision time. An optional stricter posture requires **two rounds of distinct approvers**. Static separation-of-duties exclusions are also expressed in the seeded role matrix (certain roles are mutually exclusive for one person). For privileged and high-value contracts, this is the control that prevents a lone actor from moving material through the lifecycle without independent review.

Note, importantly, that in the reference deployment the demonstration administrator account is itself SoD-gated: it can **browse** but is refused (403) on approve/close actions unless it is assigned a genuine legal role. The point for a reviewer is that "admin" is not a bypass of the legal control model — the SoD constraints bind the administrator too.

11.4.4 Organisational scoping — need-to-know within a tenant

Beyond role, access is further narrowed by **organisational scope**. Watheeq models the customer's organisation as a registry (Company → Shared-Services Unit → Legal Department → sections) with responsibility roles that the L1/L2/L3 escalation ladder walks, and the platform includes an organisation-scoped RBAC resolver. In practice this supports the principle that a member of one legal section should see that section's matters — not every matter in the enterprise — which is the essence of a need-to-know control for a large legal function.

11.5 Layer 3 — Redaction, Mutability Controls and Legal Holds

11.5.1 Viewer-facing constraints — how confidentiality between users actually works

It is important to be precise here, because "redaction" can mean different things. Watheeq's confidentiality between users is **all-or-nothing at the document/record level**, achieved by access control and encryption — **not** by presenting partially-redacted views of a single document to lesser-privileged readers. In other words:

- A user **without** the entitlement to a contract, matter, or document does **not** receive it at all: the access control refuses the read, and the encrypted fields are never decrypted for an unauthorised path. From that user's perspective the confidential content is simply absent.

- A user **with** the entitlement receives the full content. The platform does **not** implement a feature that shows, say, a privileged legal memo to a junior user with the privileged passages blacked out. Confidentiality of **content between roles** is therefore delivered by **need-to-know RBAC + organisational scoping + encryption + audit**, not by partial in-document redaction. A reviewing lawyer should understand and rely on the all-or-nothing model rather than assume field-by-field redaction of a shared document.

For completeness, the one place the platform **does** perform automatic redaction is **credential hygiene**: integration secrets (API keys, bearer tokens) are masked on read in the administration console, event log and dead-letter queue so that secrets never leak — this protects sub-processor credentials, not legal document content.

Where a redline/diff view is presented, the platform operates on the contract text within the authorised session; it is not a mechanism for exposing content to users who lack access to the underlying contract.

11.5.2 Legal holds — preventing spoliation

A distinctive and legally significant control is the **legal-hold guard**. When a contract (or other subject) is placed under an **active legal hold**, the platform refuses mutating operations that would destroy or alter the held material — a contract under an active legal hold cannot be deleted, cancelled, or terminated, and the review-desk service similarly refuses edits while a hold is in force (enforced through a shared "ensure-mutable" preservation check). This is the software embodiment of a litigation-hold / preservation obligation: it converts a policy ("do not destroy documents subject to a dispute") into an enforced technical constraint that individual users cannot casually override.

11.5.3 The contract state machine as a control

Mutability is further disciplined by a **validated contract state machine** (described in detail in Chapter 12). Status changes are not free-form; only defined transitions are permitted, and each transition records the prior status, the timestamp, and the acting user. From a confidentiality-and-integrity standpoint, this means the **history** of a contract's legal status is captured and constrained, not merely its current value.

11.6 Layer 4 — Custody Chains and Audit Trail

11.6.1 Recording who did what

WatheeQ maintains **append-only, insert-only audit trails** per legal domain — separate audit tables for cases, matters, consultations, investigations, litigation, requests, settlements, approval policies, and SLA/status transitions. These tables have no update or delete path, which makes the trail **tamper-evident**: an audit row records the action, the from/to status, a structured detail payload, the acting user, and the timestamp, and it is written **inside the same database transaction** as the mutation it records, so the change and its audit entry succeed or fail together. Approval policies additionally keep an **immutable version history**. The service also emits domain events (through its in-process event bus, with an optional durable Kafka delegate) that drive the activity timeline and notifications; the tamper-evident record of record, however, is the append-only audit tables, not the event stream.

Records also carry provenance directly — creator, owner, legal reviewer, status-change actor and time, and version lineage. Contract documents are versioned (ContractVersion), each version carrying a **content hash**, file identity, uploader and upload time, and an optional change summary. The content hash provides tamper-evidence for the stored document: a change to the bytes changes the hash, so a version's integrity can be checked after the fact. Attachment supersession is recorded (versioned, marked superseded) rather than overwritten, and document/link deletions remove the **link only**, preserving the underlying document. Together these give a **chain of custody** for a contract: which version existed when, who uploaded it, and what changed.

11.6.2 Obligations and reminders leave a trail too

Where obligations are extracted and tracked (Chapter 12), the reminder and escalation activity is persisted through an outbox with per-item status, timestamps, and provider results — so the record of **notice given** (a matter that can itself have legal consequence) is preserved rather than fire-and-forget.

11.6.3 Honest limits of the audit posture

Two points of candour for the reviewer:

- The legal suite's audit tamper-evidence rests on **database-level append-only (insert-only) tables** plus **per-version content hashes** and **link-only deletes** — a strong posture, but not the same as cryptographic hash-chaining or write-once-read-many (WORM) object-lock. The platform **does** provide true WORM/object-lock immutability elsewhere (in its disaster-recovery/compliance evidence subsystem), and whether the legal suite's audit stream is additionally written to such a WORM-backed sink in a given deployment is an **operational configuration to confirm**, not an automatic property of the legal-suite code path.
- The audit trails capture **mutations and workflow decisions** (create, edit, status change, approve, close, priority change). They do **not**, by default, record a per-read "who **opened** this privileged document, and when" access log. If the customer's regulatory posture requires read-level custody of privileged material, that requirement should be raised explicitly and satisfied through the deployment's logging configuration rather than assumed present.

Both are flagged so they can be specified explicitly in a contract or DPA where the customer's regulatory posture requires it.

11.7 Handling of Attorney–Client Privileged Content

11.7.1 A confidentiality classification exists — including a "privileged" level

WatheeQ documents carry a **four-level confidentiality classification** — public → internal → confidential → privileged — recorded per document, filterable, and rolled up in reporting (so a folder shows, for example, how many privileged documents it contains). This gives the legal team an explicit, first-class way to **mark** attorney–client privileged material as such.

Two points of candour a reviewing lawyer must appreciate:

99. **The classification is a sensitivity label, a filter, and a reporting dimension — it is not, by itself, an access-control gate.** A document classified privileged is not automatically restricted to a

narrower audience than one classified confidential at the enforcement layer; read access is gated by the `lex:document:view` permission (and organisational scoping where configured), not by the label value. The label signals sensitivity to humans and to the UI/reporting; **RBAC, org-scoping and encryption do the actual gating**.

100. Consequently, the operational discipline of assigning the right people the right roles (and scoping them to the right organisational units) is what keeps privileged material in the correct circle. The label supports and evidences that discipline; it does not substitute for it.

11.7.2 How privilege is protected in practice

The platform does **not** implement a distinct, code-level "privileged" flag that changes the storage or cryptography of an item, nor one that by itself narrows read access. Instead — and this is the honest and, in our assessment, appropriate design — **privileged and confidential content is protected by the general controls described above**, applied rigorously:

101. **Encryption at rest** protects the substance of privileged communications and opinions where they live in the encrypted fields (notably the `contract body/document_text` and the sensitive counterparty fields), so that privileged text is not readable from storage.
102. **Need-to-know RBAC and organisational scoping** ensure that privileged material is disclosed only to the individuals whose role and organisational position give them a legitimate need — the technical analogue of the "circle of confidentiality" that preserves privilege.
103. **Legal holds** prevent destruction of privileged material that must be preserved.
104. **Audit and versioning** allow the organisation to demonstrate, if challenged, the controlled handling of the material.

11.7.3 What this means for the lawyer

For a Saudi lawyer weighing whether the platform is suitable to hold privileged and highly confidential material, the practical conclusions are:

- **Privilege is preserved by controlling the circle of access, not by a magic label.** The platform gives you the tools to keep the circle tight (granular roles, SoD, org scoping) and to keep the content unreadable outside that circle (encryption). Preserving privilege therefore depends on the customer's own role assignments being disciplined — a governance responsibility, supported by the platform's fine-grained controls.
- **Operator access should be constrained contractually and technically.** Because in software mode the key is custodied by the application, the strongest posture for privileged content is external (KMS/Vault) key custody with in-Kingdom region attestation, so that even a privileged operator cannot trivially read the sensitive fields from storage. This should be an explicit term.
- **The confidentiality obligations in the SaaS agreement (see Chapter 13) are the backstop.** Technical controls reduce the risk of exposure; the contractual confidentiality, sub-processor, and audit terms allocate the residual risk and give the customer enforceable remedies.

11.7.4 Recommended configuration for privileged workloads

Control	Recommended setting for privileged/highly-confidential use
Field encryption mode	external (KMS/Vault-custodied key), region attestation

	verified operationally
Key rotation	Rotate the external key on a defined cadence (the resolver re-reads on rotation)
Roles	Assign the narrowest legal role that fits each user; do not use the admin role as a working legal identity
Classification	Mark privileged material privileged (remembering the label signals sensitivity but does not itself gate reads — pair it with tight role/org scoping)
SoD	Enable the stricter two-round distinct-approver posture for the most sensitive matters
Legal holds	Apply holds proactively to disputed or pre-litigation matters
Audit sink	Confirm the deployment writes the audit stream to an immutable/WORM-backed sink

11.8 Chapter Summary

WatheeQ protects confidential and privileged legal material through four layered, independent controls: **AES-256-GCM field-level encryption** of the most sensitive contract data at rest (on by default, forbidden to be disabled in production, and capable of externally-custodied keys); **need-to-know access control** via a 14-role separation-of-duties matrix, granular permissions, and organisational scoping; **mutability controls** including legal holds and a validated state machine that prevent spoliation and unauthorised change; and **custody chains** through versioning, content hashing, and an event/audit trail. Attorney–client privilege is preserved not by a bespoke flag but by the disciplined application of these controls to keep the circle of access tight and the content unreadable outside it. The residual assurances — production key-custody attestation and WORM-backed audit — are correctly framed as operational and contractual matters, and are addressed as reviewable terms in Chapter 13.

12. Contract Lifecycle — Legal Controls

12.1 Purpose and Scope of This Chapter

This chapter describes the **legal control points** that Watheeq places across the life of a contract — from a first draft to archival — viewed through the lens of legal risk. It is written for a lawyer who needs to understand **where** the system requires review, sign-off, and evidence, and **what** happens automatically* to reduce the risk of missed obligations, unauthorised commitments, and lapsed renewals.

The controls described here are those implemented in the platform: the contract **state machine**, the **legal-review gate**, the **approval and delegation-of-authority** framework, **obligation capture and tracking**, **execution/signature**, **renewal and expiry monitoring**, and **archival/preservation**. Where a control depends on the customer's own configuration or on operational deployment, that is noted.

12.2 The Contract as a Governed State Machine

12.2.1 Defined statuses

A Watheeq contract is not a free-form document with an ad-hoc label; it is a record that moves through a **defined set of statuses**, each of which carries a legal meaning:

Status	Meaning
draft	Under initial preparation; not yet submitted for review
internal_review	Under review by the business/owning function
legal_review	Under review by the legal function (the legal gate)
negotiation	In negotiation with the counterparty
pending_signature	Terms settled; awaiting execution/signature
active	Executed and in force
suspended	Temporarily suspended
expired	Term ended
terminated	Ended by termination
renewed	Superseded by a renewal
cancelled	Abandoned before execution

12.2.2 Only defined transitions are permitted

Crucially, movement between statuses is **constrained by a validated transition map**. A contract cannot jump arbitrarily from one status to another; only the transitions the platform recognises as legitimate are allowed, and an attempt to make an invalid transition is rejected with an explicit error. The permitted transitions encode the intended legal workflow:

- draft → internal_review or cancelled
- internal_review → legal_review or back to draft
- legal_review → negotiation, back to internal_review, or back to draft
- negotiation → pending_signature, cancelled, or back to draft
- pending_signature → active or cancelled
- active → suspended, terminated, expired, or renewed
- suspended → active or terminated

- expired → renewed

Two properties of this map are legally significant. **First, a contract cannot become `active` except from `pending\_signature`.** There is no path that makes a contract live without passing through the settled-terms/awaiting-signature stage — the platform structurally prevents "accidental activation" of an unexecuted contract. **Second, the review path is explicit and reversible:** legal review can send a contract back to internal review or to draft, capturing the reality that legal sign-off is a gate that can **reject**, not merely a rubber stamp. Each transition records the **previous status**, the **timestamp of change**, and the **acting user**, so the lifecycle is auditable after the fact.

12.3 Legal Control Point 1 — The Legal Review Gate

12.3.1 Where legal review sits

The `legal_review` status is the platform's **explicit legal gate**. A contract reaches it only via `internal_review`, i.e. after the owning business function has done its own first pass, which mirrors sound practice: legal reviews a contract that the business has already sponsored, not raw unfiltered intake.

Each contract carries a **legal reviewer** assignment (`legal_reviewer_id` / `legal_reviewer_name`), so the record shows **who** is accountable for the legal review, and the risk analysis fields (`risk_score`, `risk_level`, `analysis_status`) capture the output of the platform's clause/risk analysis to inform that review. The review desk and compliance-review capabilities support the reviewer's work and are themselves subject to the legal-hold preservation guard (a contract under hold cannot be edited through the review desk).

12.3.2 The gate can reject

Because the state machine permits `legal_review` → `internal_review` and `legal_review` → `draft`, the legal gate is a genuine control: the reviewer can return the contract for rework rather than being forced to either approve or block. This is the difference between a **control point** and a **checkbox**.

12.4 Legal Control Point 2 — Approval and Delegation of Authority

12.4.1 Approval policies with governance

Approval in Watheeq is governed by **approval policies** that are themselves subject to lifecycle governance: the platform supports **approval-policy versioning, audit, conflict handling, expiry, and templates**. This matters legally because an approval control is only trustworthy if the **rules** of approval are themselves controlled — you must be able to show which version of an approval policy was in force when a given contract was approved, and that the policy was not quietly changed to wave something through.

12.4.2 Separation of duties in approval

As described in Chapter 11, approvals enforce **dynamic separation of duties**: the author of an item cannot be its approver, evaluated per record at decision time, with an optional stricter mode requiring **two rounds of distinct approvers**. For contracts above a materiality threshold, or of a

sensitive type, this prevents a single individual from both drafting and approving a binding commitment.

12.4.3 Delegation of Authority (DoA) with cryptographic evidence

Watheeq supports **Delegation of Authority** approval evidence that can be **cryptographically validated**. When the deployment configures a bundle of trusted root certificate authorities (via LEX_APPROVAL_AUTHORITY_TRUSTED_ROOTS_PEM / ..._FILE), approval evidence is required to be presented as a certificate chain plus a detached signature, and is verified strictly: the chain must anchor to a trusted root, the signature must verify, and validity is checked. Offline **revocation checking** can be enabled against a configured list of revoked serials. If no roots are configured, the service falls back to accepting plain-text evidence and logs a warning.

For a lawyer, this is the control that answers "**did the person who approved this actually have the authority to bind us, and can we prove it?**" In its strict (roots-configured) mode, the approval is backed by a verifiable X.509-based signature of the approving authority — a materially stronger evidentiary position than a name in a log. **Reviewer's note:** whether strict cryptographic DoA is **enforced** depends on the deployment configuring trusted roots; a customer that requires it should specify it, because the default fallback accepts plain-text evidence.

12.5 Legal Control Point 3 — Obligation Capture and Tracking

12.5.1 Obligations as first-class records

A contract's legal risk does not end at signature — it lives in the **obligations** the contract creates. Watheeq treats obligations as first-class, tracked records rather than buried clauses. Each obligation has a **type**, a **status**, a **priority**, an **owner**, a **due date**, and reminder/escalation configuration, and can be linked to the originating contract (and clause) and/or a matter.

The obligation **types** are legally meaningful and cover the categories a lawyer cares about:

Category	Types
Commercial/performance	contractual, payment, delivery
Timing	renewal, notice, condition_precedent
Assurance	reporting, compliance, covenant, regulatory
Catch-all	other

The obligation **lifecycle** (open → in_progress → blocked / completed / waived / cancelled) captures not only completion but the legally-relevant states of being **blocked** or formally **waived** — a waiver is recorded, not merely implied by inaction.

12.5.2 Extraction and tracking

Obligations can be **extracted** from contract text (the analyzer produces an extraction result recording what was created and what was skipped, and why), and are then **tracked and reminded** against their due dates. Reminders and escalations are configurable per obligation (lead days, escalation lead days, escalation target) and are dispatched through a persisted **outbox** with per-item status and provider results — so that the fact of a reminder or escalation having been issued is itself recorded evidence. A reminder-planning path computes upcoming notifications over a horizon and de-duplicates them.

For a lawyer, the value is that **notice and renewal obligations do not depend on someone remembering** — they are captured as due-dated records with owners, reminders, and an evidentiary trail of the notices given.

12.6 Legal Control Point 4 — Execution and Signature

12.6.1 The signature gate

A contract transitions to active only from pending_signature, and the record captures the **signed date** (signed_date). Execution is therefore a distinct, recorded gate, not an implicit consequence of editing a field.

12.6.2 E-signature integration

WatheeQ includes a **signature provider seam**: a deterministic default and an optional external HTTP signature provider (configured via LEX_SIGNATURE_PROVIDER_*, requiring an absolute HTTPS endpoint and API key when enabled). Among the integration connectors, an **e-signature/`emdha`** connector is present in the integration catalog. Consistent with sovereign-deployment discipline, the e-signature, Najiz (Ministry of Justice portal), and Nafath (national identity) connectors are classified as **government-gated** — intended to run in a sandbox environment before production activation — whereas connectors such as SSO/SAML, HR/SCIM, e-archive, email and internal are production-ready. This is significant for a lawyer because it means the platform does not silently assert live integration with a government e-signature or court portal; those are gated behind an explicit go-live in the appropriate environment.

12.7 Legal Control Point 5 — Renewal and Expiry Monitoring

12.7.1 The renewal/expiry fields

Each contract carries the data needed to manage its end-of-life: effective_date, expiry_date, renewal_date, an auto_renew flag, and a renewal_notice_days value (the notice window before renewal). These are precisely the fields on which auto-renewal ("evergreen") risk turns.

12.7.2 Background monitors

WatheeQ runs **background monitors** that sweep these dates on a schedule and raise de-duplicated alerts:

Monitor	Default cadence	Purpose
Expiry monitor	Hourly (LEX_EXPIRY_MONITOR_INTERVAL)	Detect approaching/lapsed expiry
Renewal reminder	Every 6 hours (LEX_RENEWAL_REMINDER_INTERVAL)	Alert on contracts due for renewal within the relevant window
Compliance monitor	Every 6 hours (LEX_COMPLIANCE_MONITOR_INTERVAL)	Compliance/obligation sweeps

The renewal reminder evaluates contracts falling into the relevant expiry/renewal bucket and inserts **de-duplication-aware** alerts (so a customer is not spammed with duplicate open alerts for the same contract), scoped per tenant. For a lawyer, the control point is that **auto-renewal and expiry are**

actively surveilled: a contract that will auto-renew, or that is about to lapse, generates an alert within the notice window rather than surfacing only when it is too late to act.

12.8 Legal Control Point 6 — Archival, Preservation and Disposal Discipline

12.8.1 Versioning as the archival substrate

Every material version of a contract document is retained as a ContractVersion with a **content hash**, file identity, uploader, and timestamp. This gives an evidentiary archive of what the contract said at each stage, not merely its latest form — important for disputes about which version was agreed.

12.8.2 Preservation via legal holds

Disposal is disciplined by the **legal-hold guard**: a contract under an active legal hold cannot be deleted, cancelled, or terminated. This converts a records-preservation policy into an enforced constraint, and directly supports a litigation-hold obligation. Deletion in the ordinary course is a **soft delete** (deleted_at), preserving the record rather than physically erasing it, which supports both recoverability and defensible disposition.

12.8.3 Honest limits

The platform provides the *substrate* for records retention (versioning, hashing, soft delete, holds) and, at platform level, WORM-capable immutable storage. A **formal, time-based retention schedule** (e.g. "retain executed contracts for N years, then dispose") is a **policy configuration and operational matter**, not an automatic default. A customer with statutory retention obligations should specify the retention schedule and confirm how it is enforced and evidenced in their deployment.

12.9 The Lifecycle as a Sequence of Legal Controls — Summary View

Lifecycle stage	Status(es)	Legal control	Evidence produced
Preparation	draft	Owner assignment; state-machine entry	Creator, owner recorded
Business review	internal_review	Business sponsorship before legal	Status history
Legal review	legal_review	Legal gate (can reject); risk analysis; reviewer of record	Reviewer, risk score, status history
Negotiation	negotiation	Redline/versioning	Versions with content hashes
Approval / DoA	(policy-driven)	SoD (author≠approver); versioned policy; cryptographic DoA evidence	Approval record; verifiable signature (strict mode)
Execution	pending_signature → active	Signature gate; e-sign seam	signed_date; provider result
In force	active	Obligation tracking; reminders/escalation	Obligation records; notice outbox
End of life	expired / terminated / renewed	Expiry/renewal monitoring; hold on disposal	Alerts; hold enforcement; soft delete
Archival	(retained)	Versioning; preservation	Version chain; content hashes

12.10 Chapter Summary

Watheeq places legal controls at every material point of the contract lifecycle. The **state machine** structurally prevents illegitimate jumps — most importantly, a contract cannot go live without passing the signature stage. The **legal-review gate** is a genuine, rejecting control. **Approval** is governed by versioned policies, dynamic separation of duties, and — where configured — cryptographically-verifiable Delegation-of-Authority evidence. **Obligations** are captured as tracked, due-dated records with an evidentiary reminder trail. **Renewal and expiry** are actively surveilled by scheduled monitors that raise de-duplicated alerts within the notice window. **Archival and disposal** are disciplined by versioning, content hashing, soft deletion, and legal holds that prevent spoliation. The two matters that remain the customer's to specify — strict cryptographic DoA enforcement and a formal retention schedule — are flagged as configuration/operational decisions and are natural terms for the engagement contract addressed next.

13. SaaS Terms & Conditions — Considerations for the Lawyer

Nature of this chapter. This chapter is **not** a contract and **not** legal advice. It is a structured checklist of the terms a customer's lawyer should expect to see, negotiate, and satisfy themselves on before entering a sovereign legal-SaaS engagement for the Watheeq Legal Suite. Each section explains **why** the term matters and **what to look for**, and — where the platform's actual capabilities bear on the term — points to the relevant control so the lawyer can align the paper with the product. Specific commercial figures, party names, and binding language are deliberately omitted; those belong in the negotiated agreement.

13.1 How to Read This Chapter

A SaaS agreement for a legal-and-compliance platform is unusual in that the **subject matter of the service is itself confidential and privileged data**. That raises the stakes on the data-protection, confidentiality, and data-return terms well above a typical software subscription. The considerations below are organised as a review sequence:

105. Service description and licensing model
106. Data Processing Agreement (DPA) elements
107. Sub-processors
108. Intellectual property and ownership of customer data
109. Liability and indemnity framing
110. Confidentiality
111. Term, termination, and data return/deletion
112. Governing law and dispute resolution
113. A consolidated reviewer's checklist

13.2 Service Description and Licensing Model

13.2.1 What the service is

The lawyer should confirm the agreement accurately describes the service the customer is actually buying. Watheeq is licensed as an **application entitlement** within the broader Clario360 platform. The platform's entitlement model distinguishes **suites** (e.g. the Cyber, SIEM, Data, DataStream suites) from **apps**, and **Watheeq is an app-level entitlement keyed `app.watheeq`**. A customer can therefore subscribe to Watheeq specifically, rather than being forced to license the entire platform — a **per-suite / per-app subscription** model. The agreement's service description and order form should name the entitlement(s) being purchased and the seat count, because these are the values the licensing engine enforces.

13.2.2 Licensing states and enforcement — what to expect operationally

The lawyer should understand how the licence is enforced, because it affects service continuity:

- A tenant holds **one licence against a plan**, with a status of active or suspended, a seat count, a start and expiry date, and a **grace period** (grace_days).

- The **effective state** is computed from time, not merely stored: active while before expiry, in_grace during the grace window after expiry, expired after grace, or suspended if administratively suspended. A licence therefore cannot be "active in the database while expired in reality."
- At the gateway, access to a licensed capability is checked against the entitlement. A denial returns **HTTP 402 Payment Required** — the machine-readable signal that the tenant lacks the required entitlement. The enforcement is **fail-closed** on plan-gated routes: if the gateway cannot verify entitlement (e.g. it cannot determine the tenant), it denies rather than allowing.

For the agreement, this means two things worth negotiating explicitly: **(a) the grace period** (its length and what functionality remains available during grace), and **(b) the consequence of expiry/suspension** — in particular, that the customer must retain the ability to *export and retrieve its data* even when the licence is expired or suspended (see §13.8 on data return). A lawyer should resist a construction in which non-renewal instantly locks the customer out of its own contracts and matters.

13.2.3 Onboarding and provisioning

The lawyer should also understand what happens at onboarding, because it determines when data processing begins. A customer can **self-serve** a Watheeq-scoped tenant; on subscription the platform applies the reusable "**Legal Affairs starter template**" — the service catalog and SLAs, the KSA working calendar, the request-approval policy templates, and the organisational registry — and seeds the legal role matrix. Critically, a real onboarded client is provisioned **without** sample/demo data (that is opt-in and reserved for demo tenants), so the tenant starts clean and the customer creates its own matters. The agreement should reflect that the customer, not the vendor, populates the tenant with its confidential content, which is relevant to the controller/processor characterisation in the DPA.

13.3 Data Processing Agreement (DPA) Elements

Because Watheeq stores personal data (counterparty contacts and other individuals named in contracts and matters) and does so in a Saudi legal context, a **DPA aligned to the PDPL and its Implementing Regulations** is essential. The lawyer should ensure the DPA (as a schedule to the main agreement) addresses at least the following.

13.3.1 Roles: controller and processor

Confirm the characterisation. On the facts of the platform — the **customer** decides what contracts and matters to load, populates them, and controls role assignments; the **vendor** provides the processing infrastructure — the customer is ordinarily the **controller** and the vendor the **processor** (with sub-processors). The DPA should say so, and constrain the processor to act only on the controller's documented instructions.

13.3.2 Nature, purpose and categories of processing

DPA element	What to specify (aligned to the platform)
Purpose	Provision of the Watheeq legal/contract/matter management service
Categories of data subjects	Counterparties, their contacts, and individuals named in

	contracts/matters
Categories of personal data	Counterparty entity and contact details (party_b_entity, party_b_contact), and personal data appearing within contract text/matters
Special-category data	To the extent legal matters contain sensitive personal data, address it expressly
Duration	For the term, plus the agreed return/deletion window

13.3.3 Security measures (Article-level detail)

The DPA's security schedule should reflect the platform's actual technical controls, so the paper matches the product. In particular the lawyer can point to, and should require as contractual commitments:

- **Encryption at rest** of the sensitive fields via **AES-256-GCM**, on by default and **forbidden to be disabled in a production profile** (Chapter 11);
- **Key custody** — specify **`external` (KMS/Vault) key custody** for a sovereign deployment, and require **in-Kingdom region attestation** for the key store as an operational commitment;
- **Encryption in transit** (TLS) at the gateway and between services;
- **Access control** — need-to-know RBAC, the 14-role separation-of-duties matrix, and organisational scoping;
- **Audit trail and versioning** — and, if the customer's posture requires it, an express requirement that the audit stream be written to an **immutable/WORM-backed** sink;
- **Tenant isolation** — logical isolation by tenant_id across the service and separate service databases.

13.3.4 Data residency and sovereignty

For a Saudi engagement this is often the decisive term. The platform's legal-suite jurisdiction default is **"Saudi Arabia"** (LEX_ORG_JURISDICTION), and the broader Clario360 platform is designed for sovereign, in-Kingdom operation. The DPA should **require that personal data, backups, keys, and audit records remain resident in the Kingdom**, name the region, and constrain any cross-border transfer to the PDPL's lawful-transfer conditions. Where DR/BYOK/WORM features are engaged, the lawyer should note that certain irreversible sovereign operations (root-key ceremonies, WORM/object-lock provisioning) are, by the vendor's own operational discipline, gated behind a dedicated environment and sign-off — a point that supports, rather than undermines, a residency commitment.

13.3.5 Data-subject rights, breach notification, and assistance

Standard PDPL-aligned DPA obligations the lawyer should confirm:

- **Assistance with data-subject requests** (access, correction, deletion) within defined timeframes;
- **Personal-data-breach notification** to the controller without undue delay, with sufficient detail for the controller to meet its own regulatory notification duties;
- **Assistance with impact assessments** and with regulator engagement;
- **Records of processing** and a **right to audit** the processor's compliance (see §13.4 and §13.6).

13.4 Sub-processors

The customer's data will, in practice, touch third-party services through the platform's **integration connectors**. The lawyer should require a **named sub-processor list**, a **right to object** to new sub-processors, and **flow-down** of the DPA's obligations to each. Based on the platform's integration catalog, the connectors that may act as (or route data to) sub-processors, and their maturity classification, are:

Connector	Function	Maturity / activation posture
SSO / SAML	Identity federation	Production-ready
HR / SCIM	User provisioning	Production-ready
E-archive	Document archiving	Production-ready
Email	Notifications/correspondence	Production-ready
Internal	Internal platform services	Production-ready
e-signature (`emdha`/e-sign)	Electronic signature	Government-gated (sandbox before production)
Najiz	Ministry of Justice court portal	Government-gated (sandbox before production)
Nafath	National identity verification	Government-gated (sandbox before production)

Two points for the lawyer:

114. The **government-gated** connectors (e-sign/emdha, Najiz, Nafath) are explicitly designed to run in a **sandbox before production activation**. The sub-processor terms should reflect that these are activated only on the customer's instruction and after go-live in the appropriate environment — the customer is not silently transmitting data to a government portal by default.
115. Each connector's **secret/credential** is held encrypted at rest (reusing the same field-crypto custody) and supports **rotation**; the DPA/security schedule can reference this when addressing sub-processor credential handling.

13.5 Intellectual Property and Ownership of Customer Data

The lawyer must draw a clean line between the vendor's IP and the customer's data:

- **Customer data** — every contract, matter, obligation, document, and the personal data within — **is and remains the customer's property**. The agreement should say so unambiguously, grant the vendor only the **limited licence necessary to provide the service**, and prohibit any use of customer data for the vendor's own purposes (including, expressly, **not** using customer content to train models). Note that the platform's optional LLM enrichment is **off by default** and gated by AI-governance entitlement; a lawyer should require that any AI processing of customer content is opt-in, governed, and carries the same no-training-on-customer-data commitment.
- **Vendor IP** — the platform software, templates, and the reusable "Legal Affairs starter template" — **remains the vendor's**, licensed to the customer for the term.
- **Configuration and derived data** — role matrices, org registry, workflow configuration created by the customer should follow the customer data, not the vendor IP; confirm the drafting does not accidentally sweep customer configuration into vendor-owned "improvements".
- **Feedback** — if the customer gives feedback, a narrow, royalty-free feedback licence is acceptable, but it should not extend to the customer's substantive data.

13.6 Liability and Indemnity Framing

This section is *framing for negotiation*, not a recommended number.

- **Liability caps.** Expect a cap. For a platform holding privileged and personal data, the lawyer should press for a **higher (or uncapped) carve-out** for the vendor's breach of its confidentiality and data-protection obligations, for a personal-data breach caused by the vendor, and for the vendor's IP indemnity — these are the risks most damaging to the customer and least within its control.
- **Excluded losses.** Scrutinise exclusions of "indirect/consequential" loss; ensure they do not swallow the customer's core remedy for a data breach or confidentiality failure.
- **IP indemnity.** The vendor should indemnify the customer against third-party claims that the platform infringes IP.
- **Data-protection indemnity.** Consider an indemnity (or at least uncapped liability) for regulatory fines/claims attributable to the vendor's processing failures, to the extent permissible under PDPL and Saudi law.
- **Customer-side obligations.** Expect the vendor to require the customer to warrant it has the right to load the data it loads and that its role assignments are its own responsibility — reasonable, given the customer controls need-to-know access.
- **Security-control commitments as warranties.** Where the vendor commits to specific controls (encryption on in production, external key custody, tenant isolation), the lawyer should try to elevate the most important of these from mere description to **warranties**, so that their failure is a breach with a remedy.

13.7 Confidentiality

Given the subject matter, the confidentiality clause is not boilerplate here — it is central.

- **Mutual, but weighted.** Confidentiality should be mutual, but the customer's confidential information includes **privileged and highly sensitive legal material**; the clause should recognise that and impose a correspondingly high standard of care.
- **Alignment with technical controls.** The clause should reference and require the technical confidentiality controls (Chapter 11): field-level encryption, need-to-know RBAC, legal holds, and audit. This aligns the contractual promise with the product's actual behaviour.
- **Operator access.** Address vendor-operator access to customer data directly: it should be minimised, logged, and — for privileged content — practically constrained by external key custody so that operators cannot casually read sensitive fields from storage.
- **Privilege preservation.** State expressly that the vendor's access to, and processing of, customer data does **not** waive privilege, that the vendor will not voluntarily disclose privileged material, and that the vendor will notify the customer of any compelled disclosure (subject to law) so the customer can seek protection.
- **Survival.** Confidentiality obligations must survive termination, and dovetail with the data-return/deletion terms below.

13.8 Term, Termination, and Data Return / Deletion

This is the clause a legal customer should negotiate hardest, because it governs whether the customer can get its own privileged records back.

- **Term and renewal.** Confirm the term, renewal mechanics, and — importantly — that the licensing **grace period** and expiry behaviour (§13.2.2) do not deprive the customer of **read/export access to its data** during a wind-down. A construction where a lapsed licence instantly locks out the customer's contracts is unacceptable for a legal platform.
- **Termination rights.** Ensure the customer can terminate for the vendor's material breach (including a data-protection or confidentiality breach) and, ideally, for convenience with reasonable notice.
- **Data return.** Require a **defined data-return process**: the customer's data (contracts, matters, obligations, documents, and metadata) exported in a **usable, documented format** within a stated window after termination. Because the sensitive fields are encrypted at rest, confirm the export delivers **usable plaintext to the authorised customer** (i.e. decrypted for the legitimate recipient), not opaque ciphertext the customer cannot open.
- **Data deletion.** Require **certified deletion** of customer data (including from backups, subject to a defined backup-expiry window) after the return window, **except** where **legal-hold or statutory retention** requires preservation. The lawyer should reconcile this with the platform's legal-hold behaviour (material under an active hold is not deleted) and with any retention schedule the customer specifies (Chapter 12, §12.8.3).
- **Transition assistance.** Consider a transition-services obligation to help migrate to a successor system.

13.9 Governing Law and Dispute Resolution

- **Governing law.** For a sovereign, in-Kingdom deployment holding Saudi legal data, the natural position is **the laws of the Kingdom of Saudi Arabia** — consistent with the platform's default jurisdiction and residency posture. The lawyer should confirm this rather than accepting a foreign governing-law clause that could complicate PDPL enforcement.
- **Dispute resolution.** Consider whether Saudi courts or arbitration (e.g. under the Saudi Center for Commercial Arbitration) best suits the customer, and ensure the venue is consistent with the residency and confidentiality objectives (an arbitral seat that keeps proceedings confidential may be attractive given the subject matter).
- **Regulatory overlay.** Ensure the agreement is expressly subject to, and does not attempt to contract out of, applicable PDPL and sector-specific Saudi regulatory requirements.

13.10 Consolidated Reviewer's Checklist

#	Consideration	Look for / require
1	Service description	Correct entitlement (app.watheeQ), seats, per-app scope
2	Licensing continuity	Grace-period length; data export retained at expiry/suspension
3	Controller/processor roles	Customer = controller; vendor = processor on documented instructions

4	Data categories & purpose	Counterparty PII + contract/matter content; service-provision purpose only
5	Security schedule	AES-256-GCM at rest (on in prod), TLS in transit, RBAC/SoD, tenant isolation
6	Key custody	`external` (KMS/Vault) custody; in-Kingdom attestation
7	Data residency	Data, backups, keys, audit resident in KSA ; transfers per PDPL
8	Audit immutability	WORM/immutable audit sink if required by posture
9	Sub-processors	Named list; right to object; flow-down; gov-gated connectors opt-in/sandbox-first
10	IP / data ownership	Customer owns its data; no training on customer content ; AI opt-in only
11	Liability	Cap; carve-outs for confidentiality/data-breach/IP
12	Confidentiality	High standard; operator-access limits; privilege non-waiver + compelled-disclosure notice
13	Termination	Breach + convenience rights; no lockout of customer data on lapse
14	Data return	Usable, decrypted export in a defined window
15	Data deletion	Certified deletion incl. backups, subject to legal hold/retention
16	Governing law	Laws of Saudi Arabia ; suitable dispute venue; PDPL not contracted out

13.11 Chapter Summary

A SaaS engagement for the Watheeq Legal Suite is, in substance, an arrangement to have a third party process the customer's most confidential and privileged legal data. The reviewing lawyer's task is to ensure the paper matches the sovereign, PDPL-aligned posture the platform is built for, and to close the gaps that are **configuration or operational** rather than **automatic**: pin down **external key custody with in-Kingdom attestation**, a **WORM-backed audit sink** where required, a **named sub-processor list** with the government-gated connectors treated as opt-in, **customer ownership of data with no model-training rights**, weighted **liability carve-outs** for confidentiality and data-protection failures, **privilege non-waiver**, and — above all — a **data-return and deletion regime** that guarantees the customer can retrieve its own records in usable form and that deletion respects legal holds and statutory retention. The considerations in this chapter are offered to structure that review; the binding terms remain a matter for negotiation and for the customer's own legal judgement.

14. Service Levels & Support Commitments

14.1 Why Service Levels Are a Legal Concern

For most software, service levels are a commercial matter. For a legal system of record, they are partly a legal and regulatory one: the organization's ability to conduct legal business, meet court deadlines, and produce records on demand depends on the platform being available when needed. This chapter sets out the categories of service-level and support commitment a reviewing lawyer should expect to see, and should negotiate, for a Watheeq engagement, and notes where the platform's technical design supports those commitments. As with the terms considerations in Chapter 13, the specific figures are a matter for the individual agreement; the purpose here is to ensure the right commitments are present and appropriately framed.

14.2 Availability

The agreement should state an availability commitment — the percentage of time the platform is operational — measured over a defined period, with defined exclusions (planned maintenance windows, force majeure) and a defined measurement method. Counsel should confirm that planned-maintenance exclusions are bounded and notified in advance, that the measurement method is objective, and that the availability target is appropriate to how the legal department uses the system — a department that files and manages court deadlines through the platform needs a higher assurance of availability than one using it for back-office record-keeping.

The platform's design supports high availability through horizontally scalable services and redundant data stores in its cloud deployment; in an on-premise deployment, availability depends substantially on the customer's own infrastructure, which is a reason the deployment model (Chapter 13) must be settled before availability commitments are finalized.

14.3 Support, Response, and Resolution

Beyond raw availability, the agreement should address support: the channels through which the customer can raise issues, the hours of coverage, and — importantly — the response and resolution commitments graded by severity. A defect that prevents the legal department from meeting a court deadline is not the same as a cosmetic issue, and the support commitments should reflect that with a severity model:

- A means to classify incidents by severity, with the most severe reserved for issues that materially impede legal operations or threaten data.
- Response-time commitments (how quickly the provider acknowledges and begins work) and, for the more severe categories, resolution or workaround commitments.
- An escalation path for issues that are not resolved within the committed times, so that a stalled incident has a defined route to senior attention.

Counsel should ensure these commitments are proportionate to the criticality of the system and that the severity definitions are objective enough to avoid disputes about classification.

14.4 Incident and Breach Handling

Because Watheeq holds personal and privileged data, the support framework must connect to security incident and data breach handling. The agreement should require prompt notification of security incidents affecting the customer's data, cooperation in investigation and remediation, and support for the customer's own regulatory notification obligations under the Personal Data Protection Law. This is the operational counterpart to the breach provisions in the data processing terms (Chapter 13) and should be consistent with them.

15. Business Continuity & Disaster Recovery

15.1 The Legal Weight of Continuity

For a legal department, business continuity and disaster recovery are not merely operational hygiene — they bear on the organization's ability to meet its legal obligations. A legal system of record must be able to survive an infrastructure failure, a ransomware attack, or a data centre loss without losing the records the organization is legally required to keep and may be required to produce. This chapter describes the continuity and recovery posture in terms counsel can assess, and notes the recovery objectives and the integrity guarantees that distinguish a legally adequate recovery capability from a merely technical backup.

15.2 Recovery Objectives

Two objectives frame any recovery capability, and counsel should confirm both are committed to and are appropriate to the organization's needs:

- **Recovery Point Objective (RPO)** — how much data, measured in time, the organization could lose in a disaster. A tight RPO means that even a sudden failure loses only a short window of the most recent work. The platform's disaster recovery design targets a recovery point objective of no more than a few minutes at general availability, with a roadmap toward a much tighter objective, so that the amount of legal work at risk in a failure is small.
- **Recovery Time Objective (RTO)** — how quickly the organization can be operational again after a disaster. The platform's design targets a recovery time objective measured in a small number of minutes, with a roadmap toward tightening it further, so that a disaster is an interruption rather than a prolonged outage.

Counsel should confirm the committed objectives, how they are measured, and — importantly — how they are *demonstrated*, because an untested recovery objective is an assertion, not an assurance.

15.3 Immutable, Clean Recovery

What distinguishes Watheeq's recovery posture, and what matters most for a legal system of record, is the integrity of what is recovered. Two design features are directly relevant to counsel:

- **Immutable recovery points.** The platform's recovery points are written to write-once, read-many storage with object lock protection and per-tenant encryption, and anchored so their integrity can be verified. This means a recovery point cannot be silently altered after it is written — the copy the organization would recover from is protected against tampering in the same way as the records management archive described in Chapter 6.
- **Clean recovery point promotion.** The platform incorporates a control that governs the promotion of a recovery point to a "clean" state suitable for recovery. This is directly aimed at the modern threat that makes ordinary backups inadequate: ransomware. An attacker who encrypts or corrupts data and then waits for that corruption to propagate into backups can defeat a naïve backup regime. A governed clean recovery point gate is designed to ensure the organization recovers from a validated, uncorrupted point rather than from a compromised one,

and a legal hold placed on a protected object survives even privileged deletion until a newer validated recovery point supersedes it — a ransomware² safe floor.

For counsel, the significance is that the platform's recovery capability is built to preserve not just the *existence* of the organization's legal records after a disaster but their *integrity and trustworthiness* — which is precisely what a legal record must retain to remain useful in litigation or before a regulator.

15.4 Sovereign Recovery

Consistent with the platform's overall residency posture, the recovery capability is sovereign: recovery data is subject to the same in²Kingdom residency enforcement as primary data, so a disaster²recovery arrangement does not become a back door through which the organization's legal records leave the Kingdom. Counsel should confirm that the recovery and backup locations are contractually committed to be in²Kingdom, consistent with the technical enforcement.

15.5 Continuity Governance

Finally, counsel should confirm the governance around continuity: that recovery is periodically tested and the results are available to the customer; that roles and responsibilities in a disaster are defined (particularly in on²premise and air²gapped deployments, where the customer bears more of the operational load); and that the continuity commitments are consistent with any regulatory expectations that apply to the organization's sector.

16. Compliance Attestations & Certifications Posture

16.1 An Honest Framing

This chapter describes the platform's compliance **posture** — the controls it is designed to implement and the regulatory frameworks it is built to align with. It is written deliberately to distinguish *designed-for control coverage* from *formal third-party certification*. A reviewing lawyer should treat the two differently: designed-for coverage tells you what the platform does; formal certification tells you that an independent body has attested that it does so. Where a specific certification is required by the organization or its regulator, counsel should request the actual certificate and its scope, and should not infer certification from the presence of the underlying controls. What follows is an honest account of the control coverage, not a claim of certification.

16.2 Personal Data Protection Law (PDPL) Alignment

Watheeq is built to align with the Kingdom's Personal Data Protection Law, and that alignment is implemented as concrete, verifiable controls rather than as a policy statement. The platform's control coverage maps onto the principal PDPL themes:

- **Protection of personal data (encryption).** Sensitive personal data is encrypted at rest with authenticated encryption, and encryption in transit is enforced between components (Chapters 11 and, on transport security, the platform's mutual-TLS and HTTPS posture). Encryption is enforced by policy in protected deployments, not left to operator discretion.
- **Purpose limitation and data minimization.** The platform's egress controls allow data leaving to external integrations to be constrained to a defined set of fields, so that only the data necessary for a given integration leaves the platform.
- **Data residency.** In-Kingdom residency is enforced in code and fails closed, so personal data of Saudi data subjects is kept within the Kingdom as the PDPL contemplates (Chapter 3).
- **Data-subject rights (access and correction).** The platform's structured data model and access controls support the organization's ability to locate, provide, and correct a data subject's personal data.
- **Access control and authorization.** A fine-grained role-based and organizationally-scoped access model, with segregation of duties, governs who may access personal data (Chapters 8–10).
- **Audit and monitoring.** A tamper-evident, hash-chained audit trail records access and change to legal records, supporting the accountability the PDPL expects (Chapter 7).
- **Breach handling.** The platform's incident posture and the contractual breach-notification framework (Chapters 13–14) support the organization's breach-notification obligations.

Counsel should confirm that the organization's specific PDPL obligations — particularly around lawful basis, consent where applicable, and the appointment and role of any data-protection function — are addressed in the organization's own governance, since these are matters of the organization's operation that the platform supports but cannot itself discharge.

16.3 Sector and Framework Alignment

Beyond the PDPL, the platform is built with control families relevant to the frameworks a Saudi organization is likely to be measured against — the national cybersecurity controls promulgated by

the Kingdom's cybersecurity authority, the expectations applicable to regulated financial entities, and the internationally recognized information security control families. The platform's encryption, access control, audit, residency, and continuity controls are the substance behind alignment with these frameworks. Counsel should, however, treat framework **alignment** and framework **certification** as distinct: the former describes control coverage; the latter requires an independent audit against the framework's specific requirements, with a defined scope. Where certification is a procurement requirement, it should be requested and verified directly.

16.4 Evidentiary Support for Compliance

A distinctive strength of Watheeq's compliance posture is that it produces evidence, not just controls. The tamper-evident audit trail, the immutable records archive, the versioned and audited approval policies, and the signature evidence envelopes together mean that the organization can **demonstrate** its compliance — who accessed what, who approved what under whose authority, that records were not altered, and that data stayed in the Kingdom — rather than merely assert it. For an organization that expects to be audited or to defend its conduct before a regulator or a court, this evidentiary capability is often more valuable than any single certificate.

17. Bilingual Glossary & Appendices

17.1 How to Use This Glossary

The following glossary gives the key legal, technical, and compliance terms used throughout this document in English and Arabic, with a short definition, to support the reviewing lawyer and to assist the Arabic translation of this material. Arabic terms are given in Modern Standard Arabic as used in Saudi legal and administrative practice. Where a term has an established statutory or court usage, that usage is preferred.

17.2 Glossary — Legal & Procedural Terms

English	Arabic	Definition
Legal request	طلب قانوني	The canonical unit of legal work that enters the platform and is tracked through its lifecycle.
Legal case	قضية	A litigation matter managed through the two-phase case intake and lifecycle.
Litigation	تقاض	Legal proceedings before a court.
Dispute	نزاع	A disagreement that may lead to litigation or settlement.
Investigation	تحقيق	An internal legal or compliance inquiry managed as its own aggregate.
Consultation	استشارة	A request for legal advice or opinion.
Legal opinion	رأي قانوني	Formal legal advice on a question.
Contract	عقد	A binding agreement managed through the contract lifecycle.
Contract review	مراجعة العقد	The legal examination of a contract before approval and execution.
Settlement	تسوية	The resolution of a dispute by agreement, attached to a legal matter.
Reconciliation	مصالحة	Settlement by amicable agreement; a settlement method.
Mediation	وساطة	Settlement facilitated by a neutral third party.
Arbitration	تحكيم	Binding dispute resolution by an arbitrator.
Matter	ملف / مسألة قانونية	The general container for legal work that is not a specific case.
Beneficiary	المستفيد	The organizational entity for which legal work is performed.
Power of attorney (wakala)	وكالة	A delegated authority to act, issued through the courts portal.
Hearing	جلسة	A court session in a litigation case.
Judgment	حكم	A court's decision in a case.
Pleading	مذكرة / لائحة	A formal written submission in litigation.

Statement of claim	صحيفة الدعوى	The document that initiates a lawsuit.
Court	محكمة	A judicial body; in this platform, integrated via the courts portal.
Ministry of Justice	وزارة العدل	The Saudi ministry operating the courts and the Najiz portal.
Obligation	التزام	A duty arising under a contract, tracked by the platform.
Escalation	تصعيد	The routing of an overdue item to a higher authority.

17.3 Glossary — Governance, Signature & Identity

English	Arabic	Definition
Delegation of authority	تفويض الصلاحيات	The conferral of authority to approve or act, validated cryptographically where required.
Segregation of duties	الفصل بين المهام	The control that prevents one person from performing conflicting roles (e.g., author and approver).
Approval	موافقة / اعتماد	A recorded decision to permit an action, governed by approval policy.
Approval policy	سياسة الاعتماد	The versioned rule set determining who must approve, in what mode and quorum, and under what authority.
Quorum	النصاب	The number or proportion of approvers required for a decision.
Electronic signature	التوقيع الإلكتروني	A signature applied electronically to a legal instrument.
Qualified electronic signature	التوقيع الإلكتروني الموثق	A signature backed by a qualified certificate from a licensed trust service provider, carrying the highest evidentiary weight.
Trust Service Provider (TSP)	مزود خدمات الثقة	A licensed provider (e.g., emdha) that issues qualified certificates and signatures.
Identity verification	التحقق من الهوية	Confirmation of a person's identity (via Nafath) as a basis for signing.
Level of assurance	مستوى التوثيق	The strength of an identity confirmation, gated to a minimum for signing.
Non-repudiation	عدم الإنكار	The property that a signatory cannot credibly deny having signed.
Audit trail	سجل التدقيق	The tamper-evident, chronological record of actions in the platform.
Legal hold	الحجز القانوني	A control that preserves records against deletion for litigation.
Retention period	مدة الاحتفاظ	The period for which records must be kept before disposal.
Records management	إدارة السجلات	The governance of records through their lifecycle to disposition.

17.4 Glossary — Data Protection, Security & Continuity

English	Arabic	Definition
---------	--------	------------

Personal Data Protection Law (PDPL)	نظام حماية البيانات الشخصية	The Kingdom's data protection law governing personal data.
Electronic Transactions Law	نظام التعاملات الإلكترونية	The Saudi law governing electronic transactions and signatures.
Data residency	توطين البيانات	The requirement that data be stored and processed within a jurisdiction.
In Kingdom	داخل المملكة	Located within the Kingdom of Saudi Arabia, as required for residency.
Data controller	المتحكم في البيانات	The party determining the purposes and means of processing (typically the customer).
Data processor	معالج البيانات	The party processing data on the controller's behalf (typically the provider).
Data subject	صاحب البيانات	The individual to whom personal data relates.
Encryption	التشفير	The protection of data by rendering it unintelligible without a key.
Encryption at rest	التشفير أثناء التخزين	Encryption of stored data.
Encryption in transit	التشفير أثناء النقل	Encryption of data moving between components.
Key management	إدارة المفاتيح	The custody, rotation, and control of encryption keys.
Confidentiality	السرية	The protection of information from unauthorized disclosure.
Legal privilege	الامتياز المهني / سرية المحامي وموكله	The protection of certain lawyer-client communications from disclosure.
Access control	التحكم في الوصول	The governance of who may access what.
Tenant isolation	عزل المستأجرين	The separation of one customer's data from another's.
Business continuity	استمرارية الأعمال	The capability to continue operating through disruption.
Disaster recovery	التعافي من الكوارث	The capability to restore operations and data after a disaster.
Recovery Point Objective (RPO)	هدف نقطة الاستعادة	The maximum acceptable data loss, measured in time.
Recovery Time Objective (RTO)	هدف وقت الاستعادة	The maximum acceptable time to restore operations.
Write once, read many (WORM)	الكتابة مرة والقراءة مرات	Storage that cannot be altered after writing, used for immutable records.
Service level agreement (SLA)	اتفاقية مستوى الخدمة	A commitment on service performance (availability, response, resolution).
Hijri calendar	التقويم الهجري	The Islamic lunar calendar (Umm al-Qura) used for Saudi dates.
Working calendar	تقويم أيام العمل	The Saudi working week and holiday profile used for deadlines.

17.5 Appendix A — Government & Trust Integrations

Integration	Role	Status posture
Najiz (Ministry of Justice, via Takamul)	Court hearing/case synchronization; issuance of powers of attorney	Built and hardened; operates against sandbox transport until live government credentials are provisioned
Nafath (National Single Sign-On)	Identity confirmation of signatories, to a minimum assurance level	Built and hardened; sandbox until production service-provider credentials are provisioned
emdha (NCDC-licensed Trust Service Provider)	Qualified electronic signatures under the Electronic Transactions Law	Built and hardened; sandbox until live trust-service credentials are provisioned

In each case, the platform is honest about status: it does not report a live government connection it does not have, and its sandbox transports are explicitly labelled and never presented as live signatures or live court reconciliations.

17.6 Appendix B — Records Retention Summary

Record category	Retention posture
Legal documents in the e-archive	Write-once, read-many, with a records-management default retention measured in years; legal hold available
Audit records	Append-only and immutable; tamper-evident hash chain; retained per the organization's audit-retention policy
Disaster-recovery journal	Rolling retention with immutable, integrity-anchored recovery points

17.7 Appendix C — Legal Roles (Reference)

The platform recognizes a fourteen-role legal hierarchy spanning the requesting business, the legal function, an oversight layer, and system administration. The roles and their access are described in the companion Features document and in Chapters 8–10 of this document. The roles carry first-class Arabic names and are the basis of the platform's need-to-know access model.

17.8 Closing Note for Counsel

This document has described Watheeq's legal, compliance, and data-protection posture as it is built: residency enforced in code, sensitive data encrypted by policy, access confined and segregated, records immutable and tamper-evident, signatures capable of qualified evidentiary weight, and recovery designed to preserve integrity as well as existence. The recurring recommendation throughout has been that counsel confirm the platform's technical assurances against the organization's own legal obligations and convert them into binding contractual commitments. The platform is designed to give counsel strong ground on which to do so.